



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Julia for Joint Models

Master Thesis

Yannik Ammann

August, 2023

Advisors: Prof. Peter Bühlmann

Supervisors at Roche: Dr. Daniel Sabanes Bove, Dr. Francois Mercier, Dr. Doug Kelkhoff

SfS, Department of Mathematics, ETH Zürich

Abstract

This work implements joint models for a longitudinal process influencing a time-to-event process in Julia. The goal was to develop a tool and workflow to allow researchers to implement their joint models and iterate and explore them. Another goal was to evaluate the Julia programming language and ecosystem concerning the development of statistical tools. I have developed workflows showing how to implement survival models, mixed-effects longitudinal models, and joint models in Julia. In addition, I have developed and described how to generate diagnostic statistics and plots for these models. These workflows show the simplicity and capabilities of the Julia language. One particular achievement is the close mapping of mathematical and statistical ideas to their implementation in code.

Contents

Contents	iii
1 Introduction	1
1.1 Motivation	1
1.2 Bayesian modeling and Turing.jl	2
2 Mathematical Formulations	5
2.1 Survival Analysis	5
2.2 Longitudinal process	8
2.3 Joint Models	10
3 Methods and Design	13
3.1 Towards implementation of Joint Models	13
3.2 Case Study: Simulated Data	15
3.3 Diagnostics	17
3.4 Comparison to Stan	21
4 Implementation of Joint Models	25
4.1 Type: HazardBasedDistribution	25
4.2 Usage	27
5 Conclusion	31
A Appendix	33
A.1 Further Exploration	33
A.2 Survival Workflow	33
A.3 Longitudinal Workflow	71
A.4 Joint Molder Workflow	93
A.5 Julia in Latex	132
Bibliography	133

Chapter 1

Introduction

This is an exploratory project to evaluate the Julia programming language and its ecosystem. Julia is a relatively new programming language released in 2017 [4] with the development starting in 2012.

To judge the development process in Julia and to test the tools available I implemented joint models. This is a complicated model with applications in clinical statistics. In the next sections I will give some background and motivation for the research question and give a glimpse into Bayesian modeling.

1.1 Motivation

Joint models describe the relationship between longitudinal measurements and time-to-event survival outcomes¹. In pharmaceutical research joint models can be used to assess the efficacy of novel treatments, optimize patient care strategies by individual prediction of outcomes or explore disease progression mechanisms. Joint models offer a valuable tool to gain insights from complex data by modeling the longitudinal process and patient survival through the joining of two separate models with an additional linking function.

analyzing longitudinal processes and survival times leading to more informed decision-making and improved patient outcomes [17].

The difficulty of joint models is the combination of measurements of a continuous process and survival outcomes. Available packages for joint models such as JM [19] and JMBayes [20] in the R programming language are not capable to describe non-linear longitudinal models. One achievement of this work is to implement the joint models described by Kerioui, Mercier, Bertrand et al. [17], which features a non-linear longitudinal mixed-effects model. Gower-Page, Kazantzidis, Sabanes Bove et al. started the development of an alternative

¹Chapter 2 explains these processes.

R-package called `jmpost` [11], which allows the usage of non-linear longitudinal models.

The R-package `jmpost` uses the C++ library Stan [22] and R bindings [23], which is an example of the so-called two-language problem [21]. One programming language is used for efficient algorithms and another for the researcher developing and iterating models. This makes the implementation inherently complex and developers and researchers have to be proficient in multiple programming languages, for example R and C++/Stan to push the boundaries of knowledge. This results in inefficiencies in development and debugging because of fragmented code in different languages. Additionally, flexibility to iterate and explore joint models is lacking.

To solve the two language problem and still be able to formulate non-linear joint models and sample efficiently, we want to explore the Julia programming language. Julia solves the two-language problem by being efficient with a just in time compiler, which turns Julia code into native machine code. In addition, Julia has the ecosystem for developing complex statistical tools such as joint models in an intuitive and flexible manner. The goal is to enable researchers to deploy and refine joint models with the highest efficiency and clarity to unravel the complicated relationships between survival and longitudinal observations.

1.2 Bayesian modeling and Turing.jl

Bayesian modeling combines the prior beliefs about parameters of a model with the available data. We update our beliefs based on the observed data resulting in a posterior distribution characterizing the uncertainty about the parameters. Using the posterior distribution, we can make predictions or analyze new data.

Bayes theorem states

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

For Bayesian inference we take A to represent the model parameters and B to represent the observations. Meaning A contains the parameters and formulation of a model that tries to describe the measurements B .

This makes $P(A)$ the prior beliefs of the parameters of a model and $P(A|B)$ we call the posterior distribution of the model parameters given the observations B . Probability $P(B|A)$ describes the likelihood of measurements B given model parameters A .

Finally, $P(B)$ is the probability of measurements. This is generally unknown. Thus Bayesian inference relies on the proportion

$$P(A|B) \propto P(B|A)P(A).$$

In chapter 2 I explicitly calculate the likelihood for joint models, since this is what we need to implement joint models with Bayesian inference.

In Julia Turing.jl [10] has established itself as a popular probabilistic programming framework for implementing Bayesian sampling algorithms. In Appendix A I provide workflows showcasing how to work with framework.

Chapter 2

Mathematical Formulations

In this chapter I present the formulation of joint models and calculate its log likelihood which is a key component in Bayesian sampling algorithms [10]. For this we first look at survival and longitudinal models separately before joining them via a link to build joint models.

2.1 Survival Analysis

In survival analysis we consider a random variable T which is non negative $T \geq 0$ and we are interested in its distribution function. The random variable T describes the time to an event which is a point of interest. In pharmaceutical survival models, these events could be an adverse reaction to a drug, death or other events of interest [6].

For such a non negative random variable T we define the corresponding cumulative distribution function (CDF) F

$$F : \mathbb{R}_{\geq 0} \rightarrow [0, 1] \quad F(t) \quad := P(T \leq t),$$

as well as probability density function (PDF) $f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ via the integral

$$\int_0^t f(u) du = F(t) = P(T \leq t).$$

In survival analysis we are especially interested in the complementary cumulative distribution function (CCDF) $F^c(t) := 1 - F(t) = P(T > t)$, which we will call survival function S

$$\begin{aligned} S : \mathbb{R}_{\geq 0} &\rightarrow [0, 1] \\ S(t) &:= P(T > t), \end{aligned}$$

2. MATHEMATICAL FORMULATIONS

since it represents the probability that no event occurred until time t i.e. survival until time t .

Furthermore, we formulate a hazard function $h : \text{supp}(S)^1 \subseteq \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ to be the conditional probability of an event occurring at time t with the condition that the event did not occur until time t

$$\begin{aligned}
 h(t) &:= \lim_{\delta \rightarrow 0^+} \frac{P(t \leq T \leq t + \delta | T > t)}{\delta} \\
 &= \lim_{\delta \rightarrow 0^+} \frac{1}{\delta} \left(\frac{P(t \leq T \leq t + \delta)}{P(T > t)} \right) \quad (\text{conditional probability}) \\
 &= \lim_{\delta \rightarrow 0^+} \frac{1}{\delta} \left(\frac{F(t + \delta) - F(t)}{S(t)} \right) \\
 &= \lim_{\delta \rightarrow 0^+} \underbrace{\left(\frac{F(t + \delta) - F(t)}{\delta} \right)}_{\text{derivative: } F'(t)=f(t)} \frac{1}{S(t)} \\
 &= \frac{f(t)}{S(t)}.
 \end{aligned}$$

The hazard function can be used to formulate the PDF and Survival function and thus describe a distribution. Let us try to derive the survival function S from the hazard. Notice that

$$f(t) = \frac{d}{dt}F(t) = \frac{d}{dt}(1 - S(t)) = -\frac{d}{dt}S(t),$$

thus

$$h(t) = \frac{f(t)}{S(t)} = \frac{-\frac{d}{dt}S(t)}{S(t)} = -\frac{d}{dt} \log S(t).$$

Formulating the cumulative hazard function $H(t) : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ and plugging in the hazard formulation above

$$H(t) := \int_0^t h(u) du = \int_0^t -\frac{d}{du} \log S(u) du = -\log S(t).$$

Solving for S we get

$$S(t) = \exp(-H(t)) = \exp\left(-\int_0^t h(u) du\right).$$

¹We need the support of S for the conditional probability. Note that the support of S is an interval since S is monotonically decreasing and $S(0) = 1$.

Finally we get a formulation of the probability density function f based on the hazard

$$h(t) = \frac{f(t)}{S(t)} = \frac{f(t)}{\exp\left(-\int_0^t h(u) du\right)}$$

$$f(t) = h(t) \exp\left(-\int_0^t h(u) du\right).$$

Later on in the section about joint models 2.3 we will alter the formulation of the hazard and thus of S and f based upon a longitudinal process.

Censoring and likelihood

When measuring processes in the real world, sometimes you do not measure an event and only record the absence of an event at a given time. Censoring describes how to handle these situations. Here we will focus on so-call "right"-censoring [6].

Suppose we have a time to event random variable T and a realization $\bar{t}$ as well as a censoring time t_c , we call an event right-censored if $t_c < \bar{t}$. When taking measurements we can record a time

$$t = \min\{t_c, \bar{t}\},$$

as well as an event indicator²

$$\delta = \begin{cases} 1 & \bar{t} < t_c \\ 0 & \bar{t} \geq t_c. \end{cases}$$

Leaving us with a measurement³ (t, δ) .

What censoring describes is how to calculate the likelihood for such a measurement. In right censoring we record the censored time as "survived". The likelihood of a particular hazard which defines the PDF f and the survival function S . The likelihood of the event (t, δ) given the distribution described by the hazard h is calculated as

$$L((t, \delta)|h) = f(t)^\delta \cdot S(t)^{1-\delta} = \begin{cases} f(t) & \bar{t} < t_c \quad (\text{event time is recorded}) \\ S(t) & \bar{t} \geq t_c \quad (\text{event time is not recorded}). \end{cases}$$

The argumentation for this case distinction is that for censored events $\delta = 0$ we know that no event occurred until time t . We can say that there is survival until time t , meaning the absence of an event. This is described by the CCDF $S(t)$.

²Denoting an event is **not** censored.

³This is specific to right-censoring, other methods can have different measurement structures

Covariates

To incorporate a covariates vector $x \in \mathbb{R}^l, l \in \mathbb{N}$ we can use the exponential function. This is called the Cox proportional-hazard model based on [7]

$$h(t|x) = h_0(t) \exp(\beta \cdot x)$$

Here $\beta \in \mathbb{R}^l$ are the covariate coefficients and h_0 is the baseline hazard that could be a parametric distribution⁴.

One reason for using an exponential is the possibility of comparing the proportionality of covariates x_1 and x_2 by

$$\begin{aligned} \frac{h(t|x_1)}{h(t|x_2)} &= \frac{h_0(t) \exp(\beta \cdot x_1)}{h_0(t) \exp(\beta \cdot x_2)} \\ &= \frac{\exp(\beta \cdot x_1)}{\exp(\beta \cdot x_2)} \\ &= \exp(\beta \cdot (x_1 - x_2)). \end{aligned}$$

The exponential in the last line contains the linear combination of the covariates which helps with their interpretation.

2.2 Longitudinal process

The longitudinal process describes measurements at different time points that are connected with each other. In clinical statistics these could belong to the same individual, measuring biomarkers such as tumor growth, test results or molecule concentration in blood.

Another complexity of the longitudinal process is that we are working with mixed effects i.e. hierarchical models. In the following we will refer to individuals to separate multiple longitudinal processes. This is specific to clinical statistics and could be abstracted into separate processes with the same underlying model structure.

Assume we have $N \in \mathbb{N}$ individuals and each individual $i \in [N]$ has $n_i \in \mathbb{N}$ measurements. For simplicity we consider only one such longitudinal process in a scalar space and do not consider covariates. The set of measurements we describe as $\{y_{ij}\}_{i \in [N], j \in [n_i]}$, where i denotes the individual and j the measurement number for this individual. For each measurement, we also have an associated time $\{t_{ij}\}_{i \in [N], j \in [n_i]}$ at which the measurement was taken.

⁴Exponential, Weibull, Log-Logistic, Log-Normal are examples [6].

A longitudinal model $m : T \times P \rightarrow \mathbb{R}$ describes a function of time $T := [0, \infty)$ and a parameter space $P := \mathbb{R}^l$ for some $l \in \mathbb{N}$. For every individual⁵ $i \in [N]$ we have a parameter ψ_i . We define $m_i : T \rightarrow \mathbb{R}, m_i(t) := m(t, \psi_i)$ as the individual longitudinal model for the individual i .

Notice that in this formulation, we do not assume linearity as is done in previous R implementations of joint models, namely JM [19] and JMBayes[20]. This general formulation allows non-linear mixed effects models such as in the paper by Keroui et al. [17] to be used.

Likelihood

To formulate a statistical model, we include an error process⁶. This gives us a probability density function $f_{\text{err}}(y_{ij})$ for any measurement y_{ij} . These error process is described by parameters which we include in the parametric description of the longitudinal model.

To calculate the likelihood, we consider a set $\mathcal{M} := \{m_i\}_{i \in [N]}$ of individual models. This includes the parameters for the longitudinal prediction model and for the measurement error. As well as the set of measurements with associated time points $Y_L = \{(t_{ij}, y_{ij})\}_{i \in [N], j \in [n_i]}$.

For a single measurement (y_{ij}, t_{ij}) we calculate the likelihood as

$$L((y_{ij}, t_{ij}) | \mathcal{M}) = L((y_{ij}, t_{ij}) | m_i) = f_{\text{err}}(y_{ij})(m_i(t_{ij})).$$

This extends to the log likelihood formulation for all measurements

$$\begin{aligned} l(\{y_{ij}\}_{i \in [N], j \in [n_i]} | \mathcal{M}) &= \log L(\{y_{ij}\}_{i \in [N], j \in [n_i]} | \mathcal{M}) \\ &= \log \left(\prod_{i \in [N]} \prod_{j \in [n_i]} L(y_{ij} | m_i) \right) \quad (\text{this assumes independence}) \\ &= \sum_{i \in [N]} \sum_{j \in [n_i]} \log L(y_{ij} | m_i) \\ &= \sum_{i \in [N]} \sum_{j \in [n_i]} \log f_{\text{err}}(y_{ij})(m_i(t_{ij})) \\ &= \sum_{\substack{i \in [N] \\ j \in [n_i]}} \log f_{\text{err}}(y_{ij})(m_i(t_{ij})). \end{aligned}$$

The independence assumption mentioned here is quite strong since we are talking about a time series of measured tumor size, which under the assumption that the tumor either grows or shrinks is necessarily autocorrelated. This makes such measurement generally dependent.

⁵Mixed effects models can be described by this formulation.

⁶Examples are additive or multiplicative errors under normal distributions.

2.3 Joint Models

Joint models incorporate a longitudinal process into a survival process [17]. This is done by augmenting the hazard function of the survival model using the longitudinal process.

Suppose we have a longitudinal model m , with individual realizations m_i for individuals $i \in [N]$ as described in 2.2 a and any distribution for the survival process. We call the hazard of this distribution h_0 the baseline hazard. Similarly there is a baseline PDF and survival function.

The joint model is the distribution after we augment the hazard function. One very common way [17] to augment this hazard is by multiplying an exponential containing a "link"⁷ $g(m_i(t))$ to the longitudinal process inside an exponential function. This gives us a different hazard formulation for all individuals $i \in [N]$ which depend on longitudinal models.

$$h_i(t) = h_0(t) \exp(\beta \cdot g(m_i(t)))$$

Previous implementations have joined these models by including the link function as part of an exponential term similar to the Cox proportional hazard model [7] and the benefits shown here 2.1. This work will focus on implementing the established formulation. It should be noted that other means of augmenting the hazard formula would be possible.

The hazard formulation could also be extended by incorporating other links to the same or different longitudinal processes or covariates for the individuals.

Likelihood

From these individual hazard formulations we can derive the formulations of the individual PDF f_i and survival S_i according to survival analysis 2.1. Furthermore, we can calculate the likelihood that an event will occur for censored or uncensored events according to 2.1.

Suppose $\theta = (\theta_s, \theta_b, \theta_l)$ fully describes a parametric joint model. In particular, θ_s describes the baseline survival distribution, θ_b the link, and θ_l describes the longitudinal model $\{m_i\}_{i \in [N]}$ with its error model. And we have survival $Y_S = ((t_i, \delta_i))_{i \in [N]}$ and longitudinal $Y_L = \{(t_{ij}, y_{ij})_{i \in [N], j \in [n_i]}$ measurements. Notice that we are working with two time variables t_i for survival times and t_{ij} for longitudinal measurements. Given the parameters the log likelihood of the

⁷In literature often denoted by f but here I choose g to separate its name from the probability density function.

joint model can be calculated as follows:

$$\begin{aligned}
l(Y_S, Y_L | \theta) &= \log(L(Y_S | \theta) L(Y_L | \theta_l)) \\
&= \log \left(\prod_{i \in [N]} L((t_i, \delta_i) | \theta) \prod_{i \in [N], j \in [n_i]} L((t_{ij}, y_{ij}) | \theta_l) \right) \\
&= \sum_{i \in [N]} \log L((t_i, \delta_i) | \theta) + \sum_{i \in [N], j \in [n_i]} \log L((t_{ij}, y_{ij}) | \theta_l) \\
&= \sum_{i \in [N]} \log(f_i(t_i)^{\delta_i} \cdot S_i(t_i)^{1-\delta_i}) + \sum_{i \in [N], j \in [n_i]} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))) \\
&= \sum_{(t_i, \delta_i) \in Y_S} \log(f_i(t_i)^{\delta_i} \cdot S_i(t_i)^{1-\delta_i}) + \sum_{(t_{ij}, y_{ij}) \in Y_L} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))) \\
&= \sum_{(t_i, \delta_i) \in Y_S} \delta_i \cdot \log(f_i(t_i)) + (1 - \delta_i) \log(S_i(t_i)) + \sum_{(t_{ij}, y_{ij}) \in Y_L} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))) \\
&= \sum_{(t_i, \delta_i) \in Y_S} \delta_i \cdot \log(f_i(t_i)) - (1 - \delta_i) H_i(t_i) + \sum_{(t_{ij}, y_{ij}) \in Y_L} \log(f_{\text{err}(y_{ij})}(m_i(t_{ij}))).
\end{aligned}$$

The log-likelihood of joint models is given as the sum of the survival and longitudinal measurements, where the survival log-likelihood is described by the censoring process 2.1 and the longitudinal log-likelihood has an error model as described in 2.2.

These calculations are the basis for the implementation of joint models. The log-likelihood calculations are needed inside the Turing framework for Bayesian sampling algorithms [10].

Link functions To fully understand such joint models, we need to examine the link functions g considered in the literature. Kerioui et al. [17] mention the identity link id , the derivative $\frac{d}{dt}$ and the time to growth (TTG) as a suitable link function for their work. The TTG is domain specific to their longitudinal model.

$$\begin{aligned}
g_{id}(m_i(t)) &= m_i(t) \\
g_{\text{derivative}}(m_i(t)) &= \frac{d}{dt} m_i(t) \\
g_{\text{TTG}}(m_i(t)) &= \text{TTG}(m_i) \quad (\text{constant function in time})
\end{aligned}$$

Here we note that the link function g is not a function on $\mathbb{R}$ but the function space containing the longitudinal models m_i . And in case of Kerioui et al. [17] it might be that we do not know g_{TTG} itself, only the image $g_{\text{TTG}}(m_i(t))$ for a particular longitudinal model formulation⁸.

⁸They specify g_{TTG} for their longitudinal model SLD

Chapter 3

Methods and Design

I made interesting findings concerning the implementation of joint models in Julia. Please, take a look at the workflows in the Appendix A. They sum up the tools I have developed myself and all the packages I have found useful in the Julia/Turing ecosystem. The following sections summarize my thoughts and results, but generally do not contain code or data.

3.1 Towards implementation of Joint Models

To start with this project I implemented a survival model A.2 and longitudinal model A.3 separately.

For the survival process, the main difficulty for the implementation in Turing is the censoring. At first, I used a trick to access the survival probability by wrapping the survival function (ccdf) S_i inside a Bernoulli distribution:

```
Bernoulli(ccdf(Weibull( $\alpha$ ,  $\theta$ ), time_obs[i]))
```

This allows access to the survival function to calculate the log likelihood for a censored event.

Toward the end of my project, I found the censored functionality provided by the Distributions.jl package [3]. The reason I did not consider this earlier is that for a major part of the project duration I added the log likelihood manually.

Implementing the longitudinal process was straightforward, since Turing allows us to call standard Julia code inside the domain-specific probabilistic programming language. We can write our own model function and then use it inside the probabilistic programming language Turing. The main challenge here was to understand the mixed effects and how to built them. In the section on the mathematical formulation of the longitudinal process 2.2 I abstract this complexity by giving each individual a different model realization.

In the Appendix A.3 there are two examples of longitudinal mixed effects models with different parameter setups. In the end, all such formulations end with a vector of parameters per individual. Implementing these mixed-effects priors is straightforward, after understanding their mathematical description.

First Joint Model To implementing joint models we calculate both the likelihood for longitudinal and survival data. For the survival data, we cannot work with parametric¹ survival distributions anymore. The Turing.jl framework allows us access to the likelihood calculation, specifically the log likelihood via the function `Turing.@addlogprob!()`. We can manually specify the log likelihood according to our calculations in 2.3 for the survival data. To calculate the integral in the cumulative hazard, I used a Gauss-Konrad numerical integration procedure provided by QuadGK.jl [16]. This package was originally part of the Julia language, but has been made a standalone package.

After having first implemented a joint model, two functional limitations of that early implementation arose. First, some advanced diagnostic tools did not work. One example is the leave-one-out cross-validation (LOO-CV) through Pareto smoothed importance sampling² [24] provided by ParetoSmooth.jl [5]. This is because we lose the connection between the observation and likelihood contribution for the survival observations inside the framework when adding the log likelihood manually.

Another limitation was that predictions based on the posterior samples cannot be generated inside the Turing framework using the manual log-likelihood contributions. There is a `predict()` function in the Turing framework that uses the model formulation and posterior samples [10] to generate posterior predictions. For each posterior sample, it instantiates the model with these values and generates sample observations. But this uses the `rand()` functionality implemented by distributions inside the Distributions.jl package [3]. If we add the log probability ourselves we do not provide this functionality.

Custom Distribution The root of these two problems is that now the Turing framework does not have a distribution associated with the survival data. The solution I found is to implement a distribution type for joint models. More generally I developed a distribution type for distributions based on the hazard formulation. See Chapter 4 for a complete documentation.

I created an abstract type `HazardBasedDistribution`. This type is capable of computing the survival and probability density function based on a given hazard formulation³, which allows us to implement the log-likelihood contributions for censored data. In Turing, we can use structs of this type with the Turing

¹Distributions with names such as Weibull, Normal etc.

²State of the art diagnostic for Bayesian model development.

³According to their mathematical formulations presented in chapter 2.

tilde syntax to denote likelihood calculations A.2. Which allows us to calculate the LOO-CV statistic mentioned above.

To make predictions using the Turing framework I implemented the `rand()` function for my type `HazardBasedDistribution`. I applied the inverse transform sampling, also described in section 4.1. This allows us to generate event data based on the posterior of a Bayesian sampling which allows us to generate diagnostics such as posterior predictive checks.

Design Given the development of the project the design for a user interface for implementing joint models changed quite drastically to guarantee the functionalities described above. I came up with ideas such as using Turing sub-models [10] or extending the `@formula` macro from StatsModels.jl [8].

In the end, the necessity of creating my own distribution gave me a foundation to develop a cohesive interface. The distribution hides the calculations and presents the models relevant choices A.4 such as the baseline model or the link function. These need to be interchangeable. Therefore, a flexible interface for them is crucial.

The design goals for the interface I set up⁴ are:

- Intuitive and direct access to prior distributions.
- Simple but explicit formulation of the likelihood for survival and longitudinal processes.
- New distributions based on the hazard formulation should be simple to implement. In particular, this allows one to implement joint models.
- Lightweight structure around the Turing framework to minimize maintenance work and the complexity⁵.

3.2 Case Study: Simulated Data

After implementing a toy example the model I used as the primary use case and target for development was the joint model described by Kerioui et al. [17]. My implementation can be found in A.4. It also explains the details of this joint model. I will not go into details in this section and just showcase some parts.

This specific joint model is interesting because its longitudinal model $SLD(t, \Psi)$ is quite complex and non-linear. We have $\Psi = (BSLD, g, d, \varphi) \in \mathbb{R}^4$, which

⁴These can be altered with feedback from users who implemented joint models using `HazardBasedDistribution`.

⁵The framework first transforms the `@model` function. It is not a function but rather an description of a Bayesian model in Turing [10].

3. METHODS AND DESIGN

describes the mixed effects for an individual.

$$\text{SLD}(t, \Psi) = \begin{cases} \text{BSLD} \exp(gt) & \text{if } t < t_x \\ \text{BSLD} \exp(gt_x) \cdot [\varphi \exp(-d(t - t_x)) + (1 - \varphi) \exp(g(t - t_x))] & \text{if } t \geq t_x \end{cases}$$

Every individual has a different Ψ vector that is built by population and random effects. More details about the priors can be found in the Appendix A.4 or in the paper.

Accompanying the paper [17] Kerioui et al. provided a simulated data set with 655 total measurements of 100 individuals. This data was generated according to their model with given parameters.

Parameter	Value
μ_{BSLD}	60
μ_d	0.0055
μ_g	0.0015
μ_φ	0.2
σ	0.18
κ	1
λ	1450
β	0.01

To test my implementation of their model I am comparing my posterior samples to the parameters they used to generate the data. Here I am sampling 200 samples after 100 burn in samples using the NUTS [12] algorithm provided by the Turing framework. The statistics about the posterior samples:

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat
Symbol	Float64	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	61.0555	4.6970	0.5009	89.9910	89.3489	1.0195
μ_d	0.0058	0.0011	0.0001	116.4744	160.1451	1.0248
μ_g	0.0018	0.0002	0.0001	6.5780	104.2091	1.1427
μ_φ	0.1587	0.0581	0.0438	1.8077	28.8366	1.6211
σ	0.1763	0.0064	0.0006	121.8497	151.1699	1.0078
κ	1.0866	0.0750	0.0152	23.0509	55.6639	1.0130
λ	1359.2824	265.8978	22.9817	151.1063	178.6288	1.0031
β	0.0102	0.0010	0.0001	253.7573	152.8143	1.0045

Table 3.1: This statistics is generated by the Turing framework after sampling posterior samples. The rows describe the parameters for which posterior samples were generated. The mean and std describe the mean value and standard deviations of the posterior samples. The mcse estimates the Monte carlo mean squared error. The ess estimates the effective sample size and rhat estimates the $\hat{R}$ statistics commonly used in Bayesian statistics.

Most of the mean values of the posterior samples are relatively close to the parameters used to generate the data. An exception is the μ_ϕ parameters, which are off by quite a bit. We can see in the statistics about the posterior that its rhat value is high 1.6 and the mean squared error, which is high in relation to the value of μ_ϕ . To get better posterior distributions we can increase the burn in duration and the sample size of the posterior. This will increase the sampling duration which is a trade-off.

The output of the sampling call is not just statistics of the posterior but we also have access to the actual samples that were generated. Using them we can generate diagnostic plots. See 3.1 as an example of an individual diagnostic plot.

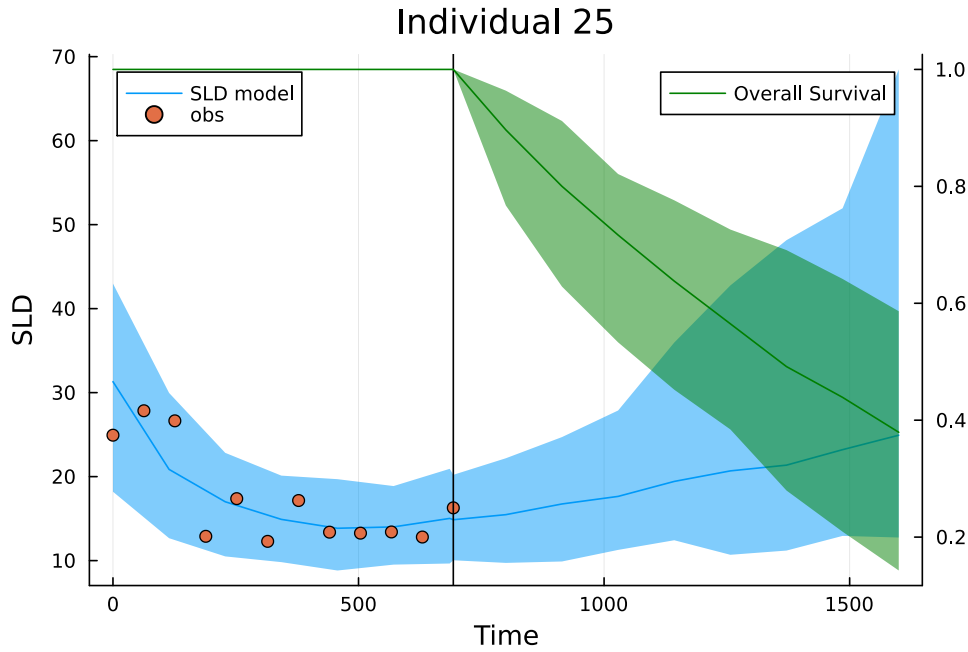


Figure 3.1: Non-parametric posterior predictive plot I made for joint models. Individual 25 is the first censored individual in data set. In blue is the posterior predictive 95% quantile of the longitudinal model and the line is the median while the orange dots are the measured values. In green is the posterior predictive 95% quantile of the survival via Kaplan-Meier estimators. See A.2 for a detailed explanation.

3.3 Diagnostics

While creating the workflows, I have developed many diagnostic plots. There are two categories of plots. One are the posterior predictive checks, where we use the Turing framework to generate sample measurements based on the

posterior distribution⁶. The other plots analyze specific values such as the cumulative hazard for Cox-Snell and Martingale residuals for survival models and the individual predictions for longitudinal models.

DiagnosticHelper / Dirac To save values such as the cumulative hazard $H(t)$ or individual predictions (IPRED) during the Turing sampling call I developed a helper distribution `DiagnosticHelper`. It takes a value α and when sampled it returns this value α . In other words the `DiagnosticHelper` represents a distribution where every sample is α .

Using this, we can save the cumulative hazard or IPRED into the output of the Bayesian sampling of the posterior. See the application in A.2 A.3.

When thinking about the name to give to this distribution, I realized that it is isomorphic to the Dirac distribution [9]. The distribution $\text{Dirac}(\alpha)$ has cumulative probability density function

$$F_{\alpha}(x) = \begin{cases} 1 & x \geq \alpha \\ 0 & x < \alpha \end{cases}.$$

Sampling a value from this distribution always gives α .

The package `Distributions.jl` [3] already has the Dirac distribution implemented. Therefore my `DiagnosticHelper` is replaced by this Dirac implementation. The functionality to save generated values is an important tool to analyze our models. I applied this method to generate Cox-Snell, Martingale, IPRED vs OBs and IRES vs covariates plots.

Posterior Predictive Check After sampling the posterior distribution of a model we can use the posterior distribution to sample observation values. The posterior predictive check allows us to detect if our model is not capable to describe certain observations and it allows one to compare different model implementations.

In the case of a mixed-effects longitudinal model, we can specify the time points at which we want to generate samples⁷. In the output of the Turing function `predict`, we predict a value for each individual and each time point, for each sample in the posterior. See the Appendix for more details A.3.

For each individual this allows us to calculate the 95% quantile at the specified time points as well as the mean, median or mode. We can then plot these values alongside the observed data. See 3.2 for an example plot.

⁶The posterior distribution was previously sampled with a Bayesian sampling algorithm from the Turing framework.

⁷Note that the measurement error is included here.

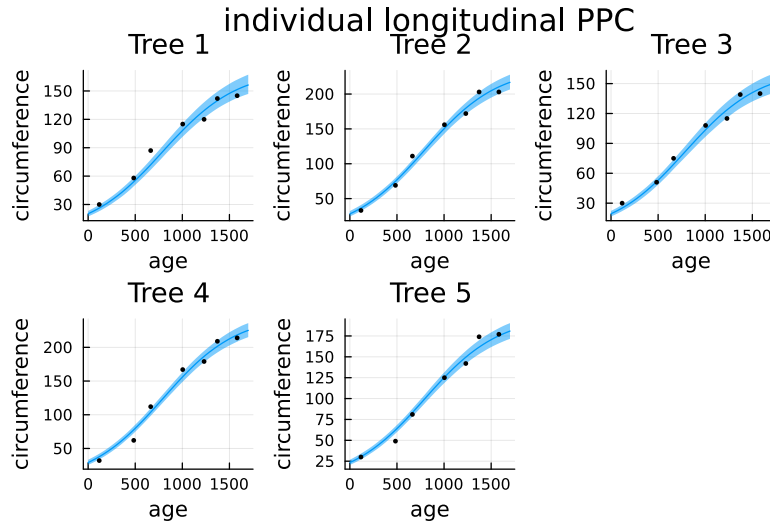


Figure 3.2: Posterior predictive check for a longitudinal model, which describes the tree circumference over time. See A.2. The black dots are the observations while the blue band represents the 95% quantile of the posterior predictive for each tree.

For survival models the predicted event times according to the posterior are not in the preferred format. We want to view the survival function and not the events themselves. For this we can use a non-parametric estimator of the survival function, such as the Kaplan-Meier estimator provided by the `Survival.jl` package [2]. See 3.3 for an example plot and the Appendix A.2 for the implementations.

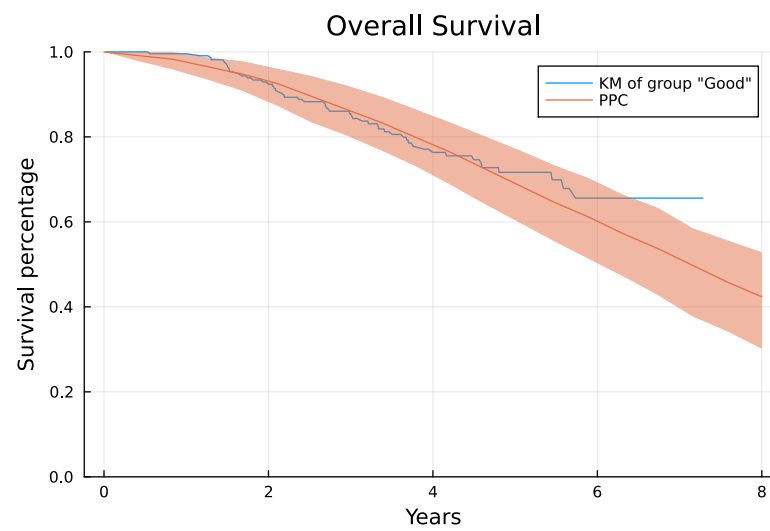


Figure 3.3: Posterior predictive check for a survival model A.2 First we calculate a Kaplan-Meier estimator for the observation data. Then we calculate a Kaplan-Meier estimator for each of the posterior predictions. From these we calculate the 95% quantile interval at different time points.

3.4 Comparison to Stan

To evaluate my implementation we are consider the joint model specified in the paper by Kerioui et al. [17] and the simulated data provided with this paper. The other implementations we are consider are the original code attached with the paper and the package `jmpost` [11] that is currently under development.

First, I compare the posterior statistics produced by the original code and my implementation as an argument for a degree of confidence about the correctness of implementation. Afterwards, I will compare the interface and functionality to the `jmpost` package in development.

Numerical Comparison I will compare the sampling output of my implementation vs the implementation done by Kerioui et al. [17]. For the sampler we use the NUTS [12] sampler with 200 warm-up samples and a target acceptance rate of 0.9. In the posterior, we sample 400 samples. We use the same prior distributions in both implementations and supply initial values for the sampling algorithm according to their specification in [17]. The table 3.2 are the statistics about the posterior sampled with the R/Stan code provided along with the paper.

	mean	se_mean	sd	n_eff	Rhat
mu_1	62.26464	0.61117	4.48100	54	1.03979
mu_2	0.00487	0.00024	0.00197	65	1.06537
mu_3	0.00181	0.00007	0.00029	20	1.15857
mu_4	0.20024	0.01110	0.08510	59	1.07103
Omega[1]	0.73227	0.00650	0.05196	64	1.07858
Omega[2]	0.56993	0.05826	0.43380	55	1.07908
Omega[3]	0.83742	0.01418	0.11937	71	1.04076
Omega[4]	1.54100	0.02664	0.35360	176	1.00189
sigma	0.17672	0.00026	0.00602	545	1.00097
lambda	1505.44167	13.41165	339.75674	642	0.99912
beta	0.01032	0.00005	0.00134	652	1.00179

Table 3.2: This statistics is generated by the Stan framework [22]. Similar to the Turing output the rows represents model parameters and the columns certain statistics.

And the statistics about the posterior generated with Julia code similar to the one found in the Appendix A.4 in table 3.3.

3. METHODS AND DESIGN

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat
Symbol	Float64	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	61.2455	4.4382	0.6618	44.3845	116.2119	1.0013
μ_d	0.0055	0.0014	0.0001	150.5143	199.8976	1.0042
μ_g	0.0018	0.0002	0.0000	104.7299	59.5683	1.0681
μ_ϕ	0.1839	0.0605	0.0057	108.7283	232.6829	1.0040
ω_{BSLD}	0.7378	0.0573	0.0021	701.9017	285.8221	1.0137
ω_d	0.3100	0.1598	0.0555	5.9030	34.0874	1.1270
ω_g	0.8736	0.1071	0.0096	123.3917	324.7381	1.0035
ω_ϕ	1.4668	0.2676	0.0308	74.4418	125.7581	1.0259
σ	0.1772	0.0064	0.0004	282.0313	249.8288	0.9989
λ	1512.4779	322.9993	16.3085	510.1417	199.3771	1.0031
β	0.0104	0.0013	0.0001	455.9621	329.9539	1.0052

Table 3.3: This statistics is generated by the Turing framework [10]. Similar to the above the rows represents model parameters and the columns certain statistics.

Considering the mean of the posterior diagnostics we can see that both sampling calls produce posteriors that resemble the parameters used to generate the data. The main exception is ω_d for which both the Julia implementation ω_d and the Stan/R implementation $\Omega[2]$ both estimate a value that is too low. This appears to be due to the randomness of the specific data set. In both cases, the value of $\hat{R}$ is high, indicating this discrepancy. This is however only clearly readable in the statistics from the Julia implementation, since its the main rhat value that is far off from one. In the Stan/R implementation, multiple rhat values are far off from one.

Design Finally, I want to take a few paragraphs to compare the interface and joint model implementation of my implementation with the one provided by `jmpost` [11].

In the Julia/Turing implementation A.4, using the data type I developed, we specify a joint model inside a function with the `@model` macro annotation. In this function we describe the prior distributions and likelihood calculations. To specify the longitudinal model, we need to write a function that represents this model. For the joint model, we need to specify the link and baseline hazard.

Afterwards, we instantiate the model with observations and call a sampling algorithm which tries to generate samples of the posterior distribution.

The `jmpost` package wraps the Stan/C++ code and R bindings to give us access to joint models. We can specify the prior setup with the function call `JointModel()`, which takes longitudinal and survival distributions that are provided by the `jmpost` package.

This is currently limited to the joint model described by Kerioui in [17] and a random slope model with a Weibull or an exponential survival distribution. After creating a model, we load the data and call a sampling function that calculates the posterior samples.

The benefits of the jmpost implementation are the clear direction of what models are possible and how to load the data. The workflow I present to implement joint models is more flexible what substituting longitudinal or joint models is concerned, as well as sampling algorithm setup. My workflow allows general implementations of joint models in addition to the one featured by Kerioui et al. [17].

Chapter 4

Implementation of Joint Models

This chapter describes the technical implementation of joint models in Julia according to the `Distributions.jl` [3] package. This `HazardBasedDistribution` type allows to formulate joint models and then use them inside the Turing framework for probabilistic programming and Bayesian sampling [10].

4.1 Type: `HazardBasedDistribution`

The `HazardBasedDistribution` type implements a survival distribution based on a given formulation of the hazard function h . In general, we assume that the cumulative hazard H does not have an analytical form. Distributions used for survival analysis [6] such as the Weibull or Log-logistic distributions have a analytical form. Joint models [17] can have a cumulative hazard which does not have an analytical representation. Therefore we will use numeric integration to calculate the cumulative hazard.

The section on mathematical formulations 2.1 describes how the hazard $h(t)$ and the cumulative hazard $H(t) = \int_0^t h(u) du$ functions can formulate the survival function $S(t) = \exp(-H(t))$ and the probability density function $f = h(t) \cdot S(t) = h(t) \cdot \exp(-H(t))$.

To represent distributions based on the hazard formulation, I implement a type¹ that can integrate the hazard function to get the cumulative hazard². We do this with a Gauss-Konrad procedure provided by the `QuadGK.jl` package[16].

```
1 abstract type HazardBasedDistribution <:
  ↳ ContinuousUnivariateDistribution end
```

¹That subtypes `ContinuousUnivariateDistribution` of the `Distributions.jl` [3] package of Juila.

²If you know the analytical formulation of the cumulative hazard of your struct you can also overload the hazard function with that formulation.

4. IMPLEMENTATION OF JOINT MODELS

```
2
3 # represents  $h(t)$ 
4 function hazard(dist::HazardBasedDistribution, t::Real) end
5
6 # calculates  $H(t) = \int_0^t h(u) du$  numerically with a Gauss-Konrad
  ↪ procedure
7 function cumulative_hazard(dist::HazardBasedDistribution, t::Real)
8     # integrate from 0.0001 to  $t$  (some hazards are not defined at 0)
9     /h = IntegralProblem(hazard, 1e-4, t, dist)
10    H = solve(/h, QuadGKJL())[1]
11    return H
12 end
```

We can then formulate the survival `ccdf()` and `pdf()` based on the formulas we derived in subsection 2.1. The `logpdf()` is also implemented which is called by the likelihood calculation inside Turing.jl framework [10].

```
1 function ccdf(dist::HazardBasedDistribution, t::Real)
2     exp(- cumulative_hazard(t, dist))
3 end
4
5 function pdf(dist::HazardBasedDistribution, t::Real)
6     hazard(dist, t) * ccdf(dist, t)
7 end
8
9 function logpdf(dist::HazardBasedDistribution, t::Real)
10    log(pdf(dist, t))
11 end
```

To use the `predict()` functionality of the Turing framework we also need to be able to generate samples using the `rand()` functionality on our distribution [10]. To generate a random sample, we use the ODE solver of the `DifferentialEquations.jl` package [18] to calculate the cumulative hazard $H(t)$ over an interval.

Then we apply inverse transform sampling to create a random sample. With the calculated cumulative hazard, we can formulate the CDF $F(t)$. We generate a random sample $x_0 \in U(0,1)^3$ and search for the time t_0 such that $F(t_0) = x_0$. For this, let us formulate $g(t) = F(t) - x_0$ and search the root t_0 such that $g(t_0) = 0$ which then gives us the sample we search for. In code this is achieved by a root finding algorithm provided by the `Roots.jl` package [25].

³This is done via a uniform random number generator present in Julia [4].

```

1 function rand(rng::AbstractRNG, dist::HazardBasedDistribution)
2     num_end = 1_000_000.
3     # calculate the ode solution of  $H(t) = \int_0^t h(u) du$ 
4     H(nLogS, p, t) = hazard(t, dist)
5     prob = ODEProblem(H, 0.0, (1e-4, num_end))
6     sol = solve(prob, Tsit5(); p=dist)
7     # calculate cdf  $F(t)$ , via survival  $S(t)$ 
8     S(t) = exp(-sol(t))
9     F(t) = 1 - S(t)
10    # inverse sampling: find  $t_0$  such that  $F(t_0) = x_0$  where  $x_0 \sim U(0,1)$ 
11    # via the root of  $g(t) := F(t) - x_0$ 
12    x0 = rand()
13    g(t) = F(t) - x0
14    return find_zero(g, (1e-4, num_end))
15 end

```

4.2 Usage

To showcase the functionality, we will first implement a log-logistic distribution and then a joint model.

LogLogistic

```

1 struct LogLogistic <: HazardBasedDistribution
2     λ
3     α
4 end
5
6 function hazard(t::Real, dist::LogLogistic)
7     α, λ = dist.α, dist.λ
8     return λ*α*(λ*t)^(α-1) / (1 + (λ*t))
9 end

```

This is all we have to define to be able to calculate the pdf, logpdf, survival, cumulative hazard function and to generate random samples. In particular we can use this distribution in the Turing framework to model our data. In the plot 4.1 I plotted the numerical pdf as well as a normalized histogram of the generated samples.

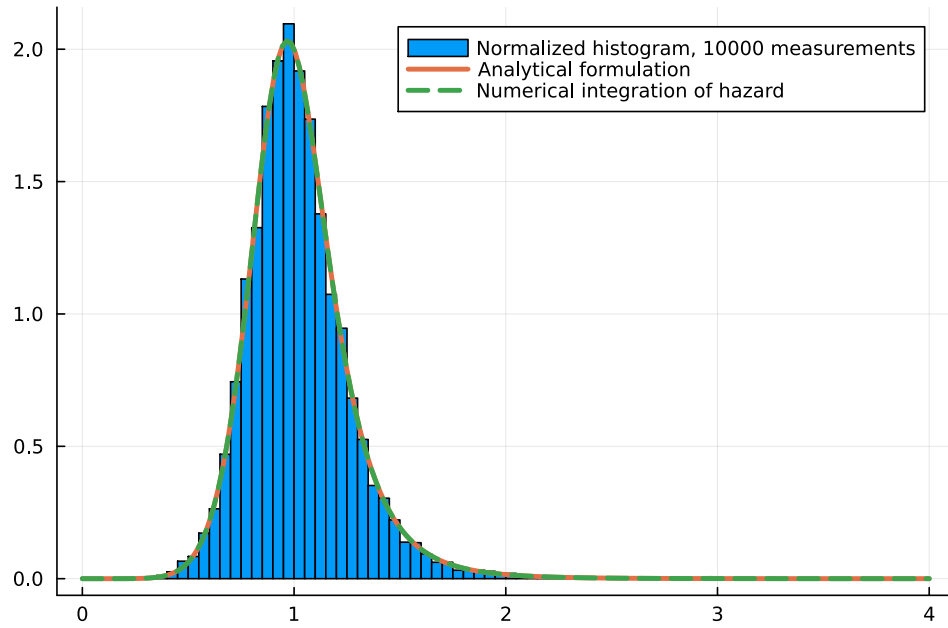


Figure 4.1: Example of $\text{LogLogistic}(1,8)$ implemented with the code above. On the plot you see the analytical pdf in orange as well as the numerical pdf in a dashed green line calculated by the `HazardBasedDistribution` type `numerical_pdf(x) = pdf(LogLogistic(1,8), x)`. The histogram is normalized to a discrete probability density and represents 10'000 samples generated with `rand(LogLogistic(1,8), 10_000)`. See A.2 for the code.

Joint Model

To implement a joint model 2.3 we need to save the baseline survival distribution, the link coefficient β , and the link $t \mapsto g(m(t))$ described as a function of time. To make use of the functionality provided by `HazardBasedDistribution`, we implement a hazard function

$$h(t) = h_0(t) \cdot \exp(\beta \cdot g(m(t)))$$

of a joint model. In code:

```

1 struct SimpleJointModel <: HazardBasedDistribution
2   base_survival::ContinuousUnivariateDistribution
3   β::Real
4   link::Function # function of time
5 end
6
7 h₀(d::SimpleJointModel, t::Real) = hazard(d.base_survival, t)
8
```

```

9 function hazard(d::SimpleJointModel, t::Real)
10     return h0(d, t) * exp(d.β * d.link(t))
11 end

```

I want to highlight the simplicity of implementation and the mapping to the mathematical formulation. The abstract type `HazardBasedDistribution` allows one to implement a joint Model in two steps. First we define a `struct` that saves the data. This `struct` corresponds to the $\theta = (\theta_s, \theta_b, \theta_l)$ mentioned in the likelihood calculation 2.3 of the joint model. Then we implement the `hazard` according to our mathematical joint model formulation.

To make use of all the distributions in the `Distributions.jl` package, we add the `hazard` function. This allows us to make use of them as a baseline survival distribution for a joint model. We can do this using $h(t) = \frac{f(t)}{S(t)}$ since the pdf f and cdf S are defined⁴ for all the distributions in `Distributions.jl` [3].

```

1 hazard(dist::ContinuousUnivariateDistribution, t::Real) = pdf(dist, t)
  ↪ / cdf(dist, t)

```

The `SimpleJointModel` distribution can now be used with a baseline distributions from the `Distributions.jl` package. Since it inherits from the `HazardBasedDistribution` type we can evaluate the pdf and the survival/cdf functions according to their mathematical definitions. And it has the functionality to generate samples of this joint distribution. This is what we need [10] to use this distribution in the Turing.jl probabilistic programming language.

Another point is the recursive definition of `SimpleJointModel`. For the baseline distribution, we can also use another `SimpleJointModel` with its own baseline distribution. This will extend the final hazard formulation to incorporate another link.

$$h(t) = \overline{h_0}(t) \cdot \exp(\overline{\beta} \cdot \overline{g(m(t))}) = \underbrace{h_0(t) \cdot \exp(\beta \cdot g(m(t)))}_{\overline{h_0} \text{ is a joint model}} \cdot \exp(\overline{\beta} \cdot \overline{g(m(t))})$$

This allows⁵ to incorporate different links to different or the same longitudinal process.

⁴Note this calculation will raise an error if the cdf is 0.

⁵A different formulation of the `struct` and `hazard` can be helpful for clarity if you want to design a joint model with multiple links.

Chapter 5

Conclusion

The Julia ecosystem prides itself on interoperability between packages of different domains that work together seamlessly. Working with Julia was simple and straightforward, and the Turing framework with its probabilistic programming language opens interesting doors.

For example in my work I have used a Gauss-Konrad numerical integrator, a differential equation solver and a root finding algorithm, provided by packages in the Julia ecosystem, to implement a distribution for joint models that can be used inside the Turing probabilistic programming framework for Bayesian sampling. The Turing documentation shows examples of implementations of Bayesian estimation of differential equations using `DifferentialEquations.jl` [18], as well as Bayesian neural networks using the main neural network package in Julia called `Flux.jl` [14].

The resulting workflows that you can find in the Appendix A shows how simple the implementation and exploration of joint models can be. This relies on the extendability and interoperability of the Julia ecosystem. I conclude that the Julia programming language is well suited for exploring joint models and developing scientific tools.

One fact not to forget is that the Julia language and the associated packages are rather new. On one hand this means less maturity of the available tools compared to languages such as SAS or R, which first appeared in 1972 and 1993 respectively. On the other hand, this gives the opportunity to do things differently. I hope this work was a convincing demonstration that doing things differently has enormous potential.

Appendix A

Appendix

The main product I handed in at Roche is a documentation website I created to summarize the tools I have developed and discovered in the Julia ecosystem. It showcases workflows for implementing survival, mixed effects longitudinal and joint models.

I have included the PDF¹ versions of the documentation. I have used Quarto [1] to generate this documentation. Its intended format was a website and not a PDF, thus some of the formatting is not ideal.

A.1 Further Exploration

Here is a list of possible further explorations:

- Explore link functions on data: Link functions represent the main idea of joint models. The workflows show how easy it is to explore the link functions.
- Compositional Samplers: Turing offers a compositional Gibbs sampler which could split the search space for longitudinal and survival parameters.
- Submodels: The longitudinal model would be suitable for a submodel. This might lead to an interface for switching longitudinal models.
- AD Mode: HMC algorithms use automatic differentiation. I have focused on using ReverseDiff.jl [15], but there are others supported, such as Zygote.jl [13].

A.2 Survival Workflow

¹Here meaning the Dataformat not probability density function.

Survival

Survival analysis, hazard based distributions and survival analytics

Table of contents

1	Survival Analysis using Hazard based formulations and Bayesian sampling	1
1.1	Basics of the Turing framework	1
1.1.1	Model agnostic diagnostics	7
1.1.2	Posterior predictions	13
1.2	Extending Distributions	15
1.3	Survival Diagnostics	25
1.3.1	Posterior predictive	33

1 Survival Analysis using Hazard based formulations and Bayesian sampling

This workflow shows Bayesian sampling of a survival process and introduces hazard based formulations for the survival distribution. (This is done with the anticipation of joint models which can have no analytical formulation of the pdf and survival function (ccdf))

1.1 Basics of the Turing framework

First we load packages

```
import ForwardDiff
using Distributions, StatsPlots, DataFrames, Turing, Survival
```

Load breastcancer data bc from R package flexsurv (via link to .rda file)

```

using RData
url = "https://github.com/chjackson/flexsurv-dev/raw/master/data/bc.rda"
temp = download(url)
parsed = RData.load(temp)
bc = parsed["bc"]

```

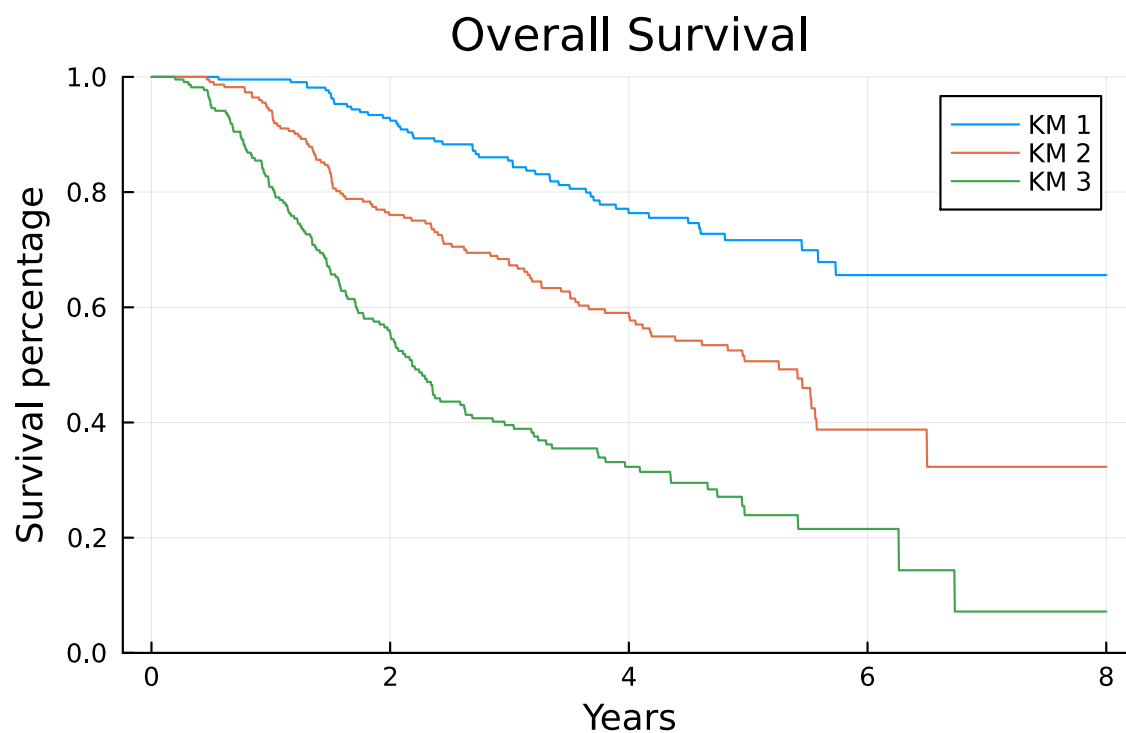
	censrec	rectime	group	recyrs
	Int32	Int32	Cat...	Float64
1	0	1342	Good	3.67671
2	0	1578	Good	4.32329
3	0	1760	Good	4.82192
4	0	1152	Good	3.15616
5	0	967	Good	2.64932
6	0	629	Good	1.72329
7	0	461	Good	1.26301
8	1	991	Good	2.71507
9	0	758	Good	2.07671
10	0	1617	Good	4.43014
11	0	1767	Good	4.8411
12	0	432	Good	1.18356
13	0	892	Good	2.44384
14	0	1933	Good	5.29589
15	0	2456	Good	6.72877
16	0	553	Good	1.51507
17	0	1833	Good	5.02192
18	0	1624	Good	4.44932
19	0	1222	Good	3.34795
20	0	1499	Good	4.10685
21	1	755	Good	2.06849
22	0	722	Good	1.97808
23	0	368	Good	1.00822
24	0	1499	Good	4.10685
25	0	1109	Good	3.03836
26	0	1435	Good	3.93151
27	0	2556	Good	7.00274
28	0	747	Good	2.04658
29	0	1432	Good	3.92329
30	0	2237	Good	6.12877
...	...	...	...	...

Plot Kaplan Meier curves for the three groups “Good”, “Medium”, “Poor” with the `Survival.jl` package [docs](#).

```

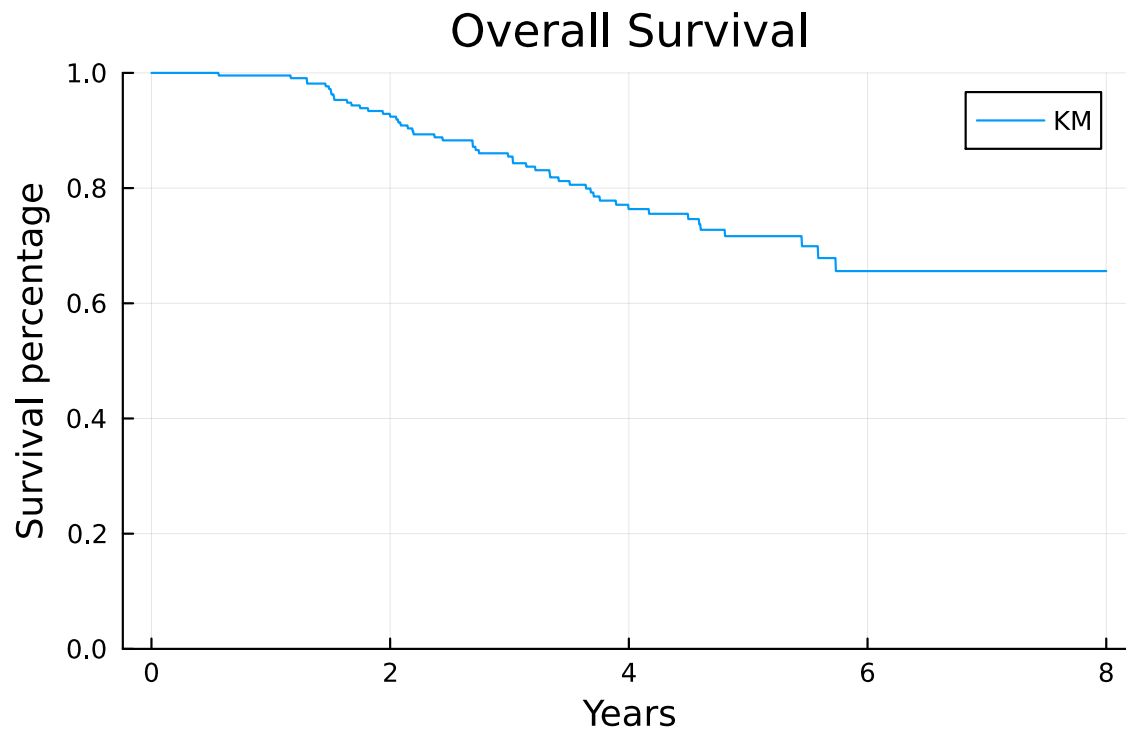
# refere to column name with symbol :column_name
groups = groupby(bc, :group)
r = range(0,8,2000)
p_survival_all = plot(title = "Overall Survival", ylabel = "Survival percentage", xlabel = "Years")
function (km::KaplanMeier)(t::Real)
    time_points = km.events.time
    survival = km.survival
    if t < time_points[1]
        return 1
    else
        id = findlast(x → x <= t, time_points)
        return survival[id]
    end
end
for (index, group) in enumerate(groups)
    # fit using the Survival package
    km_fit = fit(KaplanMeier, group.recyrs, BitVector(group.censrec))
    plot!(p_survival_all, r, km_fit.(r), label="KM $index")
end
ylims!(0,1)
plot!(p_survival_all)

```



For simplicity we only continue the group with label “Good”.

```
g1 = groups[1]
p_survival = plot(title = "Overall Survival", ylabel = "Survival percentage", xlabel = "Years")
km_fit = fit(KaplanMeier, g1.recyrs, BitVector(g1.censrec))
plot!(p_survival, r, km_fit.(r), label="KM")
ylims!(0,1)
plot!(p_survival)
```



Let us build a bayesian model with [Turing.jl docs](#) and sample its posterior. To describe the censoring process the package `Distributions` has the functionality to censor [docs](#) notice that this only provides for right and/or left censoring.

```
@model function weibull_breastcancer(time_obs, indicators)
    # Priors
    κ ~ LogNormal(0.5)
    λ ~ truncated(Normal(8), 2, 10)
    # Likelihood
    n = size(time_obs, 1)
    for id in 1:n
        if Bool(indicators[id]) # uncensored
            time_obs[id] ~ Weibull(κ, λ)
        else # right(upper) censored
            time_obs[id] ~ censored(Weibull(κ, λ), upper = time_obs[id])
        end
    end
end
# data from first group
```

```

surv_times = Vector(g1.recyrs)
event_indicators = Vector(g1.censrec)
# setup Turing model and sample posterior for 500 samples
weibull_model = weibull_breastcancer(surv_times, event_indicators)
chn_weibull = sample(weibull_model, NUTS(), 500)
display(chn_weibull)

```

Chains MCMC chain (500×14×1 Array{Float64, 3}):

```

Iterations          = 251:1:750
Number of chains    = 1
Samples per chain   = 500
Wall duration       = 26.89 seconds
Compute duration    = 26.89 seconds
parameters          = κ, λ
internals            = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
κ	1.8166	0.1764	0.0133	192.4227	212.0565	1.0081	...
λ	8.7472	0.5277	0.0391	183.8484	168.9442	0.9983	...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
κ	1.5099	1.6953	1.8094	1.9173	2.2177
λ	7.7781	8.3748	8.7669	9.1154	9.7421

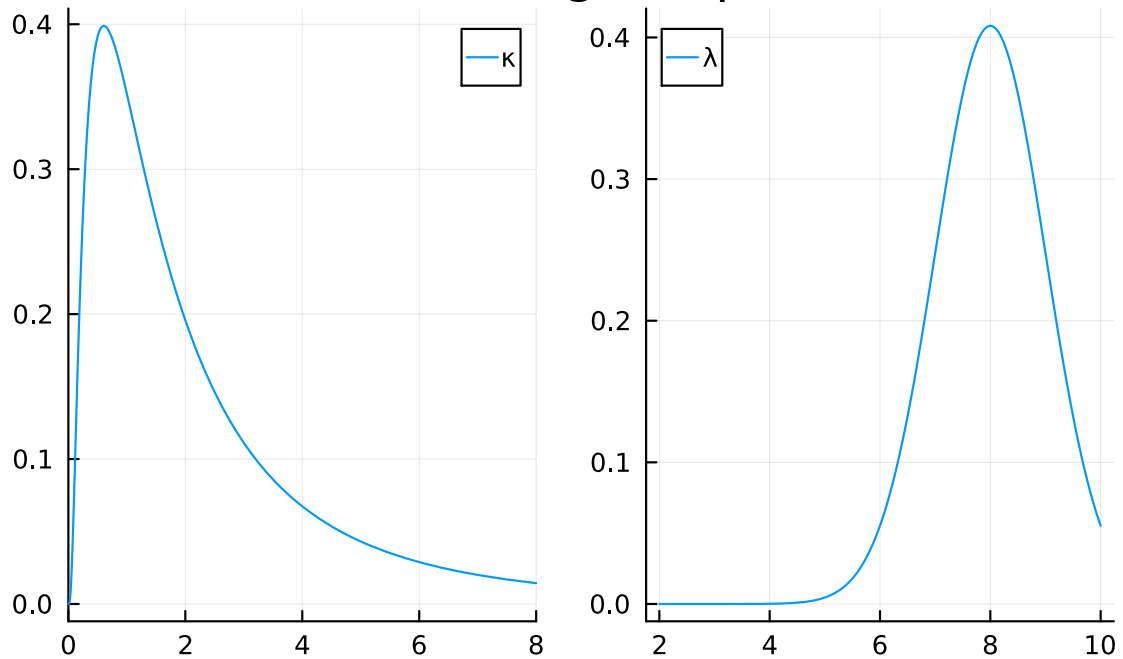
Plotting the priors. With the package StatsPlots we can plot the pdf of a distribution by just calling plot on int.

```

p1 = plot(LogNormal(0.5), label="κ")
xlims!(p1, (0,8))
p2 = plot(truncated(Normal(8),2,10), label="λ")
plot(p1, p2, plot_title = "Visualizing the priors")

```

Visualizing the priors



1.1.1 Model agnostic diagnostics

From the posterior samples we can compute different statistics and diagnostics that help us judge the convergence of the posterior and the descriptiveness of the posterior.

First we use the package `MCMCChains.jl` [docs](#) from the Turing framework (no need to load it but `StatsPlots` is needed for plotting). This package is usefull for working with the posterior in the Turing native `Chains` format, without casting it to a `DataFrame` (which we will do later on for convinience).

Statistics about the posterior samples

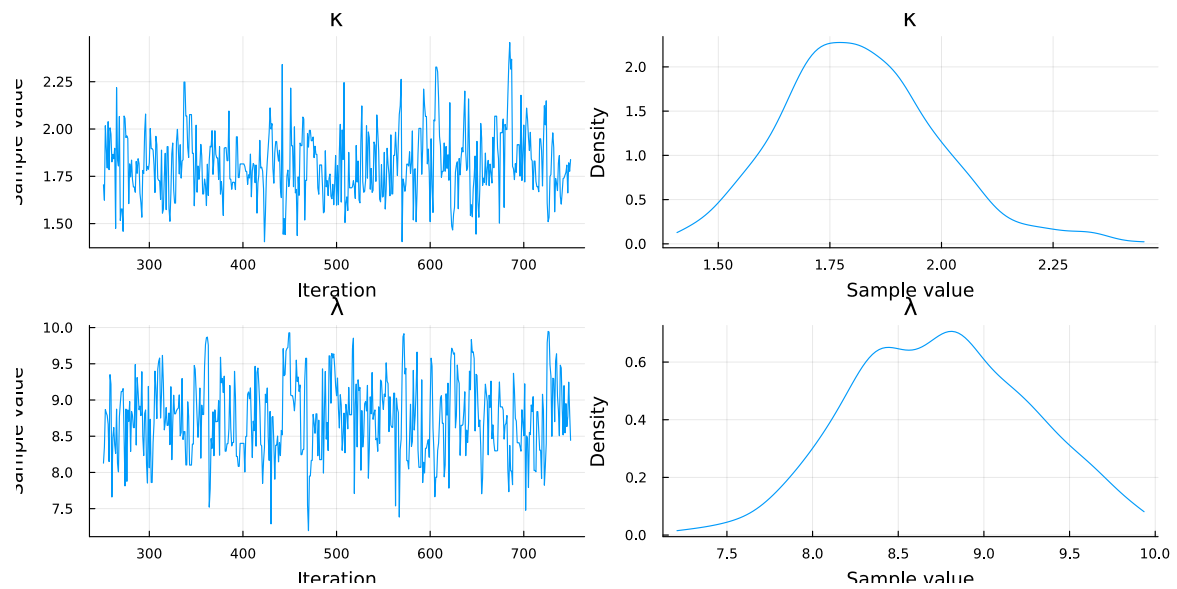
```
summarize(chn_weibull)
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
κ	1.8166	0.1764	0.0133	192.4227	212.0565	1.0081	...
λ	8.7472	0.5277	0.0391	183.8484	168.9442	0.9983	...

1 column omitted

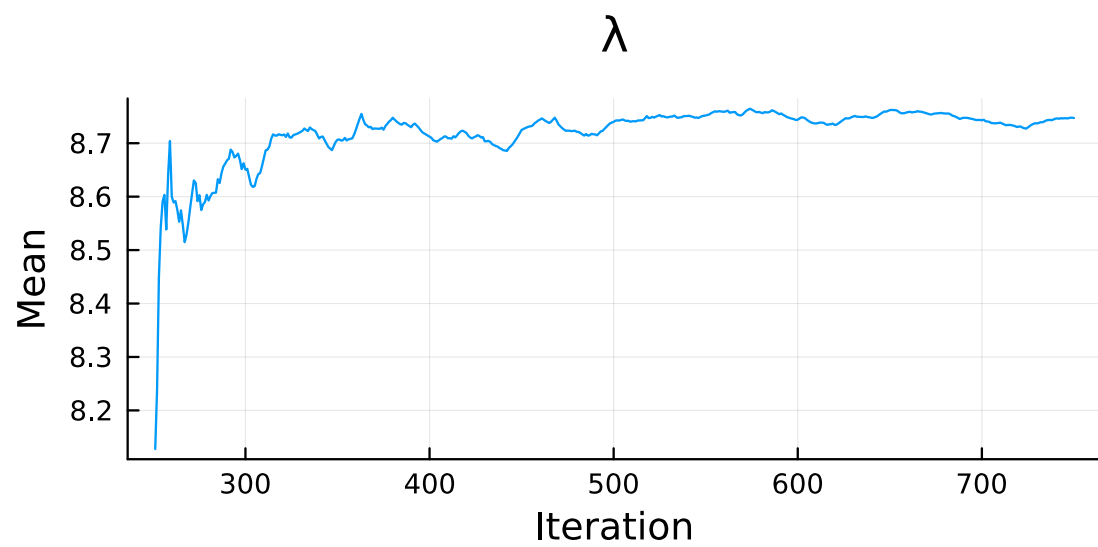
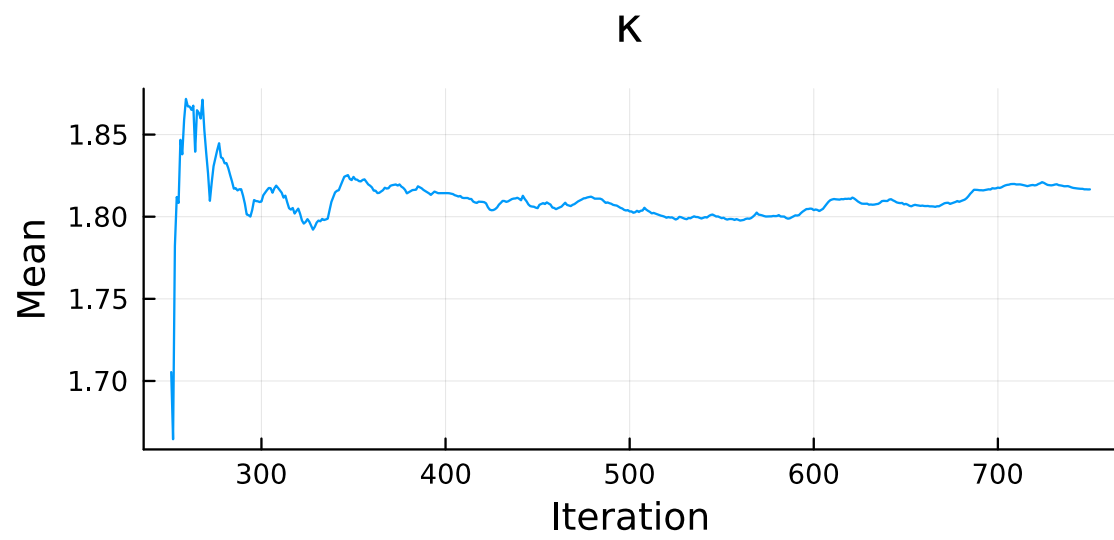
Trace and posterior densities plots

```
plot(chn_weibull)
```



Mean development

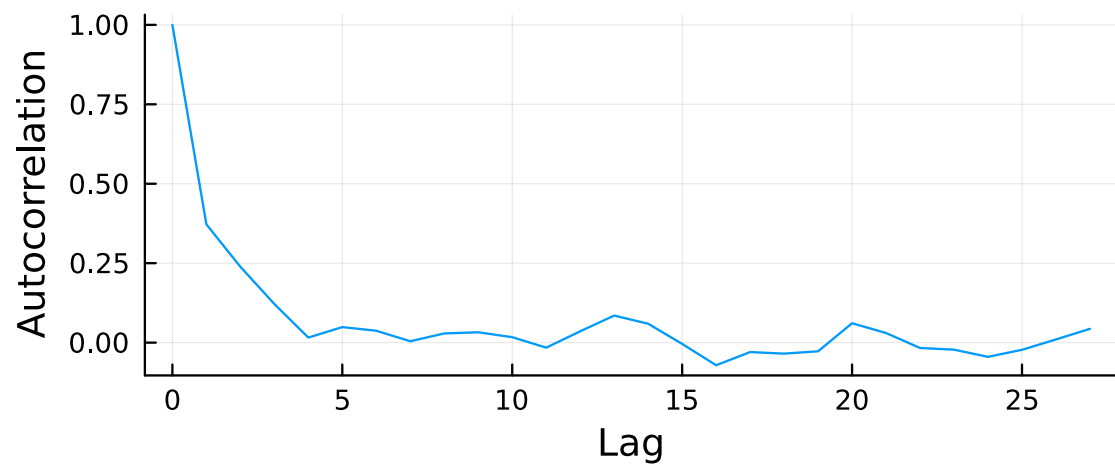
```
meanplot(chn_weibull)
```



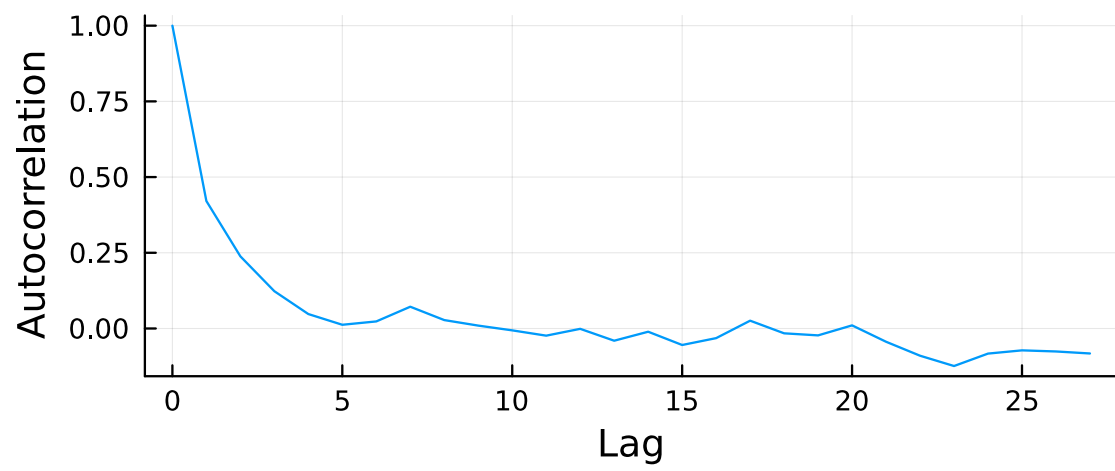
Autocorrelation

```
autocorplot(chn_weibull)
```

K

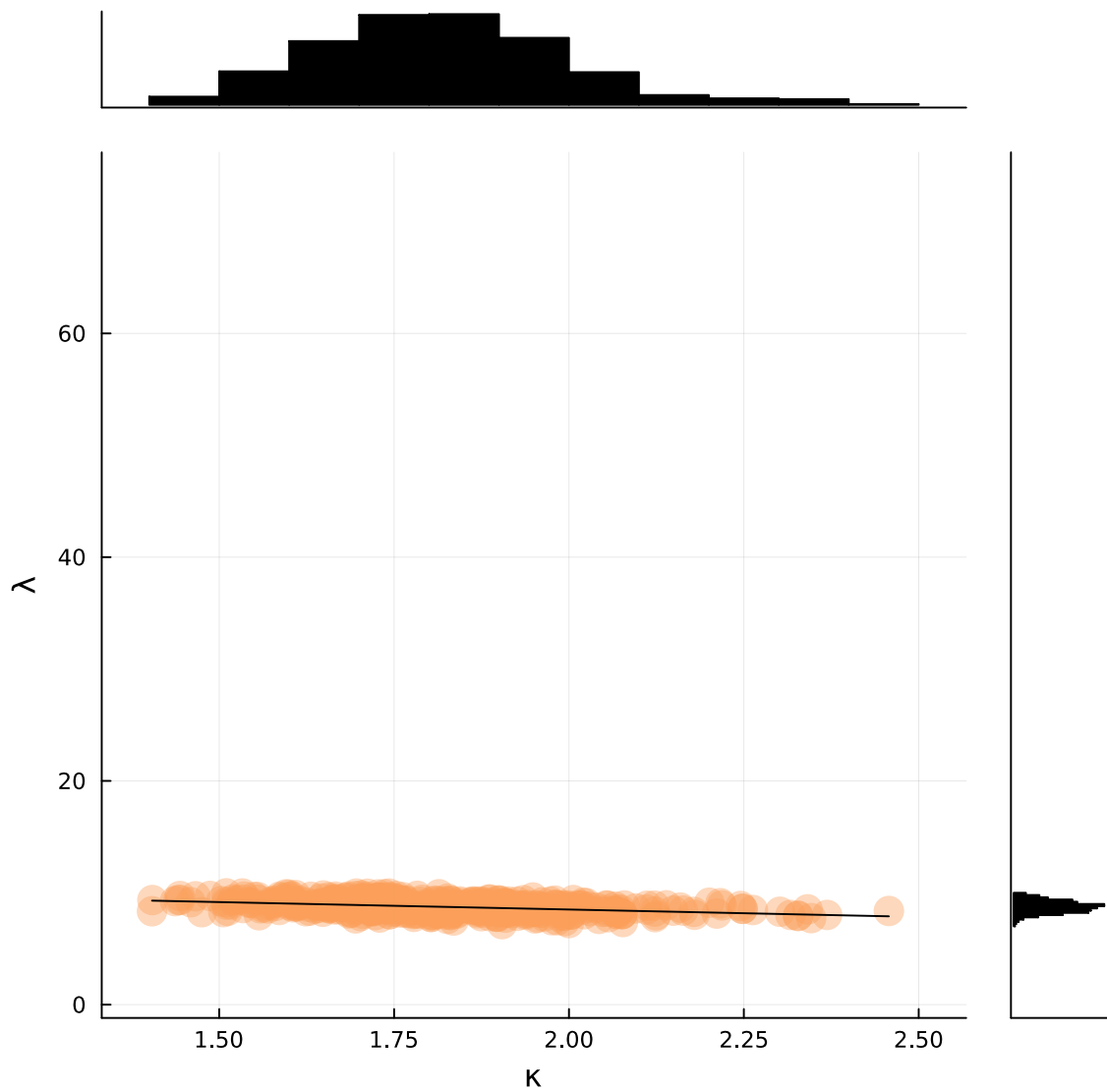


λ



Corner plot

```
corner(chn_weibull)
```



Another type of statistics we can compute from a posterior is the LOO-CV with Pareto smoothed importance sampling with the package `ParetoSmooth.jl` [docs](#) (only docstrings available). We compute this statistics by giving the model instantiation used for finding the posterior `weibull_model` as well as the posterior `chn_weibull`.

```
using ParetoSmooth
psis_loo(weibull_model, chn_weibull)
```

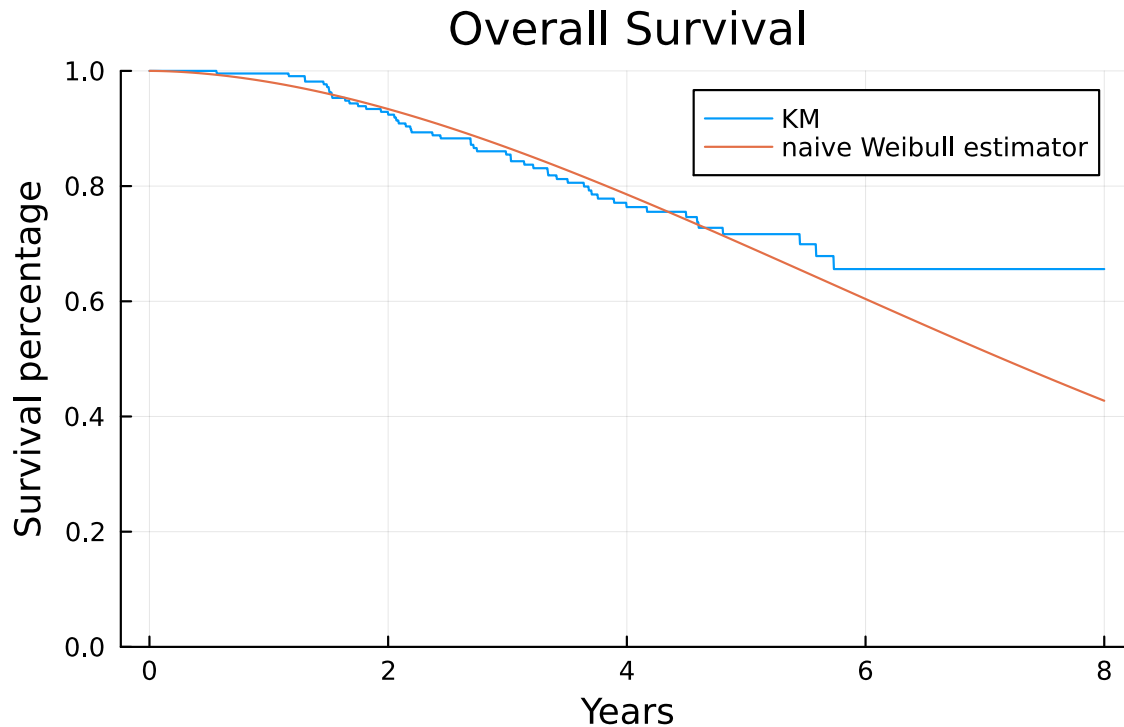
Results of PSIS-LOO-CV with 500 Monte Carlo samples and 229 data points. Total Monte Carlo SE of 0.072.

	total	se_total	mean	se_mean
cv_elpd	-188.38	15.87	-0.82	0.07
naive_lpd	-187.21	15.75	-0.82	0.07
p_eff	1.17	0.17	0.01	0.00

There are also other possibilities to analyze the posterior (which might have feature you desire) using utilities that depend on other languages. One good option would be [ArviZ.jl docs](#) that is a actively maintained interface to a python library by the same name (thus you need to install the python package as well). For such solutions you typically also need to transfere the posterior into the formate the spesific package expects.

We can also compute diagnostics that depend on the model formulation. For example we can add a naive estimator to the Survival plot. I define the naive estimator as the estimator with the mean values of the posterior samples **independently**.

```
# mean values of posterior
κ_hat, λ_hat = summarize(chn_weibull)[: ,2]
# braodcast ccdf/survival over x
S(t) = ccdf(Weibull(κ_hat, λ_hat), t)
plot!(p_survival ,r , S.(r) , label="naive Weibull estimator")
```



Notice that here we take assumptions about the model. We expect it to be a `Weibull` model and that the first two parameters in the posterior represent κ and λ . If we use a different distribution or change around the parameters we can not just run this code and get a sensible output. We would need to change the code!!

1.1.2 Posterior predictions

The Turing framework has the functionality `predict`. For every sample in the posterior `chn_weibull` we can predict time to event according to the parameters describing the survival distribution of that sample. Note: here “predict a time to event” means to sample a `Weibull` given the posterior samples of κ and λ of every posterior sample in the `MCMChain` i.e. drawing $t \sim \text{Weibull}(\kappa', \lambda')$ for all (κ', λ') in the posterior distribution `chn_weibull`.

We initialize a model with a vector of 229 (size of `g1`) missing values as survival data. Turing will then draw a sample for each missing value. We use 100 since later on in the case of Join Models the distribution of all individuals will be different.

```
# setup model with undef/missing values for event times
# for event indicator we need to give a ones vector since we want to sample from the uncensored dist
```

```

surv_missing = Vector{Union{Missing, Float64}}(undef, size(g1,1))
pred_weibull_model = weibull_breastcancer(surv_missing, ones(size(g1,1)))
# predict based on the samples in the `chn`
predictions = predict(pred_weibull_model, chn_weibull)

```

Chains MCMC chain (500×229×1 Array{Float64, 3}):

```

Iterations          = 1:1:500
Number of chains    = 1
Samples per chain   = 500
parameters          = time_obs[1], time_obs[2], time_obs[3], time_obs[4], time_obs[5], time_obs[6], time_
internals           =

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
time_obs[1]	7.6898	4.4448	0.2206	390.7550	387.6959	1.0038	...
time_obs[2]	7.6682	4.2649	0.1948	489.8972	510.8117	0.9999	...
time_obs[3]	7.6341	4.4817	0.2242	400.6552	374.3784	1.0005	...
time_obs[4]	7.8555	4.7834	0.2214	462.9812	438.4716	0.9993	...
time_obs[5]	7.6602	4.5027	0.2040	437.5396	331.8528	1.0010	...
time_obs[6]	7.6878	4.1974	0.1985	450.2196	437.5122	1.0003	...
time_obs[7]	7.9789	4.6927	0.2123	480.7518	488.3172	0.9990	...
time_obs[8]	7.7611	4.5905	0.2038	534.5944	500.0466	0.9989	...
time_obs[9]	8.0623	4.7136	0.2156	479.6581	450.2251	0.9993	...
time_obs[10]	7.5855	4.4464	0.1988	481.3801	452.7208	1.0032	...
time_obs[11]	7.8226	4.7259	0.2113	479.2433	409.4632	0.9990	...
time_obs[12]	7.6236	4.4459	0.1943	577.7954	470.5002	0.9990	...
time_obs[13]	7.5682	4.3185	0.1870	529.4034	473.9927	1.0013	...
time_obs[14]	7.6905	4.5002	0.2058	492.4089	444.6374	1.0013	...
time_obs[15]	7.9595	4.4725	0.2026	463.4535	425.6557	1.0082	...
time_obs[16]	7.6603	4.2850	0.1890	526.0781	473.4051	1.0009	...
time_obs[17]	7.9151	4.7856	0.2197	445.0917	276.8555	0.9994	...
time_obs[18]	7.8026	4.5780	0.2026	510.0766	456.6324	0.9985	...
time_obs[19]	7.5142	4.3433	0.2019	480.1798	410.7077	1.0011	...
time_obs[20]	7.6312	4.5896	0.2110	448.8757	436.0858	1.0026	...
time_obs[21]	7.5840	4.3839	0.2194	393.9659	411.9940	1.0095	...
time_obs[22]	7.5622	4.4535	0.2008	518.6294	406.7740	1.0022	...
time_obs[23]	7.8816	4.6153	0.2231	465.1456	358.8939	1.0051	...
:	:	:	:	:	:	:	⋮

1 column and 206 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
time_obs[1]	1.2829	4.5159	7.1573	10.2381	17.3845
time_obs[2]	1.1005	4.2718	7.1442	10.6876	16.6956
time_obs[3]	0.7961	4.3128	7.2392	10.2377	17.4206
time_obs[4]	0.9663	4.2639	7.1403	10.5175	18.6254
time_obs[5]	1.2997	4.1205	6.8617	10.5986	17.8534
time_obs[6]	1.1108	4.7467	7.2101	10.0726	17.4080
time_obs[7]	1.2699	4.4184	7.3678	10.7741	19.8954
time_obs[8]	1.4811	4.6330	6.8008	10.0068	19.2668
time_obs[9]	1.2644	4.4038	7.3944	10.7498	18.7482
time_obs[10]	1.1248	4.2962	6.9217	10.1305	17.9876
time_obs[11]	1.0626	4.2835	6.9338	10.6421	19.1593
time_obs[12]	1.1819	4.2188	6.8918	10.3508	17.9426
time_obs[13]	1.0395	4.2052	6.8942	10.1540	16.9788
time_obs[14]	0.9514	4.5325	6.7663	10.2075	18.0506
time_obs[15]	1.2571	4.5118	7.2716	11.0052	17.1468
time_obs[16]	0.7078	4.4267	7.3927	10.4784	16.6372
time_obs[17]	1.1923	4.4218	6.9897	10.3472	19.8677
time_obs[18]	0.9845	4.1732	7.1990	10.5089	18.1881
time_obs[19]	0.9319	4.3830	6.7718	10.2192	17.3912
time_obs[20]	0.9538	4.0334	7.1578	10.3129	17.6027
time_obs[21]	1.2329	4.2787	7.1072	10.1688	17.2408
time_obs[22]	0.9809	4.1995	6.8921	10.2420	17.7192
time_obs[23]	1.1134	4.5688	7.1365	10.6826	18.8480
⋮	⋮	⋮	⋮	⋮	⋮

206 rows omitted

1.2 Extending Distributions

The distributions `Weibull` is provided by the `Distributions.jl` package, it provides 69 univariate distributions [list](#). Note that for example the Log-Logistic distribution is missing, which might be interesting for Survival Analysis. You can implement your own distribution yourself [docs](#) and then use it inside the Turing framework [docs](#) for bayesian sampling. For the Turing framework the functionalities `rand` and `logpdf` are needed.

Since for Joint Models we will use hazard based formulations of survival distributions I designed a support structure that allows you to define a distribution based on its hazard function.

```
using Integrals, DifferentialEquations, Random, Roots, Distributions
```

```
abstract type HazardBasedDistribution <: ContinuousUnivariateDistribution end
```

```
@doc raw"""
```

```
represents ``h(t)``
```

```
needs to be implemented for any `struct` that subtypes HazardBasedDistribution  
"""
```

```
function hazard(dist::HazardBasedDistribution, t::Real) end
```

```
@doc raw"""
```

```
Different argument order to comply with sciML (Integration)  
"""
```

```
sciML_hazard(t::Real, dist::HazardBasedDistribution) = hazard(dist, t)
```

```
@doc raw"""
```

```
calculates ``H(t) = \int_0^t h(u) \; du`` numerically with a Gauss-Konrad procedure  
"""
```

```
function cumulative_hazard(dist::HazardBasedDistribution, t::Real)
```

```
    # integrate from 0.000001 to $t$ (some hazards are not defined at 0)
```

```
    /h = IntegralProblem(sciML_hazard, 1e-6, t, dist)
```

```
    H = solve(/h, QuadGKJL())[1]
```

```
    return H
```

```
end
```

```
@doc raw"""
```

```
Calculation of the ccdf / survival function at time `t` based on the cumulative hazard ``S(t) = \exp(-H(t))``  
"""
```

```
function Distributions.ccdf(dist::HazardBasedDistribution, t::Real)
```

```
    exp(- cumulative_hazard(dist, t))
```

```
end
```

```
@doc raw"""
```

```
Calculation of the log pdf function at time `t` based on the cumulative hazard ``\log f(t) = h(t) - H(t)``  
"""
```

```
function Distributions.logpdf(dist::HazardBasedDistribution, t::Real)
```

```
    log(hazard(dist, t)) - cumulative_hazard(dist, t)
```

```
end
```

```
@doc raw"""
```

```

Calculation of the pdf function at time `t` based on the log pdf
"""
function Distributions.pdf(dist::HazardBasedDistribution, t::Real)
    exp(logpdf(dist, t))
end

H(nLogS, p, t) = sciml_hazard(t,p)
@doc raw"""
Generate a random sample ``t \sim \text{dist}``
"""
function Distributions.rand(rng::AbstractRNG, dist::HazardBasedDistribution)
    num_end = 1_000_000
    # calculate ode solution of $H(t) = \int_0^t h(u) \, du$
    prob = ODEProblem(H, 0.0, (1e-4, num_end))
    sol = solve(prob, Tsit5(); p=dist)
    # calculate cdf $F(t)$, via survival $S(t)$
    S(t) = exp(-sol(t))
    F(t) = 1 - S(t)
    # inverse sampling: find $t_0$ such that $F(t_0) = x_0$ where $x_0 \sim U(0,1)$
    # via the root of $g(t) := F(t) - x_0$
    x_0 = rand()
    g(t) = F(t) - x_0
    return find_zero(g, (1e-4, num_end))
end

```

Base.rand

Here is an implementation of the LogLogistic: (This is a fabricated example, since there is a well known analytical formulation of the pdf, ccdf/survival etc. It would be better to implement this distribution with analytical calculations) We use the formulation found on [wikipedia](https://en.wikipedia.org/wiki/Log-logistic_distribution).

```

struct LogLogistic <: HazardBasedDistribution
    α::Real
    β::Real
end

function hazard(dist::LogLogistic, t::Real)
    α, β = dist.α, dist.β
    return ((β/α)*(t/α)^(β-1))/(1 + (t/α)^β)
end

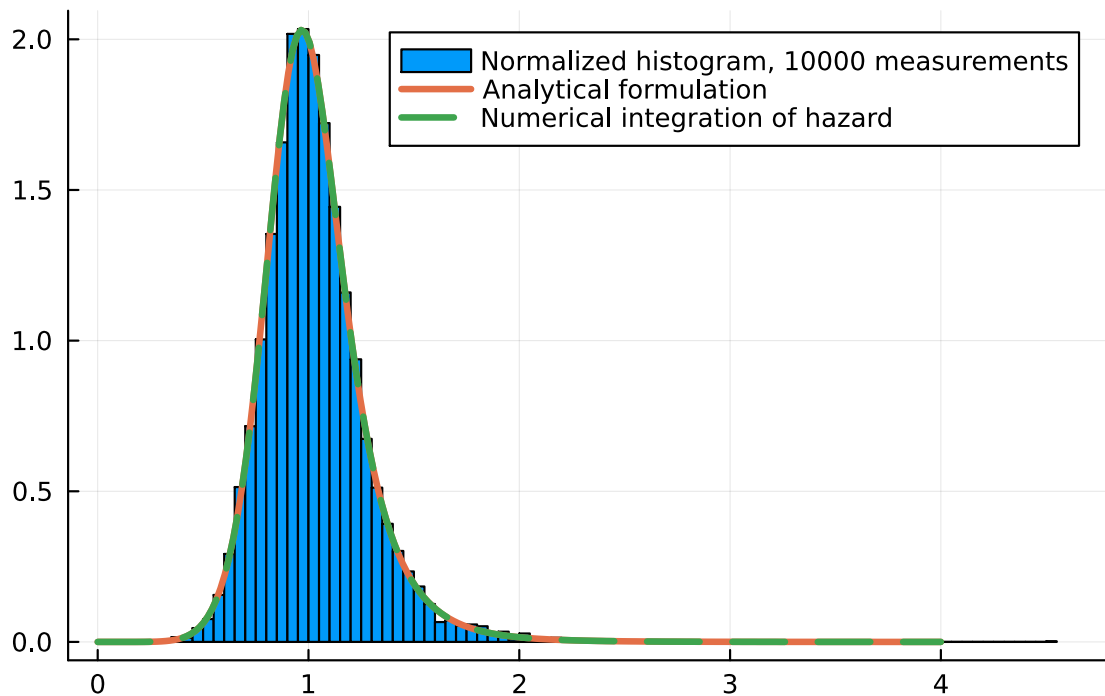
```

```
end
```

hazard (generic function with 2 methods)

To convince ourselves of this implementation we showcase the analytical pdf, numerical pdf and a histogram of 10'000 measurements:

```
 $\alpha, \beta = 1, 8$ 
n = 10_000
T = range(0, 4, length=200)
# analytical pdf
f_analytical(t) = (( $\beta/\alpha$ )*(t/ $\alpha$ )( $\beta-1$ )) / (1 + (t/ $\alpha$ ) $\beta$ )2
# numerical pdf
dist = LogLogistic( $\alpha, \beta$ )
f_hazardbased(t) = pdf(dist, t)
# 10_000 samples
measurements = [rand(dist) for i in 1:n]
# plotting
histogram(measurements, bins=90, normalize=:pdf, label = "Normalized histogram, $n measurements")
plot!(T, f_analytical.(T), label="Analytical formulation", linewidth = 3)
plot!(T, f_hazardbased.(T), label="Numerical integration of hazard", linestyle=:dash, linewidth = 3)
```



Now we can use `LogLogistic` in Turing

```
@model function loglogistic_breastcancer(time_obs, indicators)
  # Priors
  α ~ truncated(Normal(8,2),1,20)
  β ~ truncated(Normal(2),0.2,4)
  n = size(time_obs, 1)
  for id in 1:n
    if Bool(indicators[id]) # uncensored
      time_obs[id] ~ LogLogistic(α, β)
    else # right censored
      time_obs[id] ~ censored(LogLogistic(α, β), upper = time_obs[id])
    end
  end
end

# setup Turing model and sample posterior
loglogistic_model = loglogistic_breastcancer(surv_times, event_indicators)
chn_loglogistic = sample(loglogistic_model, NUTS(), 500)
```

Chains MCMC chain (500×14×1 Array{Float64, 3}):

```

Iterations          = 251:1:750
Number of chains    = 1
Samples per chain   = 500
Wall duration       = 357.38 seconds
Compute duration    = 357.38 seconds
parameters          =  $\alpha$ ,  $\beta$ 
internals           = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
α	8.1711	0.7749	0.0474	267.1429	317.0375	1.0078	...
β	1.8747	0.1926	0.0149	174.3546	210.9233	1.0115	...

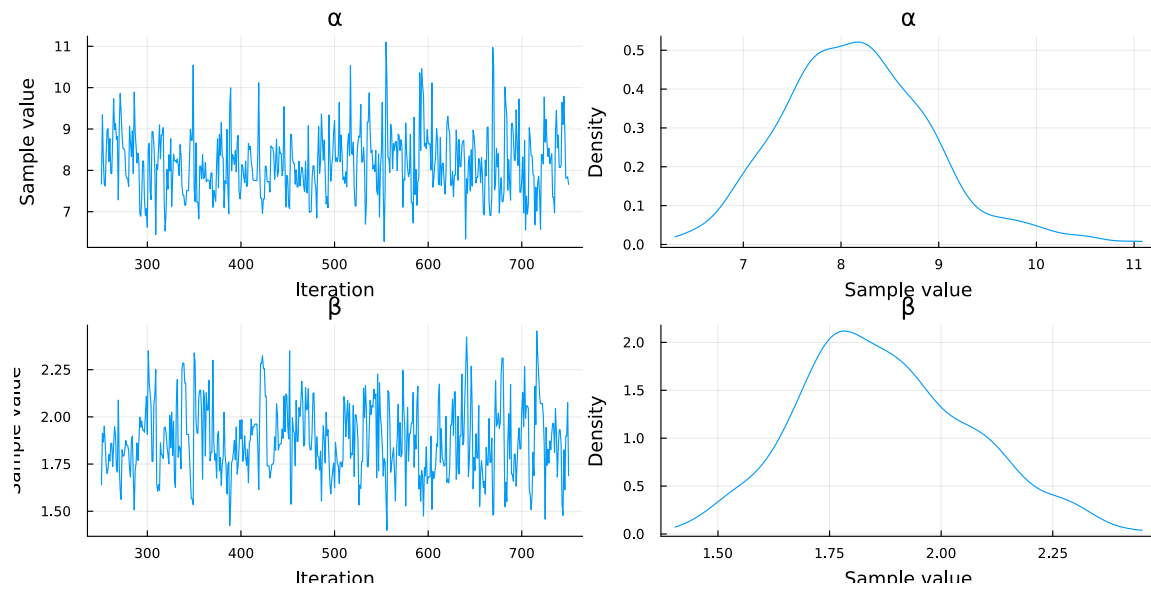
1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
α	6.8812	7.6384	8.1384	8.6484	9.9484
β	1.5306	1.7438	1.8564	2.0035	2.2841

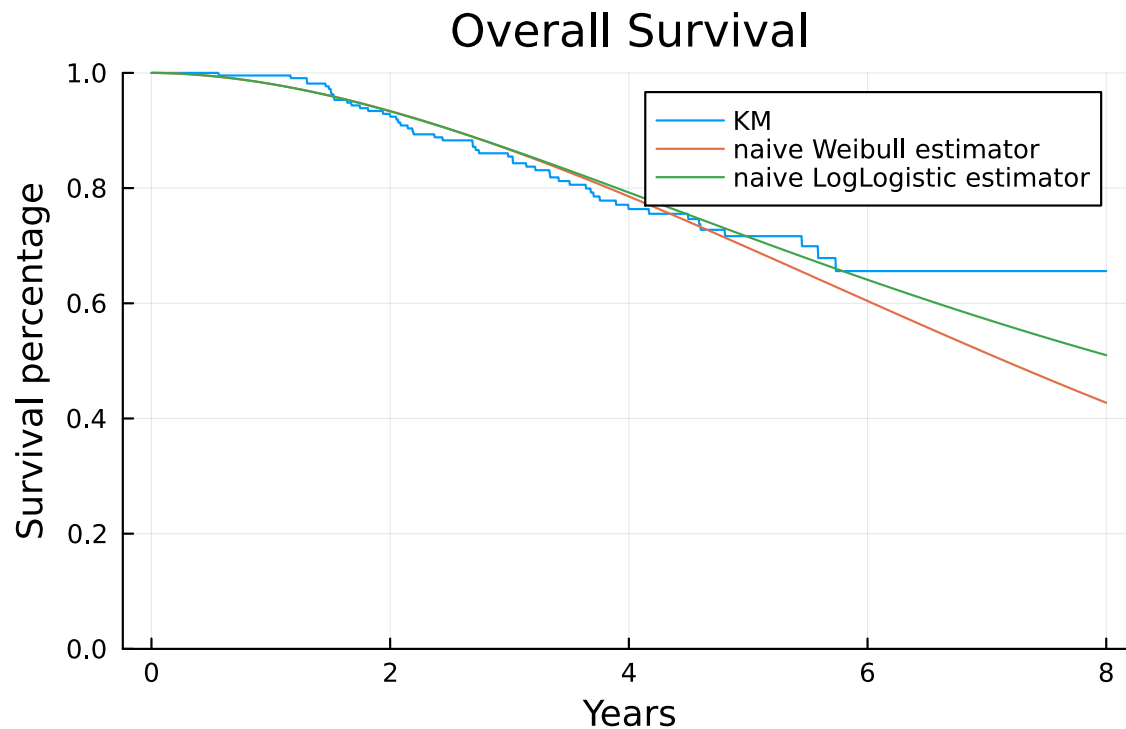
Posterior of LogLogistic model.

```
plot(chn_loglogistic)
```



Adding the naive LogLogistic estimator.

```
x = range(0,8,200)
α_hat, β_hat = summarize(chn_loglogistic)[: ,2]
# broadcast ccdf/survival over x
y = ccdf.(LogLogistic(α_hat, β_hat), x)
plot!(p_survival ,x ,y, label="naive LogLogistic estimator")
```



We can also implement the Weibull ourselves

```

struct HazardWeibull <: HazardBasedDistribution
  κ::Real
  λ::Real
end

function hazard(d::HazardWeibull, t::Real)
  return d.κ / d.λ * (t / d.λ)^(d.κ - 1)
end

@model function hazard_weibull_breastcancer(time_obs, indicators)
  # Priors
  κ ~ truncated(LogNormal(0.5), 0.4, 2)
  λ ~ truncated(Normal(8), 0.1, 10)
  n = size(time_obs, 1)
  for id in 1:n
    if Bool(indicators[id]) # uncensored

```

```

        time_obs[id] ~ HazardWeibull( $\kappa$ ,  $\lambda$ )
      else # right censored
        time_obs[id] ~ censored(HazardWeibull( $\kappa$ ,  $\lambda$ ), upper = time_obs[id])
      end
    end
  end
end
# setup Turing model and sample posterior
hazard_weibull_model = hazard_weibull_breastcancer(surv_times, event_indicators)
chn_hazard_weibull = sample(hazard_weibull_model, NUTS(), 500)

```

Chains MCMC chain (500×14×1 Array{Float64, 3}):

```

Iterations      = 251:1:750
Number of chains = 1
Samples per chain = 500
Wall duration    = 246.17 seconds
Compute duration = 246.17 seconds
parameters       =  $\kappa$ ,  $\lambda$ 
internals        = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
κ	1.7533	0.1306	0.0105	140.9916	114.7810	1.0358	...
λ	8.9127	0.4974	0.0355	188.8692	193.7651	1.0074	...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
κ	1.4962	1.6608	1.7601	1.8534	1.9714
λ	7.8704	8.5614	8.9691	9.2952	9.7351

Here we can compare the posterior statistics of the hazard Weibull model with the analytical implementation of the Distribution package.

Analytical formulation:

```

summarize(chn_weibull)

```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
κ	1.8166	0.1764	0.0133	192.4227	212.0565	1.0081	...
λ	8.7472	0.5277	0.0391	183.8484	168.9442	0.9983	...

1 column omitted

Hazard based numeric formulation:

```
summarize(chn_hazard_weibull)
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
κ	1.7533	0.1306	0.0105	140.9916	114.7810	1.0358	...
λ	8.9127	0.4974	0.0355	188.8692	193.7651	1.0074	...

1 column omitted

Q: How would you extend the structure of LogLogistic or Weibull to include covariate data? (and covariate coefficients?)

Idea

We build a new struct with a vector for covariates x and a vector for covariate coefficients b and extend the hazard formulation by multiplying with $\exp(b_1 \cdot x_1 + \dots + b_n \cdot x_n)$.

```
struct HazardWeibullCovariate <: HazardBasedDistribution
    κ::Real
    λ::Real
    x::Vector{Real}
    b::Vector{Real}
end

function hazard(d::HazardWeibullCovariate, t::Real)
    return d.κ / d.λ * (t / d.λ)^(d.κ - 1) * exp(sum(d.b .* d.x))
end
```

In the Turing @model we then also give a prior for the covariate parameters.

This is already a step towards building Joint Models later on!

1.3 Survival Diagnostics

To create diagnostics for a Bayesian model we might want to calculate some quantities for samples in our posterior distribution of models. For this I developed a `DiagnosticHelper` that allows you to save quantities based on posterior samples using the Turing framework to parse the posterior.

```
"""
The elegance of this helper is that Turing does all the legwork for parsing the posterior samples and
If you want to change the model you have to do minimal changes to the diagnostics to still get the same
"""

struct DiagnosticHelper <: ContinuousUnivariateDistribution
    value::Real
end

function Distributions.rand(rng::AbstractRNG, dist::DiagnosticHelper)
    return dist.value
end

function Distributions.logpdf(dist::DiagnosticHelper, t::Real)
    return 0
end
```

Let us calculate cox-snell residuals i.e. the cumulative hazard $H(t)$ (here we do not consider covariates) for the individuals using the `LogLogistic` model. We can calculate these residuals in the initial sampling step or use the posterior `chn_loglogistic` we have calculated before. In both cases we have to extend the `@model` definition.

```
using DynamicPPL
@model function loglogistic_breastcancer_cox_snell(time_obs, indicators)
    # Priors
    α ~ truncated(Normal(8,2),1,20)
    β ~ truncated(Normal(2),0.2,4)
    n = size(time_obs, 1)
    # this line is all the magic, here we calculate the cumulative hazard for all the individuals and
    cox_snell_residual ~ arraydist([DiagnosticHelper(
        cumulative_hazard(LogLogistic(α, β), time_obs[id]) ①
    ) for id in 1:n])

    """ # <2>
    for id in 1:n # <2>
        if Bool(indicators[id]) # uncensored # <2>
            time_obs[id] ~ LogLogistic(α, β) # <2>
        end
    end
end
```

```

        else # right censored # <2>
            time_obs[id] ~ censored(LogLogistic( $\alpha$ ,  $\beta$ ), upper = time_obs[id]) # <2>
        end # <2>
    end # <2>
    """"
end
# setup Turing model
loglogistic_model_res = loglogistic_breastcancer_cox_snell(surv_times, event_indicators)
loglogistic_cox_snell_residuals_chn = predict(loglogistic_model_res, chn_loglogistic) ③

```

- ① Calculates $H(t)$ based on hazard of posterior parametric distribution for all individuals
- ② Here we leave out (comment) the likelihood formulation since this would generate individual prediction of survival time, which we might not be interested in.
- ③ When predicting the DiagnosticHelper will be samples which returns the value we want to generate.

Chains MCMC chain (500×229×1 Array{Float64, 3}):

```

Iterations      = 1:1:500
Number of chains = 1
Samples per chain = 500
parameters      = cox_snell_residual[1], cox_snell_residual[2], cox_snell_residual[3], cox_snell_resi
internals       =

```

Summary Statistics

	parameters	mean	std	mcse	ess_bulk	ess_tail	...
	Symbol	Float64	Float64	Float64	Float64	Float64	...
	cox_snell_residual[1]	0.2071	0.0301	0.0017	324.2996	381.0044	...
	cox_snell_residual[2]	0.2710	0.0361	0.0019	358.8836	352.3842	...
	cox_snell_residual[3]	0.3233	0.0413	0.0022	353.1554	374.3784	...
	cox_snell_residual[4]	0.1598	0.0256	0.0015	284.8535	380.4440	...
	cox_snell_residual[5]	0.1180	0.0213	0.0013	253.5410	404.7724	...
	cox_snell_residual[6]	0.0552	0.0133	0.0009	204.5810	282.3352	...
	cox_snell_residual[7]	0.0316	0.0091	0.0006	191.9671	278.0979	...
	cox_snell_residual[8]	0.1232	0.0219	0.0014	256.1304	404.7724	...
	cox_snell_residual[9]	0.0769	0.0164	0.0011	220.8910	343.1262	...
	cox_snell_residual[10]	0.2820	0.0372	0.0020	357.5770	374.6154	...
	cox_snell_residual[11]	0.3253	0.0415	0.0022	353.1095	374.3784	...
	cox_snell_residual[12]	0.0281	0.0084	0.0006	191.2569	278.0979	...
	cox_snell_residual[13]	0.1025	0.0196	0.0013	241.2941	368.1898	...
	cox_snell_residual[14]	0.3748	0.0467	0.0025	345.0134	374.1132	...

cox_snell_residual[15]	0.5378	0.0659	0.0038	301.8266	398.0777	...
cox_snell_residual[16]	0.0438	0.0114	0.0008	196.4384	282.3352	...
cox_snell_residual[17]	0.3448	0.0436	0.0023	351.6535	374.3784	...
cox_snell_residual[18]	0.2840	0.0374	0.0020	356.4598	374.1132	...
cox_snell_residual[19]	0.1768	0.0272	0.0016	297.5445	380.4440	...
cox_snell_residual[20]	0.2491	0.0340	0.0018	356.4147	358.1506	...
cox_snell_residual[21]	0.0763	0.0163	0.0011	220.6501	335.0869	...
cox_snell_residual[22]	0.0705	0.0155	0.0011	214.9196	262.1514	...
cox_snell_residual[23]	0.0211	0.0068	0.0005	191.5847	306.8386	...
:	:	:	:	:	:	~

2 columns and 206 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
cox_snell_residual[1]	0.1501	0.1877	0.2064	0.2280	0.2676
cox_snell_residual[2]	0.2006	0.2472	0.2681	0.2940	0.3483
cox_snell_residual[3]	0.2431	0.2972	0.3199	0.3498	0.4145
cox_snell_residual[4]	0.1110	0.1431	0.1598	0.1753	0.2099
cox_snell_residual[5]	0.0788	0.1044	0.1186	0.1300	0.1605
cox_snell_residual[6]	0.0315	0.0459	0.0553	0.0630	0.0821
cox_snell_residual[7]	0.0164	0.0250	0.0315	0.0370	0.0504
cox_snell_residual[8]	0.0828	0.1092	0.1240	0.1353	0.1670
cox_snell_residual[9]	0.0480	0.0658	0.0777	0.0865	0.1100
cox_snell_residual[10]	0.2089	0.2573	0.2794	0.3056	0.3619
cox_snell_residual[11]	0.2447	0.2991	0.3220	0.3520	0.4170
cox_snell_residual[12]	0.0143	0.0219	0.0280	0.0331	0.0455
cox_snell_residual[13]	0.0665	0.0899	0.1031	0.1134	0.1416
cox_snell_residual[14]	0.2846	0.3426	0.3706	0.4044	0.4770
cox_snell_residual[15]	0.4128	0.4922	0.5312	0.5800	0.6703
cox_snell_residual[16]	0.0240	0.0358	0.0437	0.0505	0.0670
cox_snell_residual[17]	0.2602	0.3162	0.3419	0.3722	0.4407
cox_snell_residual[18]	0.2105	0.2592	0.2812	0.3078	0.3643
cox_snell_residual[19]	0.1248	0.1586	0.1767	0.1941	0.2312
cox_snell_residual[20]	0.1840	0.2271	0.2473	0.2714	0.3206
cox_snell_residual[21]	0.0476	0.0653	0.0771	0.0860	0.1093
cox_snell_residual[22]	0.0432	0.0599	0.0711	0.0798	0.1018
cox_snell_residual[23]	0.0101	0.0159	0.0208	0.0251	0.0353
:	:	:	:	:	:

206 rows omitted

From this output we can plot the cox snell residuals for the posterior distribution against the

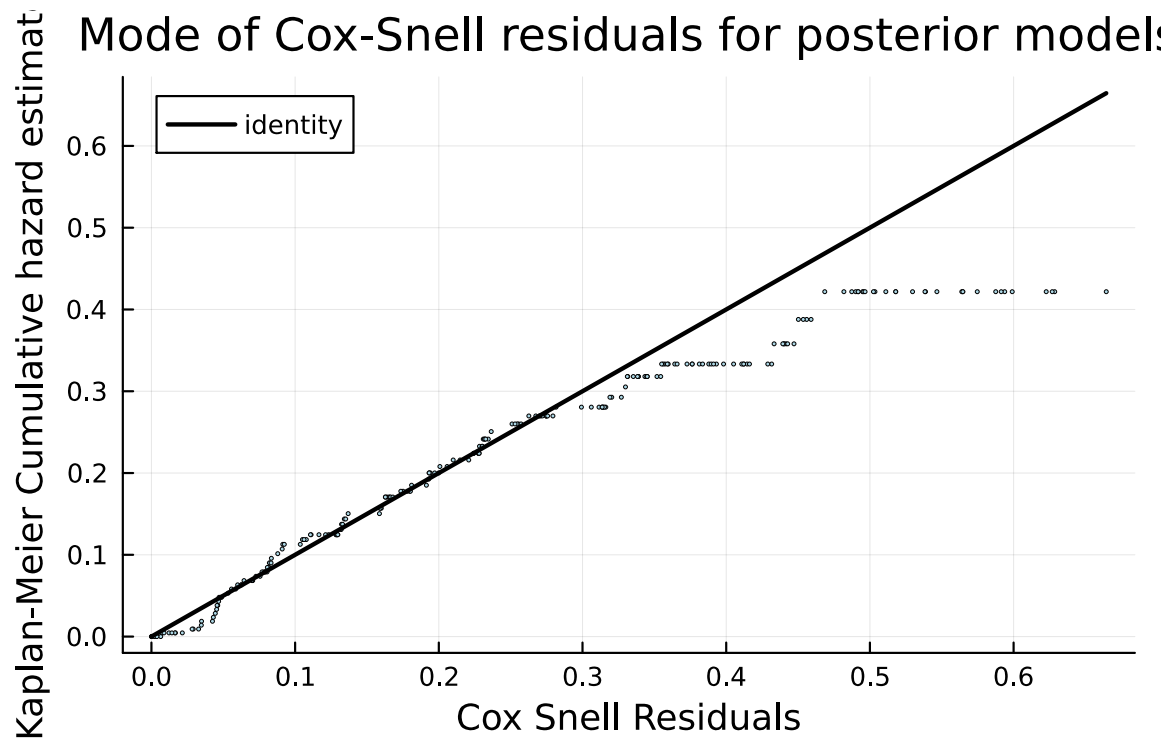
cumulative hazard of the Kaplan Meier estimator. Per individual we calculate the mode of all posterior cox snell residuals as well as the quantiles. We then plot this versus the cumulative hazard of the Kaplan-Meier estimator.

```
function plot_cox_snell(cox_snell_residuals_chn, surv_times, event_indicators; column_regex = r"cox_")
  cox_snell_df = DataFrame(cox_snell_residuals_chn)
  posterior_residuals = select!(cox_snell_df, column_regex)
  cox_snell_modes = mode.(eachcol(posterior_residuals))

  # quantiles = quantile(cox_snell_residuals_chn)
  # relative_errorBars = (cox_snell_modes .- quantiles[:,2], quantiles[:,6] .- cox_snell_modes)

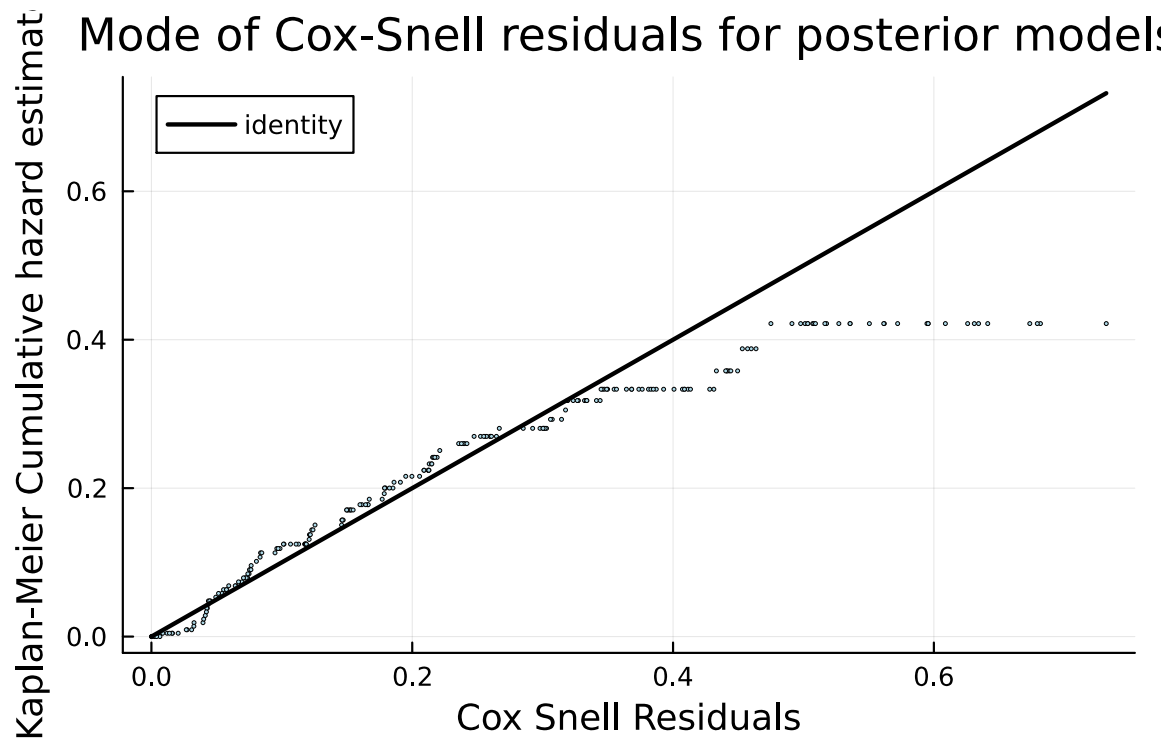
  # fit kaplan meier
  km_fit = fit(KaplanMeier, surv_times, BitVector(event_indicators))
  # sort
  surv_times_idx = [findfirst(x → x == t, km_fit.events.time) for t in surv_times]
  estimated_c_hazard_values = [-log.(km_fit.survival[i]) for i in surv_times_idx]

  p_cox_snell = scatter(cox_snell_modes, estimated_c_hazard_values,
    xlabel = "Cox Snell Residuals",
    ylabel = "Kaplan-Meier Cumulative hazard estimate",
    label = false,
    title = "Mode of Cox-Snell residuals for posterior models",
    color = :lightblue,
    # xerror = relative_errorBars,
    markersize = 1
  )
  plot!(p_cox_snell, x → x, color=:black, linewidth = 2, label = "identity")
end
plot_cox_snell(loglogistic_cox_snell_residuals_chn, surv_times, event_indicators)
```



Now comparing to Weibull

```
@model function weibull_breastcancer_cox_snell(time_obs, indicators)
  # Priors
  κ ~ truncated(LogNormal(0.5), 0.4, 2)
  λ ~ truncated(Normal(8), 0.1, 10)
  n = size(time_obs, 1)
  cox_snell_residual ~ arraydist([DiagnosticHelper(
    cumulative_hazard(HazardWeibull(κ, λ), time_obs[id])
  ) for id in 1:n])
end
weibull_model_res = weibull_breastcancer_cox_snell(surv_times, event_indicators)
weibull_cox_snell_residuals_chn = predict(weibull_model_res, chn_hazard_weibull)
plot_cox_snell(weibull_cox_snell_residuals_chn, surv_times, event_indicators)
```



Suppose we have some covariate for our data (here we just uniformly sample from $(0,1)$ to showcase the plot)

```
covariates = rand(size(g1,1),1)
```

229-element Vector{Float64}:

```
0.4769833977077421
0.3900404591881498
0.39127251042004507
0.298521913697971
0.3733602352032501
0.20816756807351144
0.4617472234823544
0.560052574239836
0.2749271905231432
0.7171704945690818
0.3452986147539191
0.7745559784764593
0.6160739902891009
⋮
```

```

0.323085079278876
0.1513891796884096
0.9389333585401363
0.30606612603964967
0.3536597442854721
0.5604234189963277
0.7501330413435687
0.8749785309266646
0.9666264558508751
0.8029164208097076
0.4646145948037529
0.4200725101093331

```

We can also plot the martingale residuals by the formulation $M = \delta - H(t)$ against this covariate, where $H(t)$ represents the cox snell residuals, here just the cumulative hazard.

```

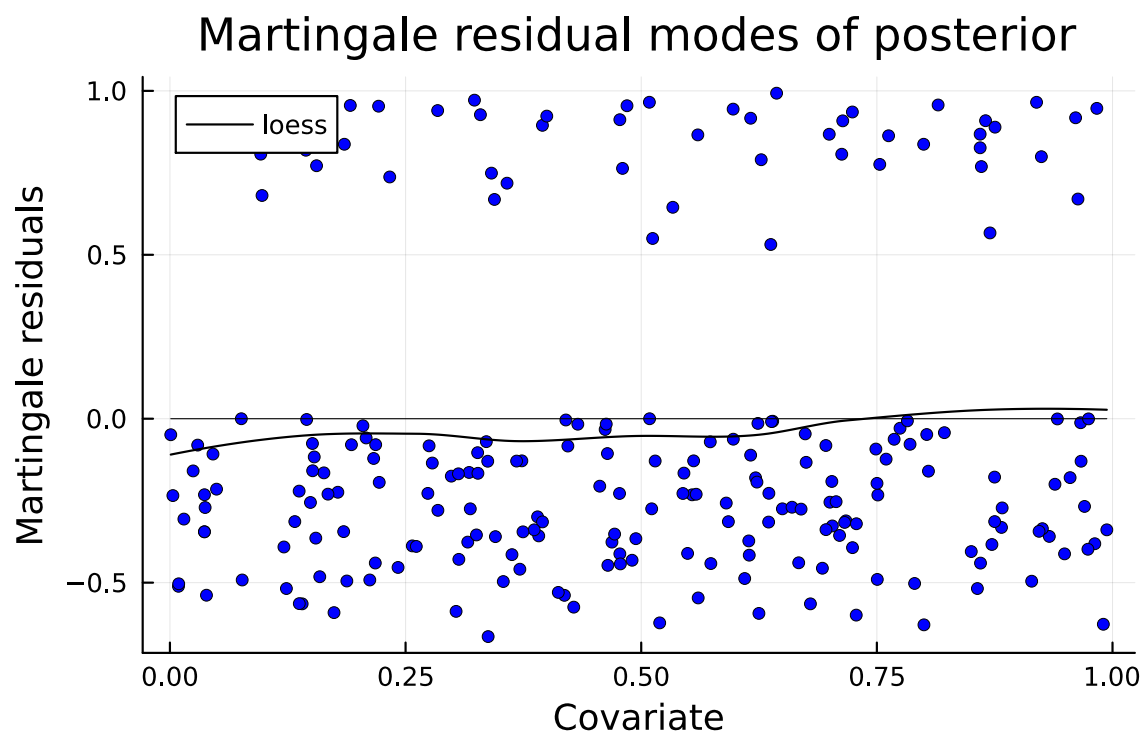
using Loess
function plot_martingale(cox_snell_residuals_chn, surv_times, event_indicators, covariates)
    cox_snell_df = DataFrame(cox_snell_residuals_chn)
    posterior_residuals = select!(cox_snell_df, r"cox_snell_residual.*")
    cox_snell_modes = mode.(eachcol(posterior_residuals))

    martingale_residual_modes = event_indicators .- cox_snell_modes

    p_martingale = scatter(covariates, martingale_residual_modes,
        xlabel = "Covariate",
        ylabel = "Martingale residuals",
        label = false,
        title = "Martingale residual modes of posterior",
        color = :blue,
        markersize = 3,
        markerstrokewidth = 0.2
    )

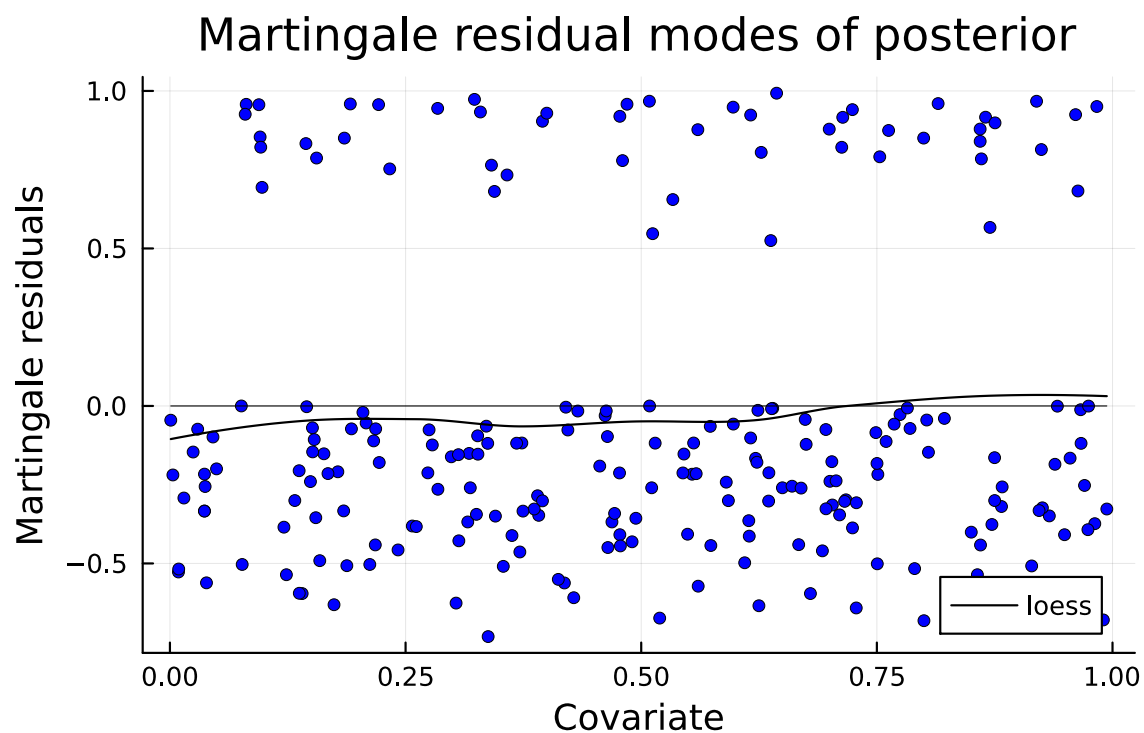
    # loess
    moving_average = loess(covariates, martingale_residual_modes)
    r = range(extrema(covariates)...,200)
    plot!(p_martingale, r, predict(moving_average, r), color = :black, linewidth = 1, label = "loess")
    plot!(p_martingale, x → 0, color = :black, linewidth = 0.5, label = false)
end
plot_martingale(loglogistic_cox_snell_residuals_chn, surv_times, event_indicators, covariates)

```



And for the hazard Weibull model

```
plot_martingale(weibull_cox_snell_residuals_chn, surv_times, event_indicators, covariates)
```



1.3.1 Posterior predictive

To showcase the posterior predictive of a posterior of a bayesian sampling I want to showcase a non-parametric plot. For this we will generate survival data via the Turing framework and then compute Kaplan-Meier estimators. My idea here is that this plotting is agnostic of the model you build. It just assumes generated survival data of your model. Above we used `predict` to generate samples for the posterior with a Weibull model.

```
display(predictions)
```

Chains MCMC chain (500×229×1 Array{Float64, 3}):

Iterations = 1:1:500

Number of chains = 1

Samples per chain = 500

parameters = time_obs[1], time_obs[2], time_obs[3], time_obs[4], time_obs[5], time_obs[6], time_obs[7]

internals =

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
time_obs[1]	7.6898	4.4448	0.2206	390.7550	387.6959	1.0038	...
time_obs[2]	7.6682	4.2649	0.1948	489.8972	510.8117	0.9999	...
time_obs[3]	7.6341	4.4817	0.2242	400.6552	374.3784	1.0005	...
time_obs[4]	7.8555	4.7834	0.2214	462.9812	438.4716	0.9993	...
time_obs[5]	7.6602	4.5027	0.2040	437.5396	331.8528	1.0010	...
time_obs[6]	7.6878	4.1974	0.1985	450.2196	437.5122	1.0003	...
time_obs[7]	7.9789	4.6927	0.2123	480.7518	488.3172	0.9990	...
time_obs[8]	7.7611	4.5905	0.2038	534.5944	500.0466	0.9989	...
time_obs[9]	8.0623	4.7136	0.2156	479.6581	450.2251	0.9993	...
time_obs[10]	7.5855	4.4464	0.1988	481.3801	452.7208	1.0032	...
time_obs[11]	7.8226	4.7259	0.2113	479.2433	409.4632	0.9990	...
time_obs[12]	7.6236	4.4459	0.1943	577.7954	470.5002	0.9990	...
time_obs[13]	7.5682	4.3185	0.1870	529.4034	473.9927	1.0013	...
time_obs[14]	7.6905	4.5002	0.2058	492.4089	444.6374	1.0013	...
time_obs[15]	7.9595	4.4725	0.2026	463.4535	425.6557	1.0082	...
time_obs[16]	7.6603	4.2850	0.1890	526.0781	473.4051	1.0009	...
time_obs[17]	7.9151	4.7856	0.2197	445.0917	276.8555	0.9994	...
time_obs[18]	7.8026	4.5780	0.2026	510.0766	456.6324	0.9985	...
time_obs[19]	7.5142	4.3433	0.2019	480.1798	410.7077	1.0011	...
time_obs[20]	7.6312	4.5896	0.2110	448.8757	436.0858	1.0026	...
time_obs[21]	7.5840	4.3839	0.2194	393.9659	411.9940	1.0095	...
time_obs[22]	7.5622	4.4535	0.2008	518.6294	406.7740	1.0022	...
time_obs[23]	7.8816	4.6153	0.2231	465.1456	358.8939	1.0051	...
:	:	:	:	:	:	:	...

1 column and 206 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
time_obs[1]	1.2829	4.5159	7.1573	10.2381	17.3845
time_obs[2]	1.1005	4.2718	7.1442	10.6876	16.6956
time_obs[3]	0.7961	4.3128	7.2392	10.2377	17.4206
time_obs[4]	0.9663	4.2639	7.1403	10.5175	18.6254
time_obs[5]	1.2997	4.1205	6.8617	10.5986	17.8534
time_obs[6]	1.1108	4.7467	7.2101	10.0726	17.4080
time_obs[7]	1.2699	4.4184	7.3678	10.7741	19.8954
time_obs[8]	1.4811	4.6330	6.8008	10.0068	19.2668
time_obs[9]	1.2644	4.4038	7.3944	10.7498	18.7482
time_obs[10]	1.1248	4.2962	6.9217	10.1305	17.9876

time_obs[11]	1.0626	4.2835	6.9338	10.6421	19.1593
time_obs[12]	1.1819	4.2188	6.8918	10.3508	17.9426
time_obs[13]	1.0395	4.2052	6.8942	10.1540	16.9788
time_obs[14]	0.9514	4.5325	6.7663	10.2075	18.0506
time_obs[15]	1.2571	4.5118	7.2716	11.0052	17.1468
time_obs[16]	0.7078	4.4267	7.3927	10.4784	16.6372
time_obs[17]	1.1923	4.4218	6.9897	10.3472	19.8677
time_obs[18]	0.9845	4.1732	7.1990	10.5089	18.1881
time_obs[19]	0.9319	4.3830	6.7718	10.2192	17.3912
time_obs[20]	0.9538	4.0334	7.1578	10.3129	17.6027
time_obs[21]	1.2329	4.2787	7.1072	10.1688	17.2408
time_obs[22]	0.9809	4.1995	6.8921	10.2420	17.7192
time_obs[23]	1.1134	4.5688	7.1365	10.6826	18.8480
:	:	:	:	:	:

206 rows omitted

For each sample in the posterior we generated 100 samples. We can estimate its survival via a Kaplan Meier estimator. We end up with 500 kaplan meier estimators. Then we plot a 95% quantile version of all these estimators at the choosen timepoints (`range(0,8,length=200)`).

```
function npe(km_fit::KaplanMeier, t::Real)
    time_points = km_fit.events.time
    survival = km_fit.survival
    if t <= time_points[1]
        return 1
    elseif t >= time_points[end]
        return survival[end]
    else
        id = findfirst(x -> x >= t, time_points)
        return survival[id]
    end
end

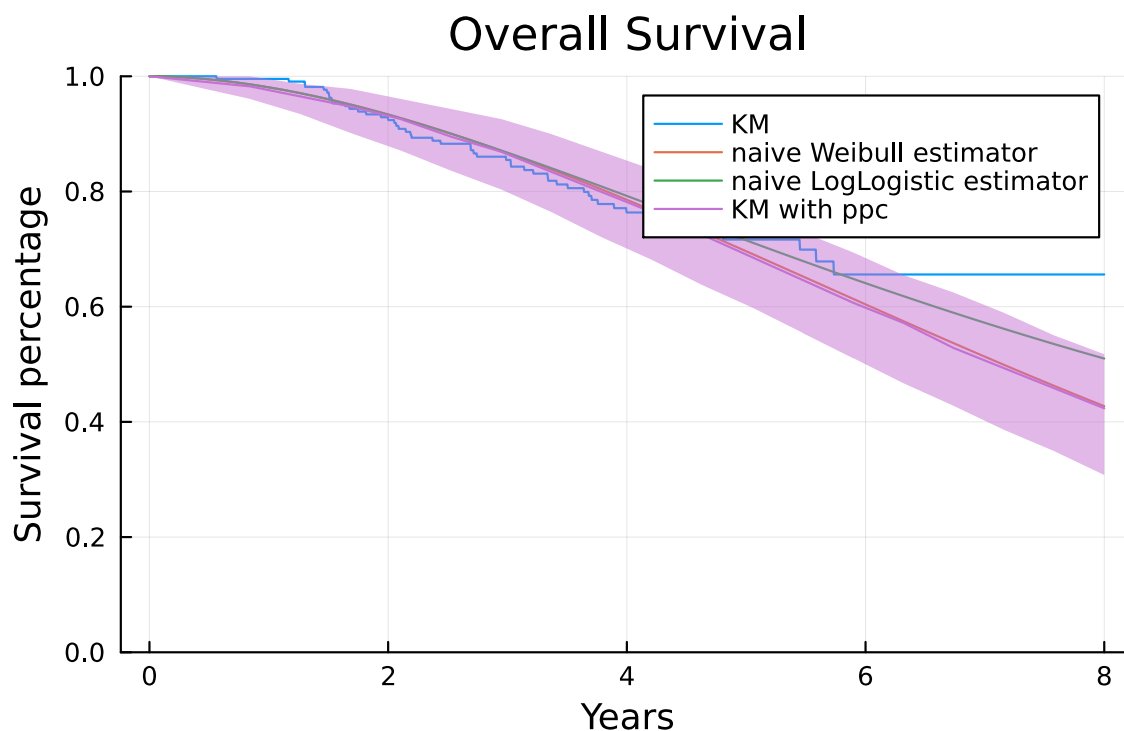
function ribbon_npe_ppc(predictions_chn, r; column_regex = r"time_obs.*", p=[0.025, 0.5, 0.975])
    chn_df = DataFrame(predictions_chn)
    predictions_df = select!(chn_df, column_regex)
    indicators = ones(size(predictions_df, 2))
    KM = [fit(KaplanMeier, row, indicators) for row in eachrow(predictions_df)]
    Q = [quantile([npe(km, time_point) for km in KM],p) for time_point in r]
    Q_mat = reduce(hcat,Q)
    return (surv = Q_mat[2,:], rib = (Q_mat[1,:], Q_mat[3,:]))
end
```

```

r = range(0,8,20)
km_fit = fit(KaplanMeier, surv_times, BitVector(event_indicators))
surv = [npe(km_fit, time_point) for time_point in r]
surv, rib = ribbon_npe_ppc(predictions, r)
ribb = (surv .- rib[1], rib[2] .- surv)

plot!(p_survival,r, surv, ribbon = ribb, label="KM with ppc")

```

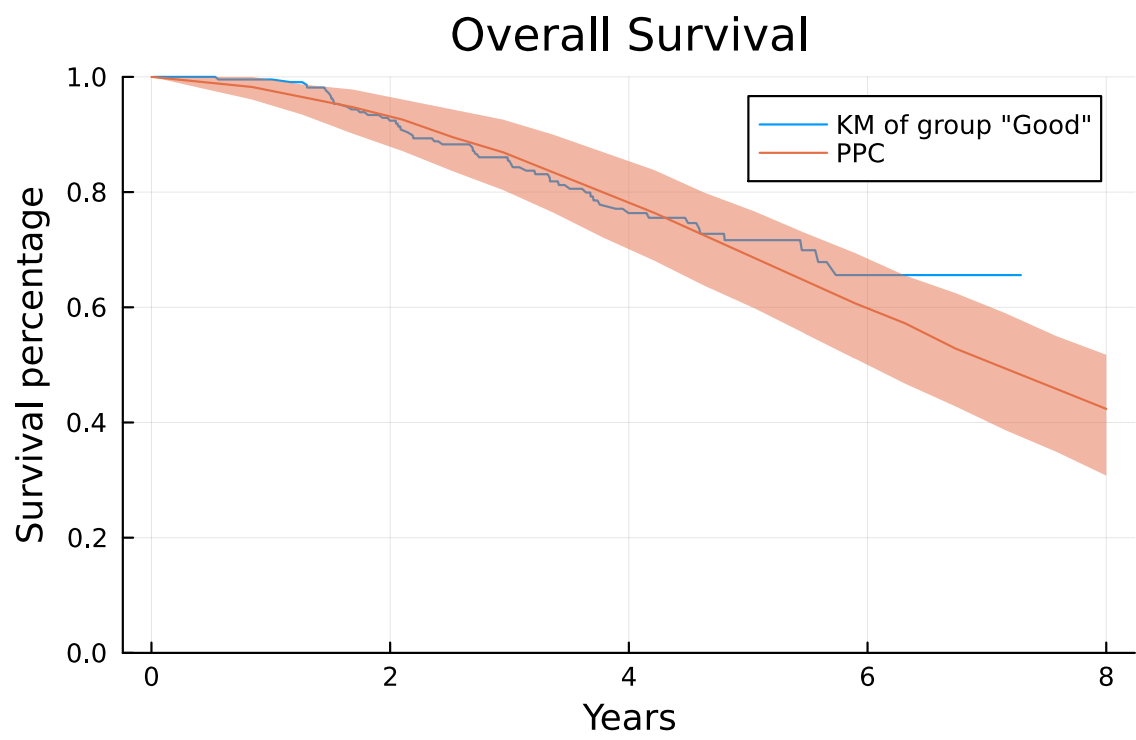


And plotting only Kaplan Meier of the observation and the ppc.

```

p_survival_ppc = plot(title = "Overall Survival", ylabel = "Survival percentage", xlabel = "Years")
km_fit = fit(KaplanMeier, g1.recyrs, BitVector(g1.censrec))
plot!(p_survival_ppc, km_fit.events.time, km_fit.survival, label="KM of group \"Good\"")
ylims!(0,1)
plot!(p_survival_ppc, r, surv, ribbon = ribb, label="PPC")
plot(p_survival_ppc)

```



A.3 Longitudinal Workflow

Longitudinal

(non linear) Longitudinal model

Table of contents

1	Lontitudinal models	1
1.0.1	Priors of complicated models	5
1.1	Longitudinal Diagnostics	9
1.2	Posterior Predictive Check	16

1 Lontitudinal models

Here we will show an example of a non-linear longitudinal models. In particular we are interested in mixed effects models (for modeling patient data) and mechanist models.

Load Some general packages

```
using Turing, Distributions, StatsPlots
```

Load orange dataset from a CSV file

```
using CSV, DataFrames
url = "https://raw.githubusercontent.com/vincentarelbundock/Rdatasets/master/csv/datasets/Orange.csv"
temp = download(url)
csv_file = CSV.File(temp)
orange = DataFrame(csv_file)
```

	Column1	Tree	age	circumference
	Int64	Int64	Int64	Int64
1	1	1	118	30
2	2	1	484	58
3	3	1	664	87
4	4	1	1004	115
5	5	1	1231	120
6	6	1	1372	142
7	7	1	1582	145
8	8	2	118	33
9	9	2	484	69
10	10	2	664	111
11	11	2	1004	156
12	12	2	1231	172
13	13	2	1372	203
14	14	2	1582	203
15	15	3	118	30
16	16	3	484	51
17	17	3	664	75
18	18	3	1004	108
19	19	3	1231	115
20	20	3	1372	139
21	21	3	1582	140
22	22	4	118	32
23	23	4	484	62
24	24	4	664	112
25	25	4	1004	167
26	26	4	1231	179
27	27	4	1372	209
28	28	4	1582	214
29	29	5	118	30
30	30	5	484	49
...	...	...	...	...

We can plot the longitudinal developments of the different trees.

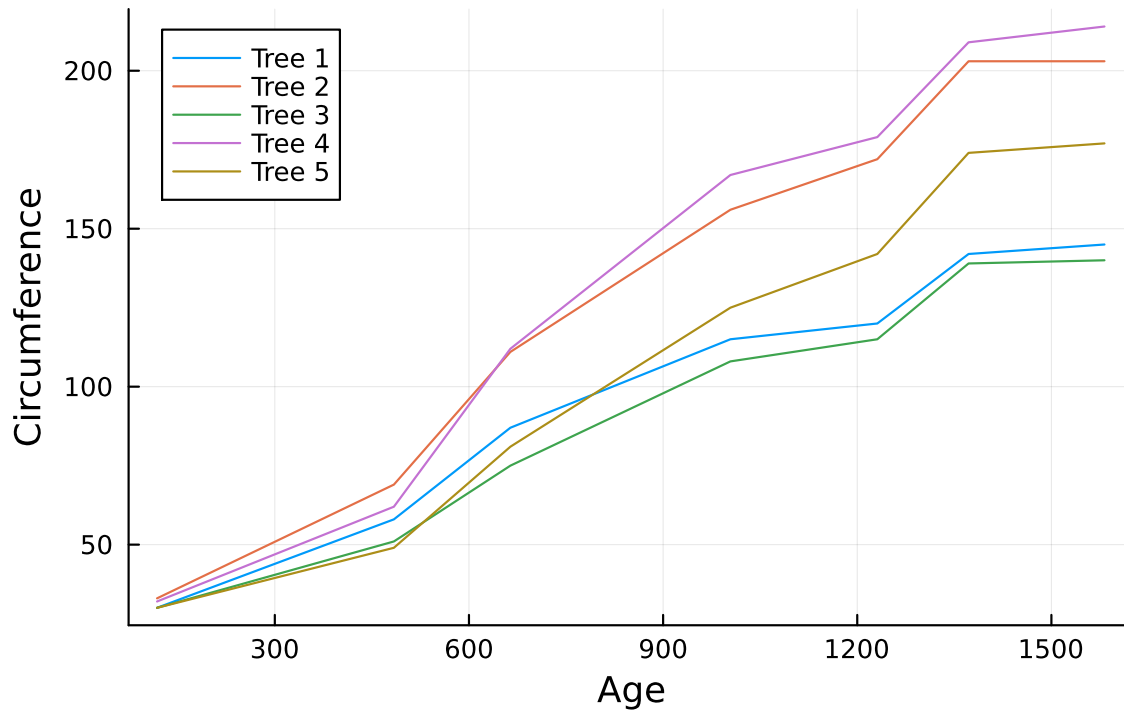
```

trees = groupby(orange, :Tree)

p = plot()
for tree in trees
    plot!(p, tree.age, tree.circumference, label = "Tree $(tree.Tree[1])")
end

```

```
plot!(p, xlabel = "Age", ylabel = "Circumference")
```



Now we can formulate a model (based on Pinheiro, Bates, and Pinheiro (1995) and Comets, Lavenu, and Lavielle (2017))

$$f_{\lambda}(x) = \frac{\lambda_1}{1 + \exp(-\frac{x-\lambda_2}{\lambda_3})}$$

in julia we write

```
pinheiro(x,λ) = λ[1] / (1 + exp(-(x-λ[2])/λ[3]))
```

pinheiro (generic function with 1 method)

Additionally we take assumption that only the intercept λ_1 varies between different trees. For example we formulate the model

```
@model function tree_model(y_age, y_circumference, y_tree ,μ = [200,800,400])
    σ ~ LogNormal(3)
    ω2 ~ TruncatedNormal(1000,200,0,Inf)
```

```

λ1 ~ filldist(Normal(μ[1], ω2[1]), 5)
λ2 ~ Normal(μ[2], 20)
λ3 ~ Normal(μ[3], 20)

for id in 1:length(y_circumference)
    tree_id = y_tree[id]
    m = pinheiro(y_age[id], [λ1[tree_id], λ2, λ3])
    y_circumference[id] ~ Normal(m, σ2)
end
end
o_ages, o_circumference, o_trees = orange.age, orange.circumference, orange.Tree
model = tree_model(o_ages, o_circumference, o_trees)
chn_orange = sample(model, NUTS(), 1000)

```

Chains MCMC chain (1000×21×1 Array{Float64, 3}):

```

Iterations      = 501:1:1500
Number of chains = 1
Samples per chain = 1000
Wall duration    = 19.69 seconds
Compute duration = 19.69 seconds
parameters       = σ, ω2, λ1[1], λ1[2], λ1[3], λ1[4], λ1[5], λ2, λ3
internals        = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
σ	2.9112	0.1956	0.0102	353.4111	384.0850	1.0013	...
ω ₂	428.2828	364.0938	36.8687	111.0248	580.0470	1.0002	...
λ ₁ [1]	172.5277	5.4608	0.3047	326.1468	395.8156	1.0021	...
λ ₁ [2]	238.7872	6.1850	0.3837	261.3365	442.4394	1.0019	...
λ ₁ [3]	163.8837	6.1110	0.3813	257.9770	393.3594	0.9997	...
λ ₁ [4]	247.9709	6.2820	0.3913	257.2429	354.2788	1.0004	...
λ ₁ [5]	198.4138	6.1074	0.3434	318.9510	432.4540	0.9999	...
λ ₂	792.0576	17.8051	1.1597	237.0089	299.5501	1.0015	...
λ ₃	393.8109	14.2899	0.8195	305.7596	371.3050	1.0008	...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64

σ	2.5856	2.7776	2.8944	3.0264	3.3357
ω_2	27.7002	64.5461	380.3437	750.1360	1081.9579
$\lambda_{-1}[1]$	161.8275	169.0456	172.4257	176.0008	183.0524
$\lambda_{-1}[2]$	226.7719	234.5385	238.9814	243.4229	250.1413
$\lambda_{-1}[3]$	153.2025	159.5362	163.4461	168.2475	176.2329
$\lambda_{-1}[4]$	235.3394	243.6461	247.9374	252.4514	260.0663
$\lambda_{-1}[5]$	185.8265	194.3787	198.6965	202.3644	209.8403
λ_{-2}	757.4845	780.1509	792.1080	803.6689	827.8064
λ_{-3}	365.6010	384.1661	394.5419	402.9742	421.4676

1.0.1 Priors of complicated models

To better understand the priors we can use the DiagnosticHelper.

```
using Random, ForwardDiff
#| code-fold: true
#| code-summary: "Disgnostic Helper"
"""
The elegance of this helper is that Turing does all the legwork for parsing the posterior samples and
If you want to change the model you have to do minimal changes to the diagnostics to still get the same
"""
struct DiagnosticHelper <: ContinuousUnivariateDistribution
    value::Real
end

function Distributions.rand(rng::AbstractRNG, dist::DiagnosticHelper)
    if typeof(dist.value) <: ForwardDiff.Dual
        return dist.value.value
    end
    return dist.value
end

function Distributions.logpdf(dist::DiagnosticHelper, t::Real)
    return -0.0001
end

Distributions.minimum(dist::DiagnosticHelper) = Inf
Distributions.maximum(dist::DiagnosticHelper) = -Inf
```

Here my idea is that conceptually you can define priors but with nonlinear models and complex mechanist models its hard to understand what this prior actually means. This should be

graphic aid for developing a sensible prior.

```
@model function tree_priors(y_age, y_circumference, y_tree,  $\mu$  = [200,800,400])
   $\sigma$  ~ LogNormal(3)
   $\omega_2$  ~ TruncatedNormal(1000,200,0,Inf)
   $\lambda_1$  ~ filldist(Normal( $\mu[1]$ , $\omega_2[1]$ ),5)
   $\lambda_2$  ~ Normal( $\mu[2]$ , 20)
   $\lambda_3$  ~ Normal( $\mu[3]$ , 20)

  l = length(y_circumference)
  M = zeros(l)
  for id in 1:l
    tree_id = y_tree[id]
    m = pinheiro(y_age[id], [ $\lambda_1$ [tree_id],  $\lambda_2$ ,  $\lambda_3$ ])
    M[id] = m
  end
  prior_pred ~ arraydist(DiagnosticHelper.(M))
end
priors_model = tree_priors(o_ages, o_circumference, o_trees)

chn_pinheiro = sample(priors_model, Prior(), 1000)
```

Chains MCMC chain (1000×45×1 Array{Float64, 3}):

```
Iterations      = 1:1:1000
Number of chains = 1
Samples per chain = 1000
Wall duration    = 1.44 seconds
Compute duration = 1.44 seconds
parameters       =  $\sigma$ ,  $\omega_2$ ,  $\lambda_1[1]$ ,  $\lambda_1[2]$ ,  $\lambda_1[3]$ ,  $\lambda_1[4]$ ,  $\lambda_1[5]$ ,  $\lambda_2$ ,  $\lambda_3$ , prior_pred[1], prior_pred[2]
internals        = lp
```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	...
Symbol	Float64	Float64	Float64	Float64	Float64	Fl ...
σ	34.1818	47.5735	1.5328	1015.6645	870.5023	0 ...
ω_2	999.2606	203.9484	6.6788	935.9743	981.1710	1 ...
$\lambda_1[1]$	259.3319	1015.1459	31.2431	1060.6365	874.6563	0 ...
$\lambda_1[2]$	173.2466	990.6744	30.4335	1064.9527	1025.7469	0 ...
$\lambda_1[3]$	187.1710	1053.1594	31.6374	1170.5152	1010.6679	0 ...
$\lambda_1[4]$	179.6115	993.4754	32.6259	924.3895	936.1210	0 ...

$\lambda_1[5]$	223.4671	1012.6806	33.0569	977.5064	944.6841	1 ...
λ_2	799.3103	19.7378	0.6213	1005.5767	1024.3146	0 ...
λ_3	399.4688	20.2104	0.6318	1023.1730	903.5186	1 ...
prior_pred[1]	40.5602	155.7692	4.8089	1057.3221	868.5186	0 ...
prior_pred[2]	81.4875	316.9440	9.7741	1056.3008	907.7013	0 ...
prior_pred[3]	108.1550	422.9201	13.0343	1055.6056	907.7013	0 ...
prior_pred[4]	161.8646	636.7811	19.6076	1058.3724	874.6563	0 ...
prior_pred[5]	193.0623	760.2449	23.4015	1059.6485	874.6563	0 ...
prior_pred[6]	208.7735	821.9173	25.2971	1060.2258	874.6563	0 ...
prior_pred[7]	226.6672	891.5280	27.4375	1061.8704	874.6563	0 ...
prior_pred[8]	27.3764	151.6893	4.6610	1068.5810	983.4446	0 ...
prior_pred[9]	54.8691	308.2294	9.4802	1060.5227	983.4446	0 ...
prior_pred[10]	72.7150	411.2542	12.6530	1062.1746	1025.7469	0 ...
prior_pred[11]	108.5171	619.6089	19.0657	1063.6771	991.7988	0 ...
prior_pred[12]	129.2480	740.2378	22.7726	1063.7433	1025.7469	0 ...
prior_pred[13]	139.6774	800.6106	24.6253	1063.9759	1025.7469	0 ...
prior_pred[14]	151.5520	868.8713	26.7170	1063.7321	1025.7469	0 ...
:	:	:	:	:	:	...

2 columns and 21 rows omitted

Quantiles

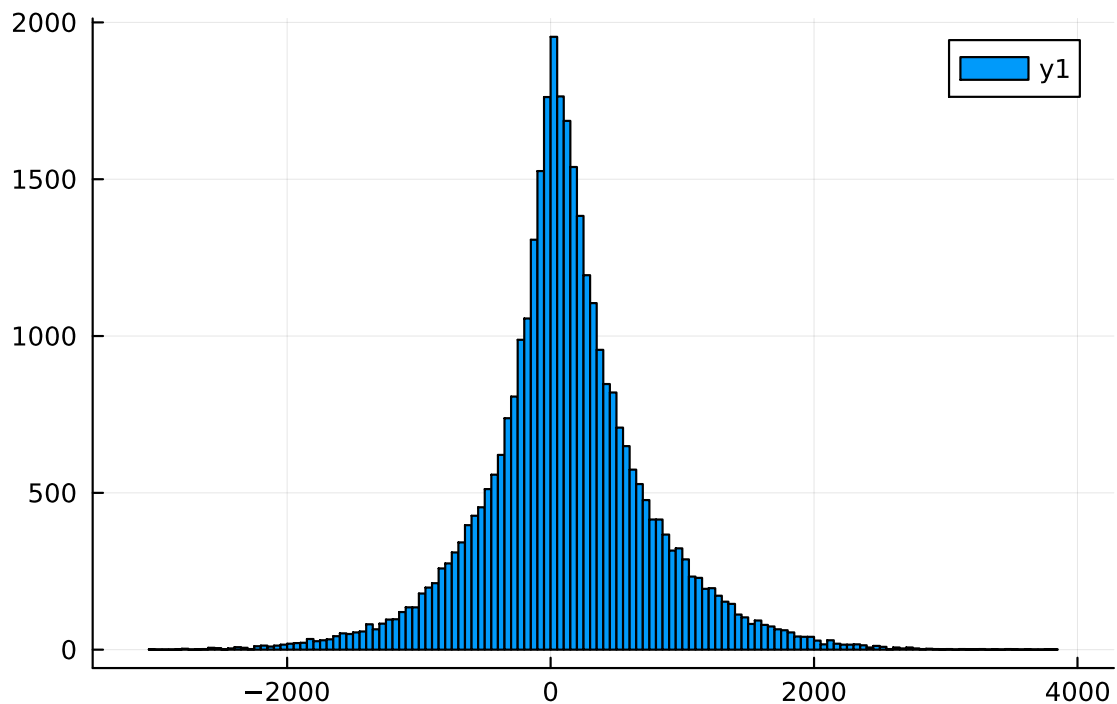
parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
σ	3.0615	9.9950	19.4222	38.1745	162.0309
ω_2	590.9267	864.1352	1004.5355	1148.1063	1385.6379
$\lambda_1[1]$	-1879.4006	-324.0031	277.2161	907.1406	2180.7339
$\lambda_1[2]$	-1767.3814	-506.7363	162.3849	798.6301	2176.7104
$\lambda_1[3]$	-1933.5621	-492.7727	168.7954	869.4045	2264.9033
$\lambda_1[4]$	-1887.5645	-423.3294	196.2055	802.3463	2106.0386
$\lambda_1[5]$	-1787.3431	-457.6290	254.2121	871.1467	2292.1924
λ_2	760.0972	786.1699	799.6306	811.9990	838.7298
λ_3	358.5116	386.3038	400.1380	413.6714	437.9721
prior_pred[1]	-290.1522	-50.2222	42.8501	138.7240	340.9901
prior_pred[2]	-599.4261	-100.7582	86.6871	279.1017	688.7577
prior_pred[3]	-787.7046	-134.7469	114.6996	372.8170	915.8075
prior_pred[4]	-1165.2533	-204.8164	174.5654	566.9772	1362.4347
prior_pred[5]	-1393.8243	-243.7228	208.8916	679.6837	1632.7541
prior_pred[6]	-1508.4385	-262.0315	225.9933	736.2273	1767.8367
prior_pred[7]	-1643.8508	-283.0052	245.0096	799.7676	1918.1037
prior_pred[8]	-264.8296	-77.6396	25.1338	122.6300	329.1407
prior_pred[9]	-549.6385	-158.4845	51.4858	249.3106	698.5110
prior_pred[10]	-723.5210	-212.6747	67.9651	330.2411	925.9683

prior_pred[11]	-1078.5324	-316.6077	100.7720	499.0439	1384.7407
prior_pred[12]	-1297.2853	-378.3956	119.2312	594.9743	1652.1765
prior_pred[13]	-1400.7108	-409.1014	128.7082	642.4997	1781.2041
prior_pred[14]	-1523.7706	-444.6243	140.0781	696.6615	1930.0705
:	:	:	:	:	:

21 rows omitted

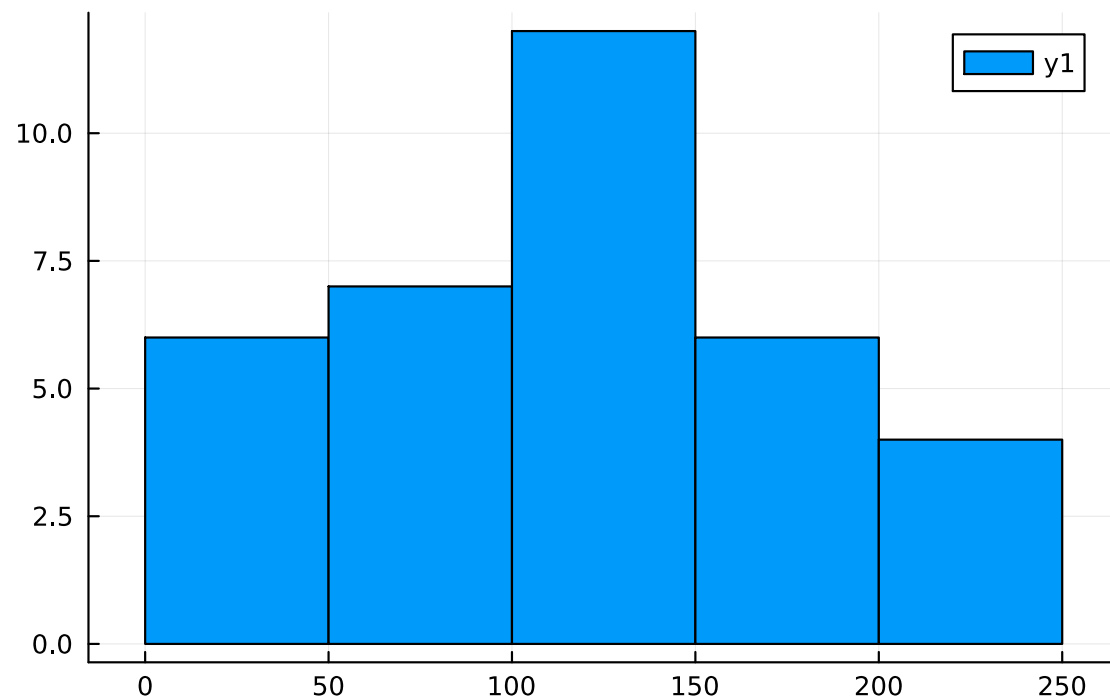
Now we can showcase the model predictions based on the priors we have defined as a histogram.

```
df = DataFrame(chn_pinheiro)
select!(df, r"prior_pred.*")
prior_pred = reduce(vcat, eachcol(df))
histogram(prior_pred)
```



This histogram describes what the model does on 1000 samples of the prior. In this case it does not make sense for the predictions to be negative and we might want to change the priors to reflect that. But at least the order of magnitude seems to be similar to the observations.

```
histogram(o_circumference)
```



1.1 Longitudinal Diagnostics

First we will calculate residuals using the Turing framework. We save the individual predictions (IPRED) using the DiagnosticHelper.

```
@model function tree_pred_residuals(y_age, y_circumference, y_tree, μ = [200,800,400])
  σ ~ LogNormal(3)
  ω2 ~ TruncatedNormal(1000,200,0,Inf)
  λ_1 ~ filldist(Normal(μ[1],ω2[1]),5)
  λ_2 ~ Normal(μ[2], 20)
  λ_3 ~ Normal(μ[3], 20)

  l = length(y_circumference)
  P = zeros(l)

  for id in 1:l
    tree_id = y_tree[id]
    m = pinheiro(y_age[id], [λ_1[tree_id], λ_2, λ_3])
    P[id] = m
  end
end
```

```

        y_circumference[id] ~ Normal(m,  $\sigma^2$ )
    end
    IPRED ~ arraydist(DiagnosticHelper.(P))
end
missing_circ = Vector{Union{Missing, Float64}}(undef, size(orange, 1))
pred_model = tree_pred_residuals(o_ages, missing_circ, o_trees)
chn_pred = predict(pred_model, chn_orange)

```

Chains MCMC chain (1000×70×1 Array{Float64, 3}):

```

Iterations      = 1:1:1000
Number of chains = 1
Samples per chain = 1000
parameters      = y_circumference[1], y_circumference[2], y_circumference[3], y_circumference[4], y_c
internals       =

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	...
Symbol	Float64	Float64	Float64	Float64	Float64	...
y_circumference[1]	26.5215	8.6051	0.2799	944.2586	793.4008	...
y_circumference[2]	54.1790	8.8550	0.2833	971.2569	761.7017	...
y_circumference[3]	72.5558	8.7563	0.2824	948.9306	894.3847	...
y_circumference[4]	109.0101	9.1954	0.2826	1053.6697	980.8705	...
y_circumference[5]	130.2090	9.5985	0.3329	823.6700	872.9888	...
y_circumference[6]	140.5166	9.7109	0.3303	861.7126	845.2239	...
y_circumference[7]	152.4904	9.3927	0.3240	846.0797	1004.2424	...
y_circumference[8]	36.9108	8.8258	0.3167	792.9519	613.7225	...
y_circumference[9]	75.5994	8.8512	0.3053	843.0140	781.6023	...
y_circumference[10]	100.1232	9.0817	0.3012	911.8562	893.9467	...
y_circumference[11]	150.9157	9.5412	0.3686	667.7489	867.0224	...
y_circumference[12]	179.9576	9.6575	0.3819	637.5660	926.2450	...
y_circumference[13]	193.7729	9.8538	0.3719	701.6080	793.0973	...
y_circumference[14]	210.5466	9.7528	0.3678	697.1658	820.8215	...
y_circumference[15]	25.0709	8.6021	0.2913	874.0917	942.0177	...
y_circumference[16]	51.6233	8.6133	0.2843	917.1268	894.4681	...
y_circumference[17]	68.3478	9.2146	0.3161	847.9604	914.1091	...
y_circumference[18]	104.0866	9.1597	0.3190	825.5621	817.8880	...
y_circumference[19]	123.0982	9.6918	0.3466	792.3981	868.2257	...
y_circumference[20]	133.1231	9.2696	0.3143	881.3058	902.4289	...
y_circumference[21]	144.2391	9.7977	0.3423	816.0424	843.2654	...
y_circumference[22]	37.7523	8.9160	0.2843	964.7803	883.3418	...

```

y_circumference[23]  77.0321    9.1271    0.2950    962.8464    903.5186    ...
      :                :                :                :                :                \
                                2 columns and 47 rows omitted

```

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
y_circumference[1]	9.7005	20.7404	26.3107	32.0589	43.8453
y_circumference[2]	36.4184	48.5039	54.2827	60.2050	70.9250
y_circumference[3]	55.9055	66.3269	72.5382	78.5190	89.2130
y_circumference[4]	90.8475	103.2145	109.0678	115.3891	126.4630
y_circumference[5]	111.1781	123.4714	130.1056	136.6357	149.0862
y_circumference[6]	121.5152	134.2661	140.6000	146.6835	159.4777
y_circumference[7]	133.9035	146.5470	152.4073	158.7100	170.8422
y_circumference[8]	20.3544	31.1093	36.5021	42.9218	54.9720
y_circumference[9]	58.6784	69.8821	75.5652	81.6596	92.3852
y_circumference[10]	83.1741	94.2583	99.8258	106.5963	117.8931
y_circumference[11]	131.9934	144.3521	151.0461	157.3351	169.6298
y_circumference[12]	161.0377	173.4596	180.2511	186.3244	197.5819
y_circumference[13]	175.2473	187.1720	193.8054	200.6736	213.3502
y_circumference[14]	190.4491	204.3913	210.1902	216.9678	229.6988
y_circumference[15]	8.7509	19.7775	24.8200	30.8274	42.0969
y_circumference[16]	34.5864	45.7086	51.7676	57.1379	68.5246
y_circumference[17]	50.0353	62.4246	67.9550	74.3148	86.0061
y_circumference[18]	86.2917	97.8435	103.7083	109.9449	122.3930
y_circumference[19]	102.3042	116.9163	123.6879	129.1289	141.9077
y_circumference[20]	114.9200	126.9483	133.4762	139.3110	151.8404
y_circumference[21]	124.8820	137.9029	144.3581	150.7603	162.9066
y_circumference[22]	18.8792	31.9214	38.0320	43.4336	55.0873
y_circumference[23]	58.7280	71.0257	76.9644	83.2128	95.6697
:	:	:	:	:	:

47 rows omitted

Getting the IPRED values out of the chains object **without** casting to a DataFrame first.

```

# get is a Turing functionality
ipred = get(chn_pred, :IPRED)
# here we use collect to cast the named tuple into a Vector
ipred_values = collect(ipred.IPRED)
# broadcast mode (this works with tuples but plotting does not!!)
IPRED_modes = mode.(ipred_values)

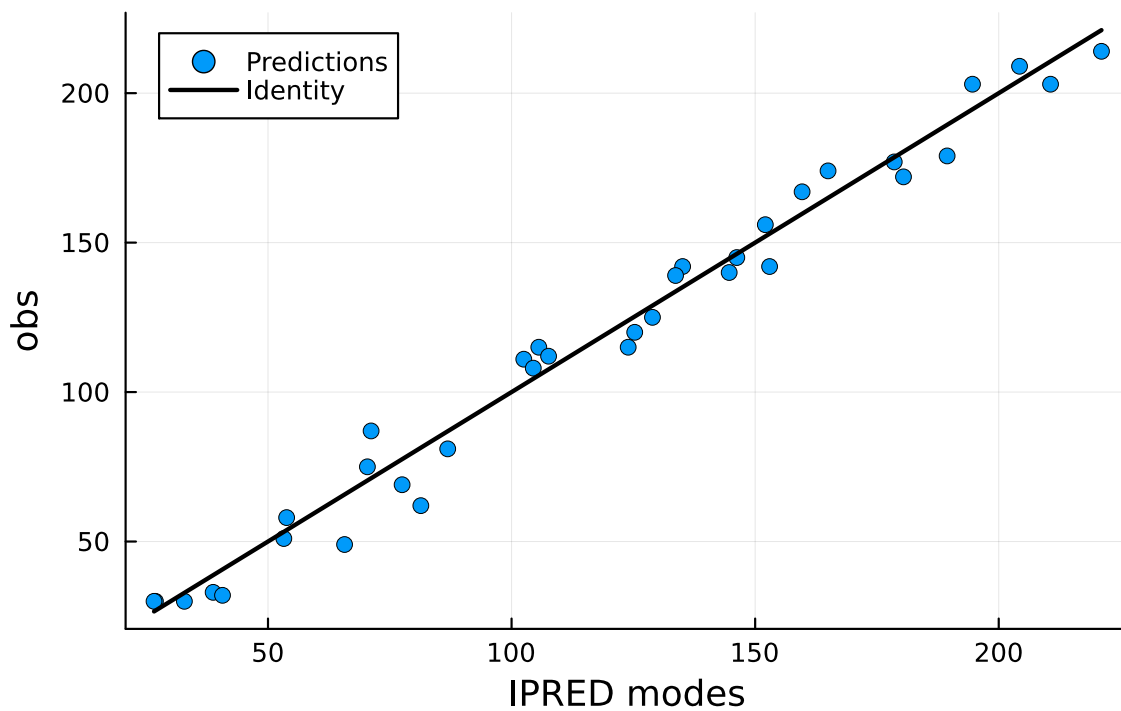
```

35-element Vector{Float64}:

```
26.89494892411376
53.82483607418825
71.16617548812911
105.58715053141226
125.27723370312343
135.1089668183206
146.23380677224475
38.739342181375804
77.52900919863518
102.50738277114311
152.08717316898944
180.44866482755015
194.61024120608334
⋮
107.59540568899561
159.6361223439269
189.40535572451958
204.2698514754824
221.0893821002141
32.83786980506807
65.71839806316224
86.89161715077759
128.91852340087286
152.9594832654181
164.96371398197294
178.54678666375975
```

Plotting IPRED vs Obs

```
scatter(IPRED_modes, o_circumference,
        label="Predictions",
        xlabel="IPRED modes",
        ylabel="obs"
)
plot!(identity, color=:black, linewidth = 2, label="Identity")
```



Calculating the residuals

```
IRES_modes = IPRED_modes .- o_circumference
```

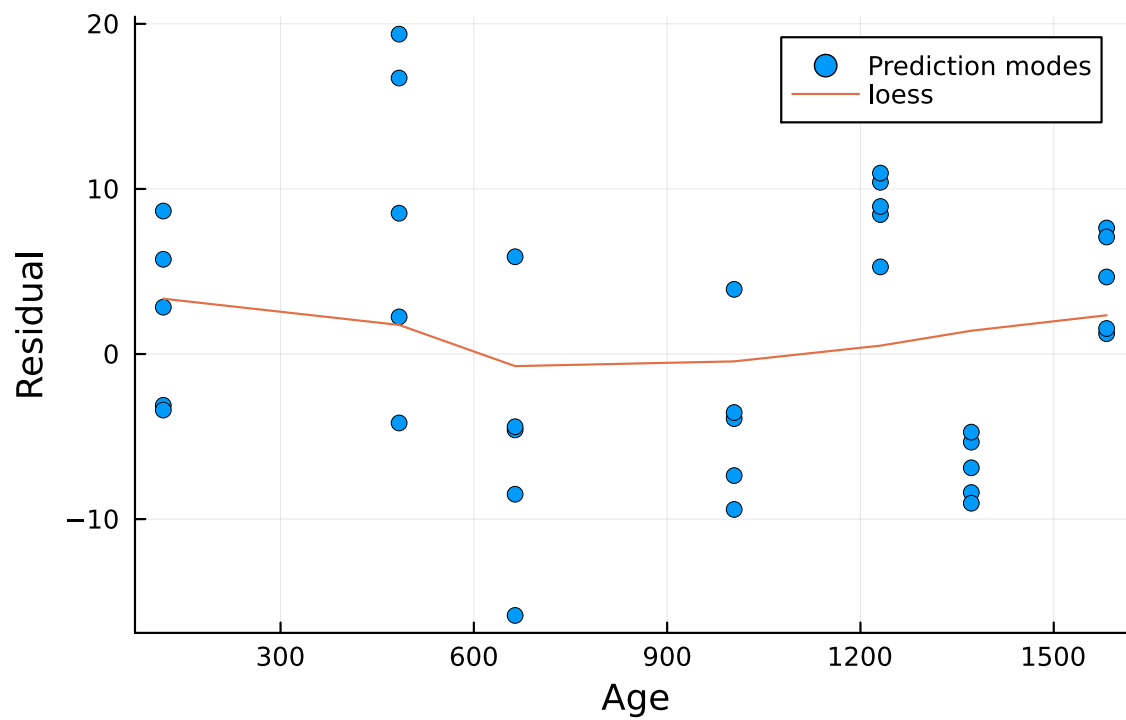
35-element Vector{Float64}:

```
-3.105051075886241
-4.175163925811752
-15.833824511870887
-9.412849468587737
 5.277233703123429
-6.891033181679404
 1.2338067722447477
 5.739342181375804
 8.52900919863518
-8.492617228856886
-3.9128268310105625
 8.448664827550147
-8.389758793916656
 ⋮
-4.404594311004388
```

```
-7.3638776560731
10.40535572451958
-4.7301485245176025
 7.089382100214095
 2.837869805068067
16.71839806316224
 5.891617150777591
 3.918523400872857
10.9594832654181
-9.036286018027056
 1.5467866637597467
```

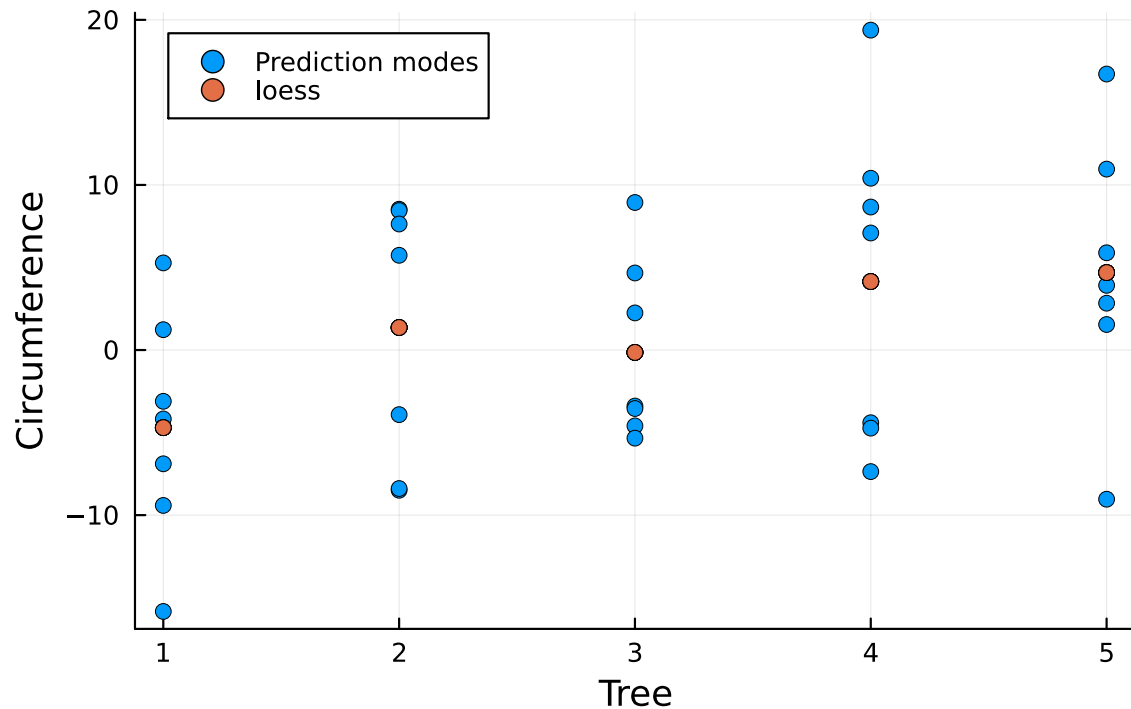
Plotting IRES vs the age

```
using Loess
# vs age
scatter(orange.age, IRES_modes,
        label="Prediction modes",
        xlabel="Age",
        ylabel="Residual"
)
x = sort(orange.age)
plot!(x, predict(loess(orange.age, IRES_modes),x), label="loess")
```



Plotting the IRES per tree

```
# per tree
scatter(orange.Tree, IRES_modes,
  label="Prediction modes",
  xlabel="Tree",
  ylabel="Circumference"
)
x = sort(orange.Tree)
scatter!(x, predict(loess(orange.Tree, IRES_modes),x), label="loess")
```



1.2 Posterior Predictive Check

For the posterior predictive check we can simulate the development according to the model and the posterior. Here we need to think the error model and how we want to include this. When we generate measurement values with turing with the same code as when sampling the posterior then Turing will sample (i.e. call rand) on the distribution on the RHS of the `~` syntax. This will include the error model we use.

To have the actual measurements of the model based on the posterior there are two options. Either we use the `DiagnosticHelper` so save the model values similar to how we did it for examining the prior. Or we manually set the error to a very small number or even zero if possible. This will cause the distribution of the measurements to be a spike.

The only difference we do compared to IPRED is that we will simulate values at other time points. For example we can choose 50 timepoints in the range of 0 to 1700. This is achieved by giving these timepoints for `y_age` and accordingly many `y_circumference` per tree.

```
@model function tree_pred_residuals(y_age, y_circumference, y_tree, μ = [200,800,400])
  σ ~ LogNormal(3)
  ω2 ~ TruncatedNormal(1000,200,0,Inf)
```

```

λ1 ~ filldist(Normal(μ[1], ω2[1]), 5)
λ2 ~ Normal(μ[2], 20)
λ3 ~ Normal(μ[3], 20)

l = length(y_circumference)
P = zeros(l)

for id in 1:l
    tree_id = y_tree[id]
    m = pinheiro(y_age[id], [λ1[tree_id], λ2, λ3])
    P[id] = m
    # Setting error to 0
    y_circumference[id] ~ Normal(m, 0)
end

# saving with diarc distribution i.e. diagnostic helper
IPRED ~ arraydist(Dirac.(P))
end

tree_ids = unique(orange.Tree)
ages = collect(range(0, 1700, length = 50))
ppc_ages = repeat(ages, length(tree_ids))
ppc_circ = Vector{Union{Missing, Float64}}(undef, size(ppc_ages, 1))
ppc_trees = reduce(vcat, fill.(tree_ids, length(ages)))
ppc_model = tree_pred_residuals(ppc_ages, ppc_circ, ppc_trees)
chn_ppc = predict(ppc_model, chn_orange)

```

Chains MCMC chain (1000×500×1 Array{Float64, 3}):

```

Iterations      = 1:1:1000
Number of chains = 1
Samples per chain = 1000
parameters      = y_circumference[1], y_circumference[2], y_circumference[3], y_circumference[4], y_c
internals       =

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	...
Symbol	Float64	Float64	Float64	Float64	Float64	F ...
y_circumference[1]	20.3616	1.4545	0.0697	437.8816	524.8824	...
y_circumference[2]	21.9959	1.5072	0.0719	441.0055	541.4275	...
y_circumference[3]	23.7411	1.5587	0.0739	445.4082	541.4275	...
y_circumference[4]	25.6015	1.6091	0.0759	450.8391	550.0226	...
y_circumference[5]	27.5809	1.6580	0.0778	455.3899	577.0110	...

y_circumference[6]	29.6827	1.7052	0.0796	460.3656	602.1363	...
y_circumference[7]	31.9097	1.7508	0.0813	465.1739	617.7657	...
y_circumference[8]	34.2640	1.7947	0.0829	470.3670	602.1363	...
y_circumference[9]	36.7468	1.8371	0.0844	474.8510	552.6359	...
y_circumference[10]	39.3587	1.8780	0.0859	480.3422	525.7278	...
y_circumference[11]	42.0990	1.9178	0.0873	484.6939	467.3928	...
y_circumference[12]	44.9659	1.9571	0.0887	488.0334	476.7627	...
y_circumference[13]	47.9565	1.9964	0.0901	491.9472	433.4725	...
y_circumference[14]	51.0667	2.0363	0.0916	495.5789	387.1864	...
y_circumference[15]	54.2909	2.0776	0.0932	500.2674	373.4138	...
y_circumference[16]	57.6223	2.1211	0.0949	503.0408	373.4138	...
y_circumference[17]	61.0528	2.1676	0.0967	505.5954	396.5879	...
y_circumference[18]	64.5729	2.2179	0.0988	507.0664	427.5617	...
y_circumference[19]	68.1719	2.2726	0.1012	507.1165	451.3732	...
y_circumference[20]	71.8382	2.3321	0.1037	507.5602	415.8648	...
y_circumference[21]	75.5588	2.3968	0.1066	508.4275	421.0318	...
y_circumference[22]	79.3203	2.4666	0.1096	508.7512	420.4209	...
y_circumference[23]	83.1084	2.5415	0.1129	507.9181	428.3132	...
:	:	:	:	:	:	...

2 columns and 477 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
y_circumference[1]	17.4398	19.3868	20.3576	21.3941	22.9789
y_circumference[2]	18.9641	20.9874	21.9867	23.0708	24.6926
y_circumference[3]	20.5781	22.6952	23.7306	24.8322	26.5447
y_circumference[4]	22.2959	24.5200	25.6111	26.7328	28.4880
y_circumference[5]	24.1641	26.4479	27.6009	28.7539	30.5622
y_circumference[6]	26.1494	28.5103	29.6947	30.8848	32.7450
y_circumference[7]	28.3038	30.7082	31.8852	33.1597	35.1485
y_circumference[8]	30.5185	33.0172	34.2448	35.5222	37.5645
y_circumference[9]	32.9171	35.5088	36.7047	38.0100	40.1840
y_circumference[10]	35.3995	38.1078	39.3372	40.6340	42.9285
y_circumference[11]	38.1032	40.7996	42.0629	43.3894	45.7287
y_circumference[12]	40.9583	43.6749	44.9454	46.2840	48.6440
y_circumference[13]	43.8227	46.5930	47.9328	49.3106	51.7462
y_circumference[14]	46.8204	49.6790	51.0268	52.4255	54.9864
y_circumference[15]	49.9907	52.8785	54.2817	55.6520	58.3194
y_circumference[16]	53.2828	56.1979	57.6226	58.9887	61.7574
y_circumference[17]	56.6504	59.5734	61.0169	62.4646	65.2846
y_circumference[18]	60.1054	63.0796	64.5292	65.9971	68.9684

y_circumference[19]	63.6037	66.6597	68.1202	69.6446	72.7765
y_circumference[20]	67.1731	70.3035	71.7811	73.3698	76.5789
y_circumference[21]	70.8042	73.9619	75.5268	77.1159	80.4364
y_circumference[22]	74.4891	77.6824	79.2574	80.9647	84.2714
y_circumference[23]	78.1883	81.4433	83.0360	84.8046	88.3156
:	:	:	:	:	:

477 rows omitted

Both the IPRED and y_circumference contain the same data.

```
test_df = DataFrame(chn_ppc)
A = Matrix(select(test_df, r"IPRED"))
B = Matrix(select(test_df, r"y_circumference"))
using LinearAlgebra # standart library
norm(A - B)
```

0.0

With these predictions we can plot the quantiles of the posterior distribution to give error bands. Let us create such ppc plot for each individual. This now depends greatly on the setup of the generated values- (specifically times ages, ppc_ages and trees ppc_trees) and thus cannot be done agnostically of that. (Notice that this data actually is inside of model_ppc, thus it is possible to create a function for this plot using the chain output and model, but this needs naming conventions etc.!!)

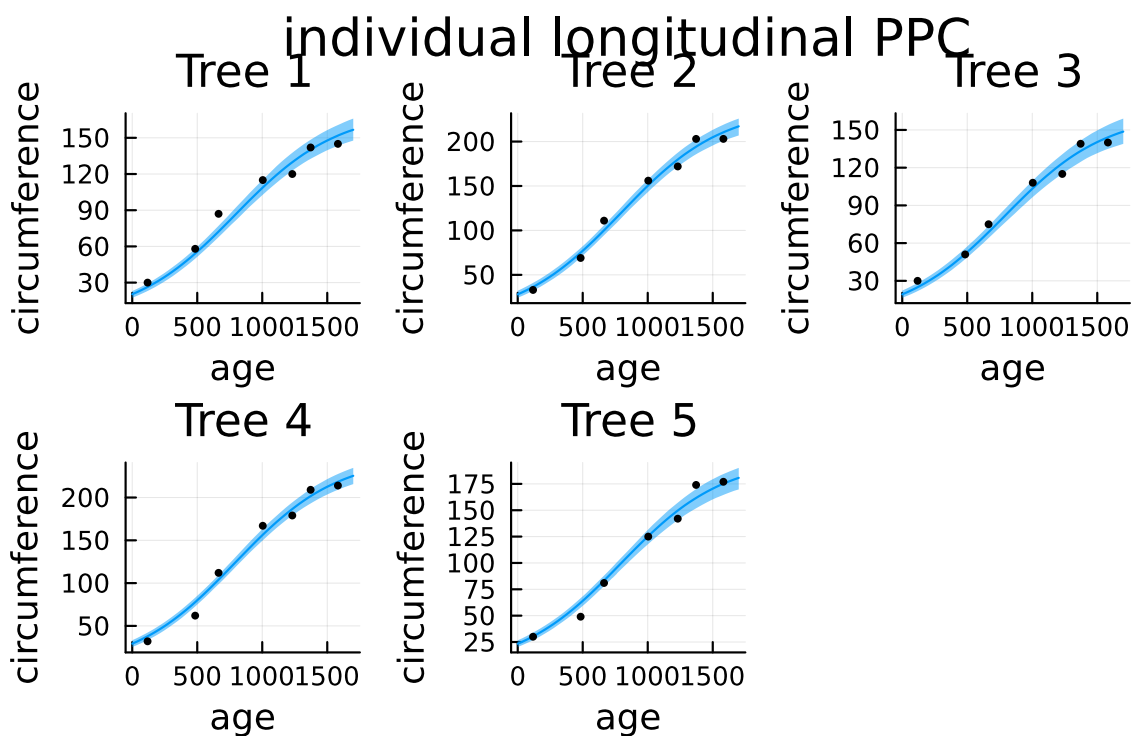
```
# only select ipred values form chain
chn_ipred = group(chn_ppc, :IPRED)
ipred_quantiles = quantile(chn_ipred)
quantiles_df = DataFrame(ipred_quantiles)
tree_plots = []
for tree in tree_ids
    df_ids = ppc_trees .== tree
    lower_q = quantiles_df[df_ids,2]
    median_q = quantiles_df[df_ids,4]
    upper_q = quantiles_df[df_ids,6]
    lower_ribbon = median_q - lower_q
    upper_ribbon = upper_q - median_q

    p = plot(ages, median_q,
             ribbon = (lower_ribbon, upper_ribbon),
             title = "Tree $tree",
```

```

        xlabel = "age",
        ylabel = "circumference",
        legend=false
    )
    # broadcast .== gives bitvector
    orange_ids = orange.Tree .== tree
    scatter!(p, orange.age[orange_ids], orange.circumference[orange_ids],
        markersize = 2,
        color = :black
        #label = "observations"
    )
    push!(tree_plots, p)
end
plot(tree_plots..., plot_title="individual longitudinal PPC", )

```



This plot shows for every tree the observations as well as the posterior descriptiveness of the model. To showcase the posterior predictions we plot a line for the median and a ribbon representing the 95% quantiles of the posterior predictions.

Comets, Emmanuelle, Audrey Lavenu, and Marc Lavielle. 2017. "Parameter Estimation in

Nonlinear Mixed Effect Models Using **saemix**, an *r* Implementation of the SAEM Algorithm.” *Journal of Statistical Software* 80 (3). <https://doi.org/10.18637/jss.v080.i03>.

Pinheiro, José C., Douglas M. Bates, and Jose C. Pinheiro. 1995. “Approximations to the Log-Likelihood Function in the Nonlinear Mixed-Effects Model.” *Journal of Computational and Graphical Statistics* 4 (1): 12. <https://doi.org/10.2307/1390625>.

A.4 Joint Molder Workflow

Joint Models

Joint Models in Turing

Table of contents

1	Build joint models	1
1.0.1	Example implementation of Joint Models	2
2	Sampling in Turing	6
2.0.1	Fitting Longitudinal part	12
2.0.2	Fitting Survival part	15
2.1	Joint models with a link function / operator	17
2.2	Diagnostics	32

Load general packages

```
using Turing, Distributions, StatsPlots, CSV, DataFrames
using LogExpFunctions: logit
```

1 Build joint models

As an example how to implement Joint Models and use bayesian sampling to sample its posterior we will reconstruct a joint model defined in Keroui et al. (2020) which has a longitudinal model called SLD: sum of longest diameters. Here $\text{BSLD} \in \mathbb{R}$ is the baseline SLD at study start t_x . We have $\Psi = (\text{BSLD}, g, d, \varphi)$.

$$\text{SLD}(t, \Psi) = \begin{cases} \text{BSLD} \exp(gt) & \text{if } t < t_x \\ \text{BSLD} \exp(gt_x) \cdot [\varphi \exp(-d(t - t_x)) + (1 - \varphi) \exp(g(t - t_x))] & \text{if } t \geq t_x \end{cases}$$

In Julia we define

```

function sld(t, Ψ, tx = 0.0)
  BSLD, g, d, φ = Ψ
  Δt = t - tx
  if t < tx
    return BSLD * exp(g * t)
  else
    return BSLD * exp(g * tx) * (φ * exp(-d * Δt) + (1 - φ) * exp(g * Δt))
  end
end

```

sld (generic function with 2 methods)

For the survival model they define the joint model with the hazard

$$h_i(t | \text{SLD}(t, \Psi_i)) = h_0(t) \exp[\beta \times f(\text{SLD}(t, \Psi_i))].$$

with a Weibull baseline hazard $h_0(t) = \frac{\kappa}{\lambda} (\frac{t}{\lambda})^{\kappa-1}$ and a link function f that operates on the space that contains the SLD functions.

Some examples of link functions are id , $\frac{d}{dt}$ or a link function that gives the TTG (time-to-growth)

$$\text{TTG}(\Psi) = \max \left\{ 0, \frac{\log(\frac{d \cdot \varphi}{g(1-\varphi)})}{g + d} + t_x \right\}$$

Notice that f is an operator that acts on a function space containing all the SLD formulations. In the case of TTG we do not actually know how f looks like, we only have the formulation of the image $f_{\text{TTG}}(\text{SLD}(t, \Psi)) = \text{TTG}(\Psi)$ of the SLD function under f_{TTG} .

1.0.1 Example implementation of Joint Models

We start with the same abstract type already introduced in the survival page.

```

using Integrals, DifferentialEquations, Random, Roots, Distributions

abstract type HazardBasedDistribution <: ContinuousUnivariateDistribution end

@doc raw"""
represents ``h(t)``
needs to be implemented for any `struct` that subtypes HazardBasedDistribution
"""
function hazard(dist::HazardBasedDistribution, t::Real) end

```

```

@doc raw"""
Different argument order to comply with sciml (Integration)
"""
sciml_hazard(t::Real, dist::HazardBasedDistribution) = hazard(dist, t)

@doc raw"""
calculates  $H(t) = \int_0^t h(u) \, du$  numerically with a Gauss-Konrad procedure
"""
function cumulative_hazard(dist::HazardBasedDistribution, t::Real)
    # integrate from 0.0001 to $t$ (some hazards are not defined at 0)
    /h = IntegralProblem(sciml_hazard, 1e-4, float(t), dist)
    H = 0
    try
        H = solve(/h, QuadGKJL())[1]
    catch e
        # ignore DomainErrors, can happen in initial sampling steps
        if !isa(e, DomainError)
            throw(e)
        end
    end
    return H
end

@doc raw"""
Calculation of the ccdf / survival function at time `t` based on the cumulative hazard  $S(t) = \exp(-H(t))$ 
"""
function Distributions.logccdf(dist::HazardBasedDistribution, t::Real)
    - cumulative_hazard(dist, t)
end

@doc raw"""
Calculation of the log pdf function at time `t` based on the cumulative hazard  $\log f(t) = h(t) - H(t)$ 
"""
function Distributions.logpdf(dist::HazardBasedDistribution, t::Real)
    log(hazard(dist, t)) - cumulative_hazard(dist, t)
end

@doc raw"""
Calculation of the pdf function at time `t` based on the log pdf
"""
function Distributions.pdf(dist::HazardBasedDistribution, t::Real)

```

```

        exp(logpdf(dist, t))
    end

H(nLogS, p, t) = sciml_hazard(t,p)
@doc raw"""
Generate a random sample ``t \sim \text{dist}``
"""
function Distributions.rand(rng::AbstractRNG, dist::HazardBasedDistribution)
    num_end = 10_000
    # calculate ode solution of  $H(t) = \int_0^t h(u) \, du$ 
    prob = ODEProblem(H, 0.0, (1e-4, num_end))
    sol = solve(prob, Tsit5()); p=dist
    # calculate cdf  $F(t)$ , via survival  $S(t)$ 
    S(t) = exp(-sol(t))
    F(t) = 1 - S(t)
    # inverse sampling: find  $t_0$  such that  $F(t_0) = x_0$  where  $x_0 \sim U(0,1)$ 
    # via the root of  $g(t) := F(t) - x_0$ 
    x0 = rand()
    g(t) = F(t) - x0
    try
        return find_zero(g, (1e-4, num_end))
    catch e
        return 0
    end
end

Distributions.minimum(d::HazardBasedDistribution) = 0.
Distributions.maximum(d::HazardBasedDistribution) = Inf

```

To make use of the distributions provided in the Distributions package I define the hazard for them computationally with the pdf and survival ccdf.

```

"""
Calculates the hazard ``h(t) = f(t)/S(t)`` based on the formulations of `pdf` and the survival function.
"""
hazard(dist::ContinuousUnivariateDistribution, t::Real) = pdf(dist, t) / ccdf(dist, t)

```

hazard

With this code we can implement a simple joint model formulation:

```

struct SimpleJointModel <: HazardBasedDistribution
    base_survival::ContinuousUnivariateDistribution
    β::Real
    link::Function # function of time
end

```

In this formulation the struct contains a baseline survival distribution, the link coefficient β and a function `link(t)` that should represent $t \mapsto f(\text{SLD}(t, \Psi_i))$.

The JointModel hazard for SimpleJointModel we formulate in julia as

```

h₀(d::SimpleJointModel, t::Real) = hazard(d.base_survival, t)

function hazard(d::SimpleJointModel, t::Real)
    link_val = d.link(t)
    # prevents infinity calculations
    exp_val = minimum([d.β * link_val, 700])
    return h₀(d, t) * exp(exp_val)
end

```

`hazard` (generic function with 3 methods)

Notice the recursiveness of this formulation: > The hazard function of a SimpleJointModel calls the hazard of `base_survival`. If the `base_survival` is a distribution from `Distributions.jl` (e.g. `Weibull`) then we use the formulation above with the `ccdf` and `pdf`. But we can also give another SimpleJointModel as `base_survival` which then extends the final hazard formulation adding exponentials with links multiplicatively. Therefore this formulation is already capable of containing multiple links to the same or to different longitudinal models.

With these three codeblocks describing the `sld`, `struct SimpleJointModel` and the `hazard` we have the joint model implemented and it is ready to be used in `Turing` for posterior sampling. (Along with the logic I provide in `HazardBasedDistribution`)

Numerical Troubles

Notice that in the joint model formulation the hazard has an exponential $\exp(\beta \cdot f(\text{SLD}(t)))$. For 64-Bit floats this will become a problem as soon as the exponential becomes greater than about 700.

```
exp(709)
```

```
8.218407461554972e307
```

```
exp(710)
```

Inf

This is simply a numerical problem. Mathematically we would assume that the β cancels out the link contribution. But for bayesian sampling we choose the parameters of the longitudinal model SLD and β at “random” (according to the sampling algorithm). Thus we need to provide a numerical bounds check for this link. In the next section we will look at what values the the longitudinal model produces and see this issue reappearing.

When sampling the posterior these bayesian algorithms start with the prior. This leads to problems when integrating the hazard with out custom distribution. Therefore I have also implemented a try & catch statement around the integration of the cumulative hazard.

2 Sampling in Turing

We will use simulated data by Kerioui et al. (2020) as well as their setup for the priors.

```
file_path = joinpath(@__DIR__, "../data/Simulated_Dataset.txt")
file = CSV.File(file_path)
df = DataFrame(file)
# longitudinal data
longit_id = df.ID
longit_time = df.Time
longit_sld = df.SLD
# survival data
surv_id = unique(df.ID)
row_first_entry = [findfirst(x → x == n, df.ID) for n in surv_id]
surv_time = df[row_first_entry, :T]
surv_indicator = df[row_first_entry, :delta]
df
```

	ID	Time	SLD	Censor_SLD	T	delta
	Int64	Int64	Float64	Int64	Float64	Int64
1	1	0	37.6992	0	628.603	1
2	1	63	48.0833	0	628.603	1
3	1	126	55.9597	0	628.603	1
4	1	189	82.2177	0	628.603	1
5	1	252	86.4839	0	628.603	1
6	1	315	77.9865	0	628.603	1
7	1	378	101.881	0	628.603	1
8	1	441	133.179	0	628.603	1
9	1	504	180.761	0	628.603	1
10	1	567	206.943	0	628.603	1
11	2	0	99.0234	0	5.84868	1
12	3	0	34.1804	0	599.556	1
13	3	63	47.9672	0	599.556	1
14	3	126	38.5505	0	599.556	1
15	3	189	37.2811	0	599.556	1
16	3	252	49.4972	0	599.556	1
17	3	315	76.0135	0	599.556	1
18	3	378	69.6563	0	599.556	1
19	3	441	74.4632	0	599.556	1
20	3	504	80.4039	0	599.556	1
21	3	567	117.914	0	599.556	1
22	4	0	24.5032	0	592.066	1
23	4	63	18.9623	0	592.066	1
24	4	126	23.9573	0	592.066	1
25	4	189	28.2187	0	592.066	1
26	4	252	16.7593	0	592.066	1
27	4	315	26.8666	0	592.066	1
28	4	378	38.2494	0	592.066	1
29	4	441	22.8257	0	592.066	1
30	4	504	30.5844	0	592.066	1
...	...	...	...	...	...	...

In the paper Kerioui et al. (2020) there is also a description of how the individual Ψ_i are build. They are formulated with a population vector $\mu = \{\mu_{BSLD}, \mu_d, \mu_g, \mu_\varphi\}$ and individual random effects vectors $\eta_i = \{\eta_{BSLD,i}, \eta_{d,i}, \eta_{g,i}, \eta_{\varphi,i}\}$ which are normal distributed around zero $\eta_i \sim \mathcal{N}(0, \Omega)$ with $\Omega = \text{diag}(\omega_{BSLD}^2, \omega_d^2, \omega_g^2, \omega_\varphi^2)$.

For biological constraints the parameters were transformed such that $\tau_q(\Psi_{q,i}) = \tau_q \mu_q + \eta_{q,i}$ for $q \in \{BSLD, d, g, \varphi\}$. They assumed log-normal transformations for BSLD, d, g ($\tau = \log$) and logit-normal for φ ($\tau = \text{logit}$).

All this is important to build the prior. Let us start with only considering the longitudinal

model. First we quickly analyze the priors.

I wrap the `sld` function with a bound to avoid numerical explosions.

```
function sld_wrap( $\mu$ , $\eta_i$ , $t_x$ )
  val = sld( $\mu$ , $\eta_i$ , $t_x$ )
  if isnan(val)
    return 0.
  end
  return min(1e4,sld( $\mu$ , $\eta_i$ , $t_x$ ))
end
```

`sld_wrap` (generic function with 1 method)

To choose the spesific priors we refere to Kerioui et al. (2020).

```
@model function longitudinal_prior(longit_id, longit_measurement, longit_time, surv_ids, surv_time, s)
  # treatment at study star
   $t_x$  = 0.0
  # number of longitudinal and survival measurements
  n = length(surv_ids)
  m = length(longit_id)

  ## priors longitudinal
  #  $\mu$  priors, population parameters
   $\mu_{BSLD}$  ~ LogNormal(3.5,1)
   $\mu_d$  ~ Beta(1,100)
   $\mu_g$  ~ Beta(1,100)
   $\mu_\phi$  ~ Beta(2,4)
   $\mu$  = (BSLD =  $\mu_{BSLD}$ , d =  $\mu_d$ , g =  $\mu_g$ ,  $\phi$  =  $\mu_\phi$ )
  #  $\omega$  priors, mixed/individual effects
   $\omega_{BSLD}$  ~ LogNormal(0,1)
   $\omega_d$  ~ LogNormal(0,1)
   $\omega_g$  ~ LogNormal(0,1)
   $\omega_\phi$  ~ LogNormal(0,1)
   $\Omega$  = (BSLD =  $\omega_{BSLD}^2$ , d =  $\omega_d^2$ , g =  $\omega_g^2$ ,  $\phi$  =  $\omega_\phi^2$ )
  # multiplicative error
   $\sigma$  ~ LogNormal(0,1)

  ##  $\eta$  describing the mixed effects of the population
   $\eta_{BSLD}$  ~ filldist(Normal(0, $\Omega$ .BSLD),n)
```

```

 $\eta_d \sim \text{filldist}(\text{Normal}(0, \Omega.d), n)$ 
 $\eta_g \sim \text{filldist}(\text{Normal}(0, \Omega.g), n)$ 
 $\eta_\phi \sim \text{filldist}(\text{Normal}(0, \Omega.\phi), n)$ 
 $\eta = [(BSLD=\eta\_BSLD[i], d=\eta\_d[i], g=\eta\_g[i], \phi=\eta\_phi[i]) \text{ for } i \text{ in } 1:n]$ 
# Transforms
 $\Psi = [[\mu\_BSLD * \exp(\eta\_BSLD[i]),$ 
 $\mu\_g * \exp(\eta\_g[i]),$ 
 $\mu\_d * \exp(\eta\_d[i]),$ 
 $\text{inverse}(\text{logit})(\text{logit}(\mu\_phi) + (\eta\_phi[i]))]$ 
for i in 1:n]

# Prior pred
P = [sld_wrap(longit_time[obs],  $\Psi[\text{Int}(\text{longit\_id}[\text{obs}])]$ , tx) for obs in 1:m]

prior_pred ~ arraydist(Dirac.(P))
end
prior_model = longitudinal_prior(longit_id, longit_sld, longit_time, surv_id, surv_time, surv_indica
prior_chn = sample(prior_model, Prior(), 5000)

```

Chains MCMC chain (5000×1065×1 Array{Float64, 3}):

```

Iterations      = 1:1:5000
Number of chains = 1
Samples per chain = 5000
Wall duration    = 3.07 seconds
Compute duration = 3.07 seconds
parameters       =  $\mu\_BSLD, \mu\_d, \mu\_g, \mu\_phi, \omega\_BSLD, \omega\_d, \omega\_g, \omega\_phi, \sigma, \eta\_BSLD[1], \eta\_BSLD[2], \eta\_BSLD[3], \eta$ 
internals        = lp

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_BSLD	56.2911	83.8049	1.1893	5060.6322	4894.1336	1.0002	...
μ_d	0.0099	0.0097	0.0001	5367.8287	4829.4178	1.0001	...
μ_g	0.0099	0.0098	0.0001	4905.0178	4821.3368	1.0004	...
μ_phi	0.3283	0.1777	0.0025	4897.4508	4900.6625	1.0000	...
ω_BSLD	1.6412	2.1055	0.0298	5294.0793	4539.9188	1.0003	...
ω_d	1.6562	2.1873	0.0308	4948.7957	4622.7096	0.9999	...
ω_g	1.5907	1.9978	0.0289	4896.4128	4792.6515	1.0004	...
ω_phi	1.6692	2.1436	0.0303	4843.1547	4681.1705	1.0001	...
σ	1.6546	2.2466	0.0324	4809.1028	4794.5693	0.9999	...

$\eta_{\text{BSLD}}[1]$	-0.1781	21.4735	0.3174	4563.5036	4921.7275	0.9998	...
$\eta_{\text{BSLD}}[2]$	-0.4286	29.2929	0.4171	5165.3975	4784.2801	1.0001	...
$\eta_{\text{BSLD}}[3]$	0.1425	37.6254	0.5322	4918.6829	4749.3310	1.0002	...
$\eta_{\text{BSLD}}[4]$	-0.7504	50.3835	0.7268	5333.0074	4784.2801	0.9998	...
$\eta_{\text{BSLD}}[5]$	-0.1132	31.3971	0.4457	5116.7208	4977.7796	0.9998	...
$\eta_{\text{BSLD}}[6]$	-0.6538	32.1971	0.4540	4875.3267	4653.0727	0.9999	...
$\eta_{\text{BSLD}}[7]$	-1.2091	47.6230	0.6749	4695.9342	4537.3734	1.0002	...
$\eta_{\text{BSLD}}[8]$	0.3420	36.2343	0.5126	5098.8656	4803.7537	0.9998	...
$\eta_{\text{BSLD}}[9]$	0.4441	40.5833	0.5750	4880.9948	5069.1465	1.0002	...
$\eta_{\text{BSLD}}[10]$	-0.9863	43.9138	0.6209	5115.0904	5105.9209	1.0000	...
$\eta_{\text{BSLD}}[11]$	-0.1288	29.7124	0.4219	4976.5836	4729.3866	1.0000	...
$\eta_{\text{BSLD}}[12]$	0.2048	26.2340	0.3749	4839.8684	4605.8675	1.0005	...
$\eta_{\text{BSLD}}[13]$	0.5549	37.1787	0.5439	4704.2443	4493.8033	1.0003	...
$\eta_{\text{BSLD}}[14]$	0.1811	30.8054	0.4333	5181.8965	5099.1240	1.0000	...
:	:	:	:	:	:	:	...

1 column and 1041 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	4.5566	16.7918	32.5689	65.7719	244.7332
μ_d	0.0003	0.0029	0.0069	0.0138	0.0356
μ_g	0.0002	0.0028	0.0068	0.0137	0.0359
μ_ϕ	0.0509	0.1877	0.3103	0.4485	0.7131
ω_{BSLD}	0.1476	0.5121	0.9955	1.9330	7.4056
ω_d	0.1444	0.4992	0.9816	1.9319	7.4504
ω_g	0.1407	0.4971	0.9501	1.9066	6.6178
ω_ϕ	0.1330	0.5045	0.9759	2.0002	7.5478
σ	0.1317	0.4882	0.9908	1.9810	7.1165
$\eta_{\text{BSLD}}[1]$	-18.9514	-0.5316	0.0073	0.6054	17.8115
$\eta_{\text{BSLD}}[2]$	-18.9199	-0.5534	0.0007	0.5840	20.3077
$\eta_{\text{BSLD}}[3]$	-18.4187	-0.5579	0.0010	0.5652	22.8262
$\eta_{\text{BSLD}}[4]$	-22.9724	-0.6716	-0.0020	0.5129	19.4004
$\eta_{\text{BSLD}}[5]$	-19.8551	-0.5953	-0.0005	0.5668	17.7433
$\eta_{\text{BSLD}}[6]$	-24.4232	-0.5478	0.0023	0.5821	18.0100
$\eta_{\text{BSLD}}[7]$	-20.4081	-0.6419	-0.0065	0.4970	18.0236
$\eta_{\text{BSLD}}[8]$	-18.0704	-0.5806	0.0022	0.5718	18.7669
$\eta_{\text{BSLD}}[9]$	-20.0766	-0.5776	-0.0029	0.5226	18.6483
$\eta_{\text{BSLD}}[10]$	-22.4677	-0.5587	-0.0004	0.5594	21.6092
$\eta_{\text{BSLD}}[11]$	-19.3243	-0.5788	-0.0044	0.5507	20.2619
$\eta_{\text{BSLD}}[12]$	-21.4161	-0.5881	0.0032	0.5984	19.9787
$\eta_{\text{BSLD}}[13]$	-19.4901	-0.5860	-0.0002	0.5403	19.9378

```

η_BSLD[14]    -20.3504    -0.5971    -0.0028    0.5152    19.5826
      ⋮              ⋮              ⋮              ⋮              ⋮
                                1041 rows omitted

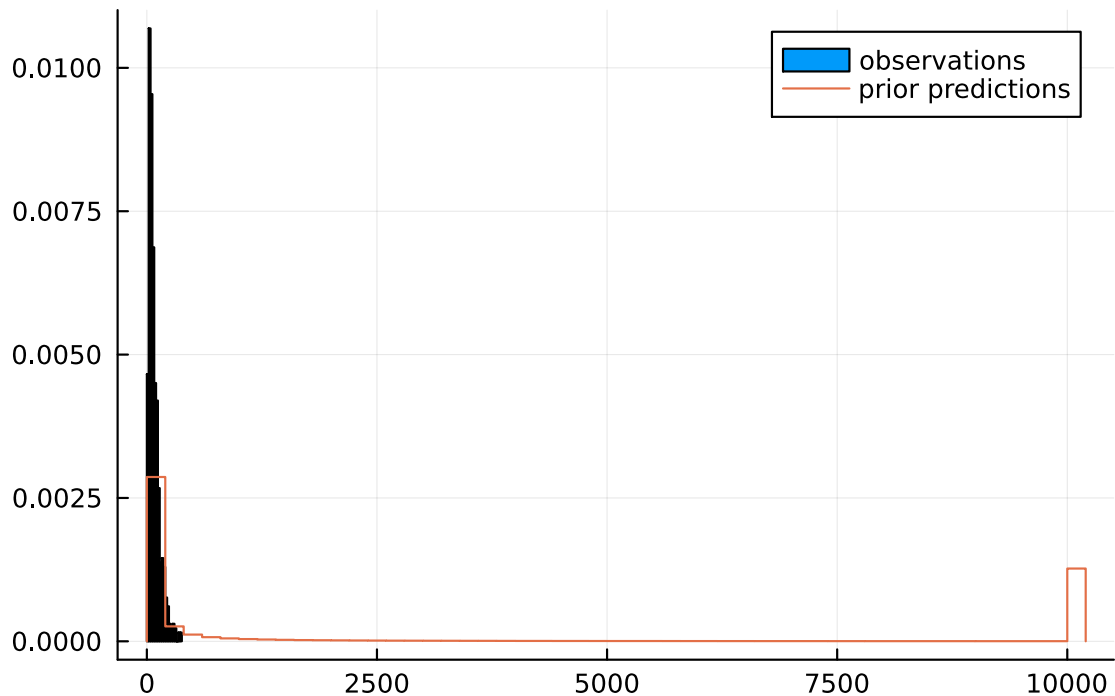
```

Analyze the prior predictions compared to observations, here note the acumulation at 10000 representing all the values greater than 10000 because of the wrap_sld.

```

histogram(longit_sld, normalize=:pdf, label="observations")
prior_df = DataFrame(prior_chn)
select!(prior_df, r"prior_pred.*")
prior_pred = reduce(vcat, eachcol(prior_df))
histogram!(prior_pred, normalize=:pdf, label="prior predictions")

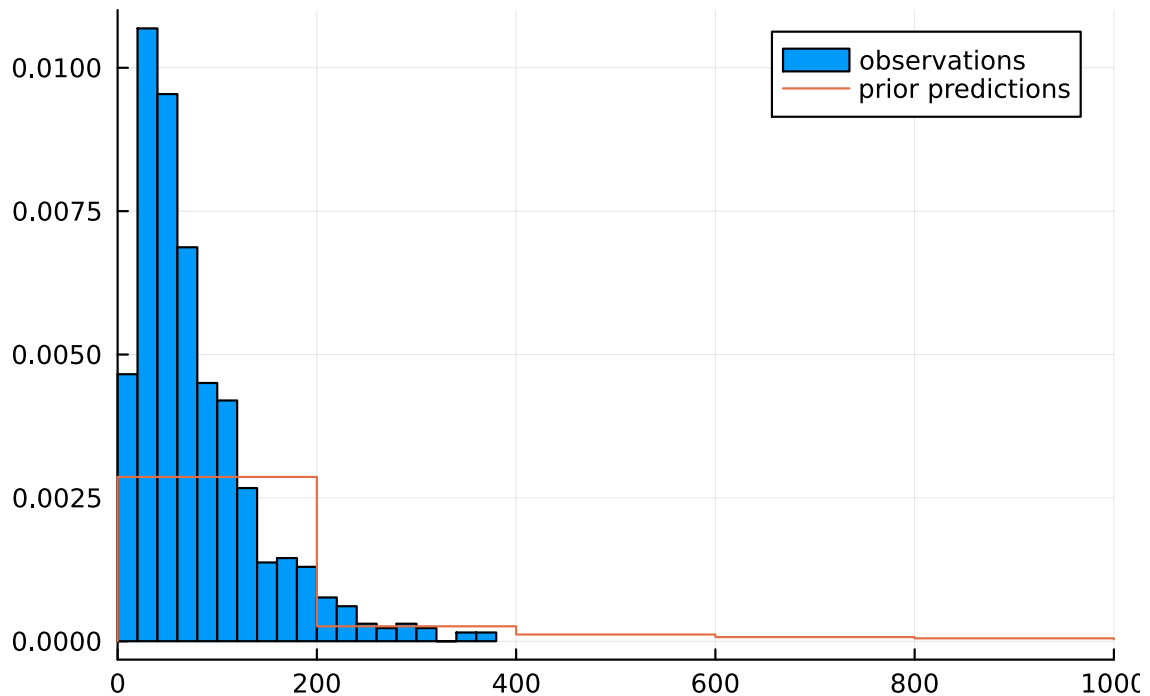
```



The bump at 10000 represents all measurements that are bigger than this. So the posterior describes very large values. In particular even parameter combinations that make sld return NaN. This is also filtered out by wrap_sld.

Only showing the range 0 to 1000 it seems to do a good job.

```
xlims!((0,1000))
```



2.0.1 Fitting Longitudinal part

Setup `ReverseDiff.jl` automatic differentiation backend for faster computations, see [ad backend](#) and [reversediff.jl](#).

```
using ReverseDiff
Turing.setadbackend(:reversediff)
Turing.setrdcache(true)
```

```
true
```

Sampling the posterior of the longitudinal part. We setup the NUTS sampler with an acceptance rate of 0.9 and a maximum depth of 8. For NaN values we just set the `sld_prediction` to 0.

```
@model function longitudinal_sld(longit_ids, longit_times, longit_measurements, surv_ids, surv_times
    # treatment at study star
```

```

tx = 0.0
# number of longitudinal and survival measurements
n = length(surv_ids)
m = length(longit_ids)

# ----- Priors -----

## priors longitudinal
#  $\mu$  priors, population parameters
 $\mu_{\text{BSLD}} \sim \text{LogNormal}(3.5, 1)$ 
 $\mu_d \sim \text{Beta}(1, 100)$ 
 $\mu_g \sim \text{Beta}(1, 100)$ 
 $\mu_\phi \sim \text{Beta}(2, 4)$ 
 $\mu = (\text{BSLD} = \mu_{\text{BSLD}}, d = \mu_d, g = \mu_g, \phi = \mu_\phi)$ 
#  $\omega$  priors, mixed/individual effects
 $\omega_{\text{BSLD}} \sim \text{LogNormal}(0, 1)$ 
 $\omega_d \sim \text{LogNormal}(0, 1)$ 
 $\omega_g \sim \text{LogNormal}(0, 1)$ 
 $\omega_\phi \sim \text{LogNormal}(0, 1)$ 
 $\Omega = (\text{BSLD} = \omega_{\text{BSLD}}^2, d = \omega_d^2, g = \omega_g^2, \phi = \omega_\phi^2)$ 
# multiplicative error
 $\sigma \sim \text{LogNormal}(0, 1)$ 

##  $\eta$  describing the mixed effects of the population
 $\eta_{\text{BSLD}} \sim \text{filldist}(\text{Normal}(0, \Omega_{\text{BSLD}}), n)$ 
 $\eta_d \sim \text{filldist}(\text{Normal}(0, \Omega_d), n)$ 
 $\eta_g \sim \text{filldist}(\text{Normal}(0, \Omega_g), n)$ 
 $\eta_\phi \sim \text{filldist}(\text{Normal}(0, \Omega_\phi), n)$ 
 $\eta = [(\text{BSLD} = \eta_{\text{BSLD}}[i], d = \eta_d[i], g = \eta_g[i], \phi = \eta_\phi[i]) \text{ for } i \text{ in } 1:n]$ 
# Transforms
 $\Psi = [(\mu_{\text{BSLD}} * \exp(\eta_{\text{BSLD}}[i]),$ 
       $\mu_g * \exp(\eta_g[i]),$ 
       $\mu_d * \exp(\eta_d[i]),$ 
       $\text{inverse}(\text{logit})(\text{logit}(\mu_\phi) + (\eta_\phi[i]))]$ 
      for i in 1:n]

# add the likelihood of the longitudinal process
for measurement in 1:m
  id = Int(longit_ids[measurement])
  meas_time = longit_times[measurement]

```

```

    sld_prediction = sld(meas_time,  $\Psi$ [id], t_x)
    if isnan(sld_prediction) || sld_prediction < 0
        sld_prediction = 0
    end
    longitud_measurements[measurement] ~ Normal(sld_prediction, sld_prediction *  $\sigma$ )
end
end

longitudinal_model = longitudinal_sld(longit_id, longit_time, longit_sld, surv_id, surv_time, surv_i
longitudinal_chn = sample(longitudinal_model, NUTS(200, 0.9, max_depth = 8), 400)

```

Chains MCMC chain (400×421×1 Array{Float64, 3}):

```

Iterations      = 201:1:600
Number of chains = 1
Samples per chain = 400
Wall duration    = 135.63 seconds
Compute duration = 135.63 seconds
parameters      =  $\mu_{\text{BSLD}}$ ,  $\mu_d$ ,  $\mu_g$ ,  $\mu_\phi$ ,  $\omega_{\text{BSLD}}$ ,  $\omega_d$ ,  $\omega_g$ ,  $\omega_\phi$ ,  $\sigma$ ,  $\eta_{\text{BSLD}}[1]$ ,  $\eta_{\text{BSLD}}[2]$ ,  $\eta_{\text{BSLD}}[3]$ ,  $\eta_{\text{BSLD}}[4]$ ,  $\eta_{\text{BSLD}}[5]$ ,  $\eta_{\text{BSLD}}[6]$ ,  $\eta_{\text{BSLD}}[7]$ ,  $\eta_{\text{BSLD}}[8]$ 
internals       = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltonian_energy_grad,

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_{BSLD}	62.3995	4.7265	0.4993	92.9178	215.9947	1.0088	...
μ_d	0.0006	0.0004	0.0001	27.0486	75.8707	1.0472	...
μ_g	0.0022	0.0003	0.0001	42.0494	88.4496	1.0410	...
μ_ϕ	0.5897	0.0640	0.0230	9.7025	19.0506	1.0750	...
ω_{BSLD}	0.8510	0.0312	0.0014	489.0546	365.0671	0.9998	...
ω_d	1.2925	0.1452	0.0225	44.7599	68.8610	1.0093	...
ω_g	0.9205	0.0518	0.0035	229.6519	305.6297	1.0031	...
ω_ϕ	0.3298	0.1485	0.1018	2.3084	11.0000	1.3545	...
σ	0.1820	0.0064	0.0007	83.7811	214.3616	1.0177	...
$\eta_{\text{BSLD}}[1]$	-0.3038	0.1301	0.0087	224.3612	190.5559	0.9985	...
$\eta_{\text{BSLD}}[2]$	0.4448	0.1848	0.0115	293.2780	209.6268	1.0084	...
$\eta_{\text{BSLD}}[3]$	-0.4845	0.1368	0.0103	185.1868	253.3737	0.9985	...
$\eta_{\text{BSLD}}[4]$	-0.9719	0.1268	0.0095	180.3767	287.6290	0.9982	...
$\eta_{\text{BSLD}}[5]$	-0.4019	0.1501	0.0090	277.0003	282.9223	1.0041	...
$\eta_{\text{BSLD}}[6]$	0.8721	0.1435	0.0098	223.6190	221.4700	1.0025	...
$\eta_{\text{BSLD}}[7]$	0.2248	0.2239	0.0172	178.5987	171.1193	0.9985	...
$\eta_{\text{BSLD}}[8]$	-0.2109	0.1693	0.0098	297.4222	265.8666	0.9977	...

$\eta_{\text{BSLD}}[9]$	0.2685	0.1269	0.0086	214.5855	263.6577	0.9984	...
$\eta_{\text{BSLD}}[10]$	-0.7370	0.1595	0.0100	251.8245	234.5218	1.0100	...
$\eta_{\text{BSLD}}[11]$	-0.1806	0.1773	0.0115	233.4303	309.0164	1.0036	...
$\eta_{\text{BSLD}}[12]$	1.4969	0.1768	0.0099	320.4003	293.1243	1.0061	...
$\eta_{\text{BSLD}}[13]$	-0.4429	0.1445	0.0089	268.8267	338.9069	1.0012	...
$\eta_{\text{BSLD}}[14]$	0.5084	0.1491	0.0094	256.7911	309.0164	1.0140	...
:	:	:	:	:	:	:	...

1 column and 386 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	53.6021	59.2161	61.9720	65.7979	71.7370
μ_d	0.0001	0.0004	0.0005	0.0008	0.0017
μ_g	0.0015	0.0020	0.0022	0.0025	0.0029
μ_ϕ	0.4312	0.5603	0.5995	0.6331	0.6942
ω_{BSLD}	0.7956	0.8291	0.8487	0.8716	0.9177
ω_d	1.0383	1.1990	1.2796	1.3761	1.6212
ω_g	0.8303	0.8835	0.9184	0.9539	1.0328
ω_ϕ	0.1607	0.2190	0.2858	0.4037	0.7072
σ	0.1703	0.1777	0.1818	0.1863	0.1938
$\eta_{\text{BSLD}}[1]$	-0.5446	-0.3895	-0.3074	-0.2169	-0.0581
$\eta_{\text{BSLD}}[2]$	0.1015	0.3236	0.4364	0.5596	0.8307
$\eta_{\text{BSLD}}[3]$	-0.7159	-0.5833	-0.4834	-0.3951	-0.2044
$\eta_{\text{BSLD}}[4]$	-1.2121	-1.0561	-0.9801	-0.8941	-0.6938
$\eta_{\text{BSLD}}[5]$	-0.6801	-0.5028	-0.4114	-0.3076	-0.1002
$\eta_{\text{BSLD}}[6]$	0.6019	0.7784	0.8658	0.9644	1.1564
$\eta_{\text{BSLD}}[7]$	-0.1429	0.0629	0.2088	0.3551	0.7491
$\eta_{\text{BSLD}}[8]$	-0.5308	-0.3324	-0.2051	-0.1022	0.1440
$\eta_{\text{BSLD}}[9]$	-0.0110	0.1893	0.2685	0.3554	0.5071
$\eta_{\text{BSLD}}[10]$	-1.0524	-0.8327	-0.7392	-0.6369	-0.4180
$\eta_{\text{BSLD}}[11]$	-0.4679	-0.3004	-0.1998	-0.0668	0.1766
$\eta_{\text{BSLD}}[12]$	1.1841	1.3640	1.5066	1.6128	1.8611
$\eta_{\text{BSLD}}[13]$	-0.7167	-0.5404	-0.4365	-0.3441	-0.1831
$\eta_{\text{BSLD}}[14]$	0.2126	0.4039	0.5056	0.6038	0.8148
:	:	:	:	:	:

386 rows omitted

2.0.2 Fitting Survival part

Only fitting the survival using a SimpleJointModel but with $\beta = 0$ and $\text{link}(t) = 0$.

```

@model function survival_sld(longit_ids, longit_times, longit_measurements, surv_ids, surv_times, su
# treatment at study star
tx = 0.0
# number of longitudinal and survival measurements
n = length(surv_ids)
m = length(longit_ids)

# ----- Priors -----
## priors survival
κ ~ truncated(LogNormal(0,1),0.8,1.2)
λ ~ LogNormal(5,1)
## prior link
β = 0

# add the likelihood of the survival model with link
for individual in 1:n
  id = Int(surv_ids[individual])
  censoring = Bool(surv_event[id]) ? Inf : surv_times[id]
  surv_times[individual] ~ censored(
    SimpleJointModel(Weibull(κ,λ),
      β,
      x → 0 # zero function
    ), upper = censoring)
end
end
survival_model = survival_sld(longit_id, longit_time, longit_sld, surv_id, surv_time, surv_indicator
survival_chn = sample(survival_model, NUTS(200, 0.9, max_depth = 8), 400)

```

Chains MCMC chain (400×14×1 Array{Float64, 3}):

```

Iterations      = 201:1:600
Number of chains = 1
Samples per chain = 400
Wall duration    = 29.72 seconds
Compute duration = 29.72 seconds
parameters      = κ, λ
internals        = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...

κ	1.0902	0.0741	0.0051	121.6737	61.7937	1.0158	...
λ	478.5908	45.5606	3.4782	167.0772	145.7966	1.0100	...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
κ	0.9256	1.0475	1.0984	1.1513	1.1928
λ	400.7639	442.0767	478.1608	506.0505	572.1548

Computation time: This can surly be improved with optimizations and logic changes. Let us quickly estimate the number of numerical integrations we solve. Per likelihood evaluation we solve 100 integrals. From the chains output we can compute how many samples a were generated by summing up the number of steps per sample that was generated (NUTS has a rejection component). Notice that we warmup is not included here so we multiply by 1.5 (50% of sample size is warmup) to estimate it.

```
steps_per_sample = survival_chn[:n_steps]
number_of_steps = sum(steps_per_sample)
number_of_steps_with_warmup = number_of_steps * 1.5
number_of_integrations = 100 * number_of_steps_with_warmup
```

321000.0

Which is quite a huge number of integrations, which we could reduce by globally solving the integral using the `DifferentialEquations.jl` package and then evaluating its solution with spesific parameter and time combinations later on.

But also notice that the survival process was sampled faster than the longitudinal process. This is because of the number of parameters: 2 for the survival vs 404 for the longitudinal (4 population and 400 mixed effects).

2.1 Joint models with a link function / operator

With both the longitudinal and survival models we can combine them and use a link function. This link function is dependent on the longitudinal model and influences the hazard of the survival model. Here we use the current `sld` value as the link function. We call this the identity link.

Also we initialize parameters since the NUTS algorithms struggles to find initial values for the joint model (Particle Gibbs which allows to group parameters might help here).

```
n = 100
init_params = (
    mu_BSLD=60, mu_d=0.0055, mu_g=0.0015, mu_phi=0.2,
    omega_BSLD=0.7, omega_d=1.0, omega_g=1.0, omega_phi=1.5, sigma=0.18,
    kappa = 1, lambda=1450, beta = 0,
    eta_BSLD=zeros(n), eta_d=zeros(n), eta_g=zeros(n), eta_phi=zeros(n),
)
```

($\mu_{BSLD} = 60$, $\mu_d = 0.0055$, $\mu_g = 0.0015$, $\mu_\phi = 0.2$, $\omega_{BSLD} = 0.7$, $\omega_d = 1.0$, $\omega_g = 1.0$, $\omega_\phi = 1.5$, $\sigma =$

```
@model function identity_link(longit_ids, longit_times, longit_measurements, surv_ids, surv_times, s
    # treatment at study star
    tx = 0.0
    # number of longitudinal and survival measurements
    n = length(surv_ids)
    m = length(longit_ids)

    # ----- Priors -----

    ## priors longitudinal
    # mu priors, population parameters
    mu_BSLD ~ LogNormal(3.5,1)
    mu_d ~ Beta(1,100)
    mu_g ~ Beta(1,100)
    mu_phi ~ Beta(2,4)
    mu = (BSLD = mu_BSLD, d = mu_d, g = mu_g, phi = mu_phi)
    # omega priors, mixed/individual effects
    omega_BSLD ~ LogNormal(0,1)
    omega_d ~ LogNormal(0,1)
    omega_g ~ LogNormal(0,1)
    omega_phi ~ LogNormal(0,1)
    Omega = (BSLD = omega_BSLD^2, d = omega_d^2, g = omega_g^2, phi = omega_phi^2)
    # multiplicative error
    sigma ~ LogNormal(0,1)
    ## priors survival
    kappa ~ LogNormal(0,1)
    # some more prior knowledge on kappa
    kappa ~ truncated(LogNormal(0,1),0.8,1.2)
```

```

λ ~ LogNormal(5,1)
## prior joint model
#β ~ Normal(0,0.5)
# some more prior knowledge on β, can help with calculating individual hazard (computationally)
β ~ truncated(Normal(0,0.5),-0.1,0.1)

## η describing the mixed effects of the population
η_BSLD ~ filldist(Normal(0,Ω.BSLD),n)
η_d ~ filldist(Normal(0,Ω.d),n)
η_g ~ filldist(Normal(0,Ω.g),n)
η_φ ~ filldist(Normal(0,Ω.φ),n)
η = [(BSLD=η_BSLD[i], d=η_d[i], g=η_g[i], φ=η_φ[i]) for i in 1:n]
# Transforms
Ψ = [[μ_BSLD * exp(η_BSLD[i]),
      μ_g * exp(η_g[i]),
      μ_d * exp(η_d[i]),
      inverse(logit)(logit(μ_φ) + (η_φ[i]))]
      for i in 1:n]

# add the likelihood of the longitudinal process
for measurement in 1:m
  id = Int(longit_ids[measurement])
  meas_time = longit_times[measurement]
  sld_prediction = sld(meas_time, Ψ[id], t_x)
  if isnan(sld_prediction) || sld_prediction < 0
    sld_prediction = 0
  end
  longit_measurements[measurement] ~ Normal(sld_prediction, sld_prediction * σ)
end

# add the likelihood of the survival model with link
for individual in 1:n
  id = Int(surv_ids[individual])
  id_link(t) = sld(t, Ψ[id], t_x)
  censoring = Bool(surv_event[id]) ? Inf : surv_times[id]
  surv_times[individual] ~ censored(SimpleJointModel(Weibull(κ,λ), β, id_link), upper = censoring)
end
end

identity_link_model = identity_link(longit_id, longit_time, longit_sld, surv_id, surv_time, surv_ind)

```

```
identity_link_chn = sample(identity_link_model, NUTS(100, 0.9, max_depth = 8), 200, init_params=init.
```

Chains MCMC chain (200×424×1 Array{Float64, 3}):

```
Iterations          = 101:1:300
Number of chains    = 1
Samples per chain   = 200
Wall duration       = 481.74 seconds
Compute duration    = 481.74 seconds
parameters          = μ_BSLD, μ_d, μ_g, μ_φ, ω_BSLD, ω_d, ω_g, ω_φ, σ, κ, λ, β, η_BSLD[1], η_BSLD[2], η_B
internals           = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni
```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_BSLD	62.3067	5.1594	0.9021	34.2901	25.2039	1.0793	...
μ_d	0.0051	0.0016	0.0003	22.4300	59.7929	1.0170	...
μ_g	0.0018	0.0003	0.0001	18.7343	74.0781	1.0280	...
μ_φ	0.2040	0.0828	0.0277	8.9323	20.8298	1.0127	...
ω_BSLD	0.8579	0.0308	0.0018	326.5713	183.8178	0.9990	...
ω_d	0.6509	0.0944	0.0415	6.4775	49.0305	1.1687	...
ω_g	0.9300	0.0649	0.0208	10.2194	51.3966	1.1015	...
ω_φ	1.1780	0.1440	0.0526	7.5942	14.3972	1.0091	...
σ	0.1776	0.0063	0.0006	112.3502	154.4957	1.0038	...
κ	1.0615	0.0771	0.0124	38.7812	70.7286	1.0003	...
λ	1396.8675	324.4998	33.5119	99.4443	173.8298	0.9979	...
β	0.0101	0.0013	0.0001	90.2115	73.6088	0.9955	...
η_BSLD[1]	-0.3699	0.1402	0.0389	13.4785	132.5010	1.0848	...
η_BSLD[2]	0.4833	0.1993	0.0161	157.6525	99.7835	1.0118	...
η_BSLD[3]	-0.5100	0.1417	0.0144	96.4638	154.2736	1.0273	...
η_BSLD[4]	-0.9687	0.1364	0.0173	66.5535	116.9354	1.0395	...
η_BSLD[5]	-0.4945	0.1711	0.0198	66.8169	106.9151	1.0442	...
η_BSLD[6]	0.8686	0.1492	0.0155	98.1912	73.1560	1.0247	...
η_BSLD[7]	0.0878	0.2126	0.0168	171.3454	138.3085	0.9989	...
η_BSLD[8]	-0.2079	0.1530	0.0131	134.0370	108.1423	1.0055	...
η_BSLD[9]	0.2293	0.1437	0.0176	64.5491	112.6307	1.0336	...
η_BSLD[10]	-0.7728	0.1819	0.0195	81.2318	120.2976	1.0301	...
η_BSLD[11]	-0.2039	0.1485	0.0237	40.2739	27.2493	1.0586	...
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

1 column and 389 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	53.2820	58.8252	62.4147	64.9709	74.0376
μ_d	0.0024	0.0042	0.0050	0.0059	0.0089
μ_g	0.0013	0.0017	0.0018	0.0020	0.0023
μ_ϕ	0.0639	0.1386	0.2014	0.2630	0.3676
ω_{BSLD}	0.8031	0.8347	0.8593	0.8813	0.9128
ω_d	0.4895	0.5730	0.6481	0.7208	0.8407
ω_g	0.7986	0.8921	0.9363	0.9704	1.0421
ω_ϕ	0.9386	1.0750	1.1522	1.2884	1.4727
σ	0.1659	0.1729	0.1775	0.1822	0.1903
κ	0.9022	1.0020	1.0702	1.1204	1.1843
λ	939.4693	1160.1905	1361.1520	1591.9245	2086.2158
β	0.0078	0.0093	0.0101	0.0110	0.0126
$\eta_{\text{BSLD}}[1]$	-0.6693	-0.4547	-0.3664	-0.2762	-0.1233
$\eta_{\text{BSLD}}[2]$	0.1365	0.3488	0.4590	0.6012	0.9250
$\eta_{\text{BSLD}}[3]$	-0.7587	-0.6083	-0.5030	-0.4160	-0.2193
$\eta_{\text{BSLD}}[4]$	-1.2059	-1.0525	-0.9793	-0.8871	-0.6896
$\eta_{\text{BSLD}}[5]$	-0.7986	-0.6041	-0.4951	-0.3942	-0.1301
$\eta_{\text{BSLD}}[6]$	0.6186	0.7654	0.8509	0.9604	1.1469
$\eta_{\text{BSLD}}[7]$	-0.3357	-0.0608	0.0833	0.2312	0.5156
$\eta_{\text{BSLD}}[8]$	-0.5265	-0.2867	-0.2056	-0.1278	0.0924
$\eta_{\text{BSLD}}[9]$	-0.0220	0.1378	0.2217	0.3331	0.5120
$\eta_{\text{BSLD}}[10]$	-1.0682	-0.9132	-0.7797	-0.6630	-0.3641
$\eta_{\text{BSLD}}[11]$	-0.5098	-0.2965	-0.2151	-0.1027	0.0867
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

389 rows omitted

Other common link functions in Keroui et al. (2020) are the time to growth TTG and the derivative $\frac{d}{dt}$. In Julia we define:

```
function TTG( $\Psi$ ,  $t_x = 0$ )
    BSLD, g, d,  $\phi = \Psi$ 
    val = log((d* $\phi$ )/(g*(1- $\phi$ )))/(g+d) +  $t_x$ 
    return maximum((0,val))
end
```

TTG (generic function with 2 methods)

This lets us use the link function $\text{ttg_link}(t) = \text{TTG}(\Psi)$. Notice that ttg_link is a constant

function in time. But in the `SimpleJointModel` definition we specifically ask for a function in time (this can be designed differently).

Putting this into the Turing model.

```
@model function ttg_link(longit_ids, longit_times, longit_measurements, surv_ids, surv_times, surv_e
  # treatment at study star
  tx = 0.0
  # number of longitudinal and survival measurements
  n = length(surv_ids)
  m = length(longit_ids)

  # ----- Priors -----

  ## priors longitudinal
  #  $\mu$  priors, population parameters
   $\mu_{\text{BSLD}} \sim \text{LogNormal}(3.5, 1)$ 
   $\mu_d \sim \text{Beta}(1, 100)$ 
   $\mu_g \sim \text{Beta}(1, 100)$ 
   $\mu_\phi \sim \text{Beta}(2, 4)$ 
   $\mu = (\text{BSLD} = \mu_{\text{BSLD}}, d = \mu_d, g = \mu_g, \phi = \mu_\phi)$ 
  #  $\omega$  priors, mixed/individual effects
   $\omega_{\text{BSLD}} \sim \text{LogNormal}(0, 1)$ 
   $\omega_d \sim \text{LogNormal}(0, 1)$ 
   $\omega_g \sim \text{LogNormal}(0, 1)$ 
   $\omega_\phi \sim \text{LogNormal}(0, 1)$ 
   $\Omega = (\text{BSLD} = \omega_{\text{BSLD}}^2, d = \omega_d^2, g = \omega_g^2, \phi = \omega_\phi^2)$ 
  # multiplicative error
   $\sigma \sim \text{LogNormal}(0, 1)$ 
  ## priors survival
   $\kappa \sim \text{LogNormal}(0, 1)$ 
  # some more prior knowledge on  $\kappa$ 
   $\kappa \sim \text{truncated}(\text{LogNormal}(0, 1), 0.8, 1.2)$ 
   $\lambda \sim \text{LogNormal}(5, 1)$ 
  ## prior joint model
   $\beta \sim \text{Normal}(0, 0.5)$ 
  # some more prior knowledge on  $\beta$ , can help with calculating individual hazard (computationally)
   $\beta \sim \text{truncated}(\text{Normal}(0, 0.5), -0.1, 0.1)$ 

  ##  $\eta$  describing the mixed effects of the population
   $\eta_{\text{BSLD}} \sim \text{filldist}(\text{Normal}(0, \Omega_{\text{BSLD}}), n)$ 
```

```

η_d ~ filldist(Normal(0,Ω.d),n)
η_g ~ filldist(Normal(0,Ω.g),n)
η_φ ~ filldist(Normal(0,Ω.φ),n)
η = [(BSLD=η_BSLD[i], d=η_d[i], g=η_g[i], φ=η_φ[i]) for i in 1:n]
# Transforms
Ψ = [[μ_BSLD * exp(η_BSLD[i]),
      μ_g * exp(η_g[i]),
      μ_d * exp(η_d[i]),
      inverse(logit)(logit(μ_φ) + (η_φ[i]))]]
      for i in 1:n]

# add the likelihood of the longitudinal process
for measurement in 1:m
  id = Int(longit_ids[measurement])
  meas_time = longit_times[measurement]
  sld_prediction = sld(meas_time, Ψ[id], t_x)
  if isnan(sld_prediction) || sld_prediction < 0
    sld_prediction = 0
  end
  longit_measurements[measurement] ~ Normal(sld_prediction, sld_prediction * σ)
end

# add the likelihood of the survival model with link
for individual in 1:n
  id = Int(surv_ids[individual])
  ttg_link(t) = TTG(Ψ[individual])
  censoring = Bool(surv_event[id]) ? Inf : surv_times[id]
  surv_times[individual] ~ censored(SimpleJointModel(Weibull(κ,λ), β, ttg_link), upper = censoring)
end
end
ttg_link_model = ttg_link(longit_id, longit_time, longit_sld, surv_id, surv_time, surv_indicator)
ttg_link_chn = sample(ttg_link_model, NUTS(100, 0.9, max_depth = 8), 200, init_params=init_params)

```

Chains MCMC chain (200×424×1 Array{Float64, 3}):

```

Iterations      = 101:1:300
Number of chains = 1
Samples per chain = 200
Wall duration    = 404.45 seconds
Compute duration = 404.45 seconds
parameters       = μ_BSLD, μ_d, μ_g, μ_φ, ω_BSLD, ω_d, ω_g, ω_φ, σ, κ, λ, β, η_BSLD[1], η_BSLD[2], η_B
internals        = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltoni

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_{BSLD}	61.4998	4.1965	0.5445	67.1530	67.2816	1.0052	...
μ_d	0.0063	0.0016	0.0002	42.3589	26.2743	1.0133	...
μ_g	0.0018	0.0002	0.0000	30.7046	115.8790	1.0186	...
μ_ϕ	0.1704	0.0641	0.0327	3.9326	13.6280	1.1756	...
ω_{BSLD}	0.8486	0.0319	0.0017	348.2709	154.0692	0.9965	...
ω_d	0.4936	0.1184	0.0521	7.7663	12.0863	1.1199	...
ω_g	0.8821	0.0516	0.0094	32.0971	75.5262	1.0402	...
ω_ϕ	1.1645	0.1559	0.1003	2.5077	12.9789	1.3222	...
σ	0.1800	0.0064	0.0006	107.5682	128.7536	1.0017	...
κ	1.1334	0.0529	0.0108	22.5617	32.6393	1.0406	...
λ	370.0765	53.6595	5.5920	90.4789	85.9561	1.0019	...
β	-0.0062	0.0026	0.0004	51.9406	158.6263	1.0159	...
$\eta_{\text{BSLD}}[1]$	-0.3552	0.1420	0.0119	139.1751	114.8812	1.0096	...
$\eta_{\text{BSLD}}[2]$	0.4666	0.1911	0.0143	165.6820	94.9333	1.0057	...
$\eta_{\text{BSLD}}[3]$	-0.4923	0.1415	0.0122	135.1093	137.6368	0.9956	...
$\eta_{\text{BSLD}}[4]$	-0.9560	0.1397	0.0117	140.3426	147.0051	1.0007	...
$\eta_{\text{BSLD}}[5]$	-0.4537	0.1718	0.0162	127.2505	67.8222	1.0068	...
$\eta_{\text{BSLD}}[6]$	0.8645	0.1310	0.0115	126.0985	82.1475	1.0058	...
$\eta_{\text{BSLD}}[7]$	0.2007	0.2266	0.0129	334.0393	185.8821	1.0037	...
$\eta_{\text{BSLD}}[8]$	-0.2115	0.1382	0.0112	155.0950	154.4957	0.9994	...
$\eta_{\text{BSLD}}[9]$	0.2494	0.1477	0.0131	124.8698	83.7055	1.0145	...
$\eta_{\text{BSLD}}[10]$	-0.7439	0.1774	0.0129	194.3232	151.9978	0.9982	...
$\eta_{\text{BSLD}}[11]$	-0.1820	0.1258	0.0130	85.5691	83.3378	1.0300	...
:	:	:	:	:	:	:	...

1 column and 389 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	54.2683	58.9396	61.3881	64.0791	70.8601
μ_d	0.0040	0.0051	0.0061	0.0074	0.0097
μ_g	0.0014	0.0016	0.0018	0.0019	0.0022
μ_ϕ	0.0606	0.1253	0.1734	0.2134	0.2986
ω_{BSLD}	0.7946	0.8266	0.8467	0.8716	0.9142
ω_d	0.3467	0.4218	0.4577	0.5096	0.7966
ω_g	0.7762	0.8462	0.8765	0.9141	0.9860
ω_ϕ	0.9106	1.0615	1.1463	1.2442	1.4906

σ	0.1694	0.1756	0.1799	0.1837	0.1931
κ	1.0053	1.1017	1.1437	1.1779	1.1948
λ	286.3277	327.5294	364.5275	404.6632	490.3827
β	-0.0120	-0.0075	-0.0060	-0.0043	-0.0016
$\eta_{\text{BSLD}}[1]$	-0.6121	-0.4553	-0.3624	-0.2769	-0.0413
$\eta_{\text{BSLD}}[2]$	0.1584	0.3098	0.4415	0.5797	0.8747
$\eta_{\text{BSLD}}[3]$	-0.7945	-0.5842	-0.4792	-0.4093	-0.2003
$\eta_{\text{BSLD}}[4]$	-1.2483	-1.0515	-0.9578	-0.8581	-0.7126
$\eta_{\text{BSLD}}[5]$	-0.7572	-0.5835	-0.4695	-0.3398	-0.0895
$\eta_{\text{BSLD}}[6]$	0.6127	0.7802	0.8625	0.9410	1.1415
$\eta_{\text{BSLD}}[7]$	-0.1972	0.0494	0.1872	0.3734	0.6227
$\eta_{\text{BSLD}}[8]$	-0.4531	-0.3076	-0.2145	-0.1125	0.0582
$\eta_{\text{BSLD}}[9]$	-0.0151	0.1367	0.2556	0.3324	0.5331
$\eta_{\text{BSLD}}[10]$	-1.0556	-0.8761	-0.7553	-0.6361	-0.3657
$\eta_{\text{BSLD}}[11]$	-0.4211	-0.2611	-0.1842	-0.0965	0.0802
:	:	:	:	:	:

389 rows omitted

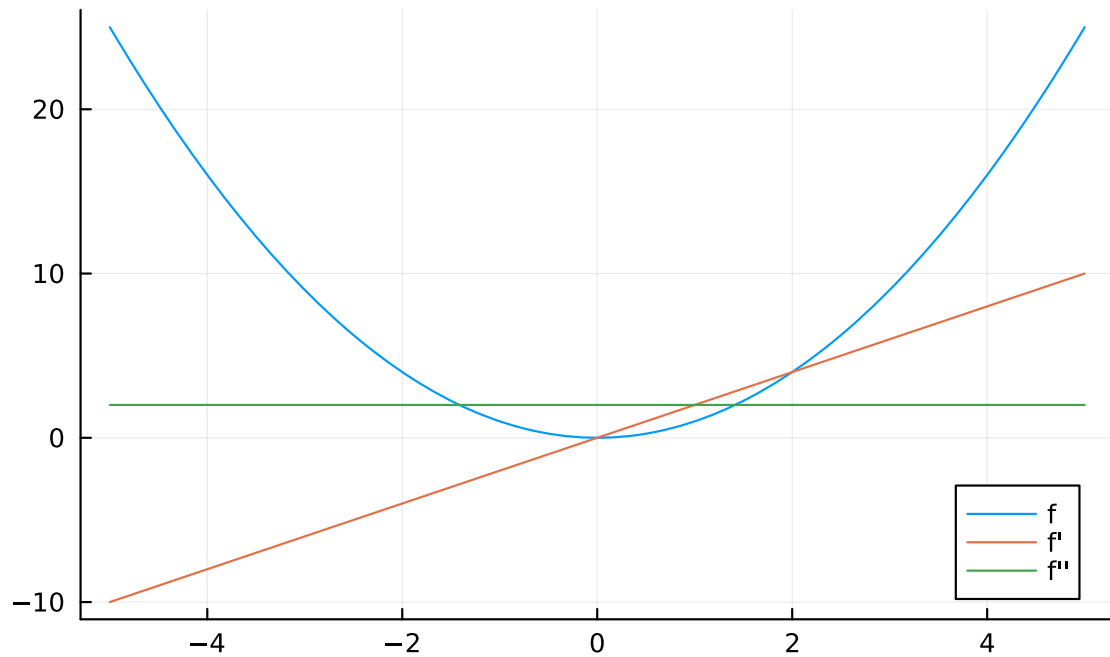
For the derivatives we can use any of Julia's automatic differentiation libraries (here a list of a few dozen [julia diff](#)). The strong support for automatic differentiation is central to the Julia ecosystem and a foundation of its design. Turing interacts with these automatic differentiation packages via the automatic differentiation backend mentioned at the top of this page.

Here we are going to make use of automatic differentiation ourselves to calculate the derivative, specifically ForwardDiff.jl [docs](#). This is the derivative link `derivative_link(t) = ForwardDiff.derivative(t → sld(t,  $\Psi$ ,  $t_x$ ), t)`.

Quickly illustrating how this works

```
using ForwardDiff
f(x) = x^2
Dxf(x) = ForwardDiff.derivative(x → f(x), x) # first derivative
Dx2f(x) = ForwardDiff.derivative(x → Dxf(x), x) # second derivative
plot(f, label = "f", title="Derivatives")
plot!(Dxf, label = "f'")
plot!(Dx2f, label = "f''")
```

Derivatives



In Turing we write

```
@model function derivative_link(longit_ids, longit_times, longit_measurements, surv_ids, surv_times,
  # treatment at study star
  tx = 0.0
  # number of longitudinal and survival measurements
  n = length(surv_ids)
  m = length(longit_ids)

  # ----- Priors -----

  ## priors longitudinal
  #  $\mu$  priors, population parameters
   $\mu_{\text{BSLD}} \sim \text{LogNormal}(3.5, 1)$ 
   $\mu_d \sim \text{Beta}(1, 100)$ 
   $\mu_g \sim \text{Beta}(1, 100)$ 
   $\mu_\phi \sim \text{Beta}(2, 4)$ 
   $\mu = (\text{BSLD} = \mu_{\text{BSLD}}, d = \mu_d, g = \mu_g, \phi = \mu_\phi)$ 
  #  $\omega$  priors, mixed/individual effects
```

```

ω_BSLD ~ LogNormal(0,1)
ω_d ~ LogNormal(0,1)
ω_g ~ LogNormal(0,1)
ω_φ ~ LogNormal(0,1)
Ω = (BSLD = ω_BSLD^2, d = ω_d^2, g = ω_g^2, φ = ω_φ^2)
# multiplicative error
σ ~ LogNormal(0,1)
## priors survival
κ ~ LogNormal(0,1)
# some more prior knowledge on κ
κ ~ truncated(LogNormal(0,1),0.8,1.2)
λ ~ LogNormal(5,1)
## prior joint model
#β ~ Normal(0,0.5)
# some more prior knowledge on β, can help with calculating individual hazard (computationally)
β ~ truncated(Normal(0,0.5),-0.1,0.1)

## η describing the mixed effects of the population
η_BSLD ~ filldist(Normal(0,Ω.BSLD),n)
η_d ~ filldist(Normal(0,Ω.d),n)
η_g ~ filldist(Normal(0,Ω.g),n)
η_φ ~ filldist(Normal(0,Ω.φ),n)
η = [(BSLD=η_BSLD[i], d=η_d[i], g=η_g[i], φ=η_φ[i]) for i in 1:n]
# Transforms
Ψ = [[μ_BSLD * exp(η_BSLD[i]),
      μ_g * exp(η_g[i]),
      μ_d * exp(η_d[i]),
      inverse(logit)(logit(μ_φ) + (η_φ[i]))]
      for i in 1:n]

# add the likelihood of the longitudinal process
for measurement in 1:m
  id = Int(longit_ids[measurement])
  meas_time = longit_times[measurement]
  sld_prediction = sld(meas_time, Ψ[id], t_x)
  if isnan(sld_prediction) || sld_prediction < 0
    sld_prediction = 0
  end
  longit_measurements[measurement] ~ Normal(sld_prediction, sld_prediction * σ)
end

```

```

# add the likelihood of the survival model with link
for individual in 1:n
  id = Int(surv_ids[individual])
  derivative_link(t) = ForwardDiff.derivative(t → sld(t,  $\Psi$ [individual], tx), t)
  censoring = Bool(surv_event[id]) ? Inf : surv_times[id]
  surv_times[individual] ~ censored(SimpleJointModel(Weibull( $\kappa, \lambda$ ),  $\beta$ , derivative_link), upper =
end
end

derivative_link_model = derivative_link(longit_id, longit_time, longit_sld, surv_id, surv_time, surv_
derivative_link_chn = sample(derivative_link_model, NUTS(100, 0.9, max_depth = 8), 200, init_params=

```

Chains MCMC chain (200×424×1 Array{Float64, 3}):

```

Iterations          = 101:1:300
Number of chains    = 1
Samples per chain   = 200
Wall duration       = 544.77 seconds
Compute duration    = 544.77 seconds
parameters          =  $\mu_{\text{BSLD}}, \mu_d, \mu_g, \mu_\phi, \omega_{\text{BSLD}}, \omega_d, \omega_g, \omega_\phi, \sigma, \kappa, \lambda, \beta, \eta_{\text{BSLD}}[1], \eta_{\text{BSLD}}[2], \eta_{\text{BSLD}}[3], \eta_{\text{BSLD}}[4]$ 
internals            = lp, n_steps, is_accept, acceptance_rate, log_density, hamiltonian_energy, hamiltonian_energy_grad,

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_{BSLD}	61.4991	4.1028	0.3959	110.9246	81.5437	1.0081	...
μ_d	0.0054	0.0014	0.0001	106.9923	144.5965	1.0073	...
μ_g	0.0017	0.0002	0.0000	49.6677	47.0810	1.0066	...
μ_ϕ	0.2142	0.0568	0.0088	39.8961	54.3615	1.0156	...
ω_{BSLD}	0.8543	0.0329	0.0017	366.1000	152.9748	0.9972	...
ω_d	0.6510	0.1421	0.0716	3.6089	23.5970	1.2585	...
ω_g	0.9221	0.0558	0.0127	23.6853	86.2407	1.0658	...
ω_ϕ	1.1283	0.0972	0.0407	5.6626	33.8472	1.1965	...
σ	0.1776	0.0058	0.0006	116.7459	152.8143	0.9991	...
κ	1.0949	0.0684	0.0103	40.7367	63.8346	1.0164	...
λ	492.3269	51.5820	4.0294	166.4592	118.1337	1.0260	...
β	0.0533	0.0324	0.0086	8.6848	48.0874	1.1106	...
$\eta_{\text{BSLD}}[1]$	-0.3574	0.1251	0.0100	157.5043	148.7446	1.0096	...
$\eta_{\text{BSLD}}[2]$	0.4506	0.1782	0.0147	152.8990	110.2781	1.0014	...
$\eta_{\text{BSLD}}[3]$	-0.4803	0.1415	0.0126	123.0456	113.4994	1.0129	...
$\eta_{\text{BSLD}}[4]$	-0.9567	0.1209	0.0099	162.8969	158.6263	0.9971	...

$\eta_{\text{BSLD}}[5]$	-0.4866	0.1607	0.0120	181.2544	130.3530	0.9988	...
$\eta_{\text{BSLD}}[6]$	0.8795	0.1557	0.0154	103.9005	113.4994	1.0406	...
$\eta_{\text{BSLD}}[7]$	0.1750	0.2379	0.0166	372.3107	86.9207	1.0042	...
$\eta_{\text{BSLD}}[8]$	-0.2210	0.1353	0.0094	208.1667	149.0770	0.9967	...
$\eta_{\text{BSLD}}[9]$	0.2575	0.1302	0.0107	149.5534	152.8143	0.9992	...
$\eta_{\text{BSLD}}[10]$	-0.7577	0.1592	0.0111	195.9054	190.3419	0.9985	...
$\eta_{\text{BSLD}}[11]$	-0.1948	0.1439	0.0193	56.2157	112.4998	0.9954	...
:	:	:	:	:	:	:	...

1 column and 389 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
μ_{BSLD}	54.6171	59.0855	61.3127	63.8656	71.1413
μ_d	0.0031	0.0046	0.0053	0.0062	0.0083
μ_g	0.0012	0.0015	0.0017	0.0019	0.0022
μ_Φ	0.1193	0.1722	0.2061	0.2528	0.3376
ω_{BSLD}	0.7919	0.8307	0.8530	0.8746	0.9213
ω_d	0.4634	0.5487	0.6215	0.7019	0.9594
ω_g	0.8241	0.8887	0.9174	0.9502	1.0450
ω_Φ	0.9422	1.0620	1.1252	1.1936	1.3103
σ	0.1666	0.1734	0.1776	0.1822	0.1886
κ	0.9529	1.0480	1.0960	1.1564	1.1839
λ	408.2135	452.3890	493.4927	527.1626	591.8449
β	-0.0246	0.0328	0.0604	0.0765	0.0961
$\eta_{\text{BSLD}}[1]$	-0.5666	-0.4397	-0.3684	-0.2757	-0.0917
$\eta_{\text{BSLD}}[2]$	0.1450	0.3237	0.4254	0.5753	0.8290
$\eta_{\text{BSLD}}[3]$	-0.7713	-0.5610	-0.4763	-0.4110	-0.1799
$\eta_{\text{BSLD}}[4]$	-1.1843	-1.0306	-0.9737	-0.8851	-0.6950
$\eta_{\text{BSLD}}[5]$	-0.7757	-0.6035	-0.4918	-0.3894	-0.1572
$\eta_{\text{BSLD}}[6]$	0.5808	0.7803	0.8740	0.9849	1.1821
$\eta_{\text{BSLD}}[7]$	-0.2260	0.0134	0.1401	0.3135	0.7548
$\eta_{\text{BSLD}}[8]$	-0.4734	-0.3210	-0.2218	-0.1251	0.0565
$\eta_{\text{BSLD}}[9]$	0.0116	0.1708	0.2579	0.3286	0.4984
$\eta_{\text{BSLD}}[10]$	-1.0604	-0.8657	-0.7645	-0.6457	-0.4321
$\eta_{\text{BSLD}}[11]$	-0.4466	-0.2811	-0.1927	-0.0908	0.0963
:	:	:	:	:	:

389 rows omitted

Comparing the chain results for parameters of interests (population, error, survival and link)

```
params = [:μ_BSLD, :μ_d, :μ_g, :μ_φ, :σ, :κ, :λ, :β]
```

8-element Vector{Symbol}:

```
:μ_BSLD
:μ_d
:μ_g
:μ_φ
:σ
:κ
:λ
:β
```

Here is a table showing the values used to generate the simulated data provided by Kerioui et al. (2020).

Parameters (Unit)	Fixed Effects
BSLD (mm)	60
d (day ⁻¹)	0.0055
g (day ⁻¹)	0.0015
	0.2
	0.18
	1
(days)	1450
(mm ⁻¹)	0.01

Identity link (true model / used to generate data)

```
summarize(identity_link_chn[params])
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_BSLD	62.3067	5.1594	0.9021	34.2901	25.2039	1.0793	...
μ_d	0.0051	0.0016	0.0003	22.4300	59.7929	1.0170	...
μ_g	0.0018	0.0003	0.0001	18.7343	74.0781	1.0280	...
μ_φ	0.2040	0.0828	0.0277	8.9323	20.8298	1.0127	...
σ	0.1776	0.0063	0.0006	112.3502	154.4957	1.0038	...
κ	1.0615	0.0771	0.0124	38.7812	70.7286	1.0003	...
λ	1396.8675	324.4998	33.5119	99.4443	173.8298	0.9979	...

β	0.0101	0.0013	0.0001	90.2115	73.6088	0.9955	...
---------	--------	--------	--------	---------	---------	--------	-----

1 column omitted

TTG link

```
summarize(ttg_link_chn[params])
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_{BSLD}	61.4998	4.1965	0.5445	67.1530	67.2816	1.0052	...
μ_d	0.0063	0.0016	0.0002	42.3589	26.2743	1.0133	...
μ_g	0.0018	0.0002	0.0000	30.7046	115.8790	1.0186	...
μ_ϕ	0.1704	0.0641	0.0327	3.9326	13.6280	1.1756	...
σ	0.1800	0.0064	0.0006	107.5682	128.7536	1.0017	...
κ	1.1334	0.0529	0.0108	22.5617	32.6393	1.0406	...
λ	370.0765	53.6595	5.5920	90.4789	85.9561	1.0019	...
β	-0.0062	0.0026	0.0004	51.9406	158.6263	1.0159	...

1 column omitted

Derivative link

```
summarize(derivative_link_chn[params])
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_{BSLD}	61.4991	4.1028	0.3959	110.9246	81.5437	1.0081	...
μ_d	0.0054	0.0014	0.0001	106.9923	144.5965	1.0073	...
μ_g	0.0017	0.0002	0.0000	49.6677	47.0810	1.0066	...
μ_ϕ	0.2142	0.0568	0.0088	39.8961	54.3615	1.0156	...
σ	0.1776	0.0058	0.0006	116.7459	152.8143	0.9991	...
κ	1.0949	0.0684	0.0103	40.7367	63.8346	1.0164	...
λ	492.3269	51.5820	4.0294	166.4592	118.1337	1.0260	...
β	0.0533	0.0324	0.0086	8.6848	48.0874	1.1106	...

1 column omitted

No link: Longitudinal

```
summarize(longitudinal_chn[params[1:5]])
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
μ_{BSLD}	62.3995	4.7265	0.4993	92.9178	215.9947	1.0088	...
μ_d	0.0006	0.0004	0.0001	27.0486	75.8707	1.0472	...
μ_g	0.0022	0.0003	0.0001	42.0494	88.4496	1.0410	...
μ_ϕ	0.5897	0.0640	0.0230	9.7025	19.0506	1.0750	...
σ	0.1820	0.0064	0.0007	83.7811	214.3616	1.0177	...

1 column omitted

Survival

```
summarize(survival_chn[params[6:7]])
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
κ	1.0902	0.0741	0.0051	121.6737	61.7937	1.0158	...
λ	478.5908	45.5606	3.4782	167.0772	145.7966	1.0100	...

1 column omitted

2.2 Diagnostics

The pages about survival and longitudinal models already describe a huge number of diagnostics and plots that are applicable here. You could try to implement them for this joint model as an exercise.

Here I want to present a plot specific for joint models / scenarios with longitudinal and survival data. For an individual i we have the longitudinal measurements and a survival time with an event indicator. Suppose this individual has a censored event. We plot the joint model predictions of the longitudinal development as well as the survival starting from the censored time i.e. conditioned on surviving until the censored time. This allows to judge the connection of longitudinal development and survival outcome inside the joint model.

One design decision I took throughout many of the diagnostics and plots is to use nonparametric plots to allow swapping the longitudinal model (sld formulation) or survival model (baseline hazard, link contribution) and use the same Turing `@model` to make the predictions necessary to generate the plot. Setting up the prediction model for individual 25 (first with censored event).

```
# data for individual
individual = 25
```

```

obs_for_individual = longit_id .== individual
individual_obs = longit_sld[obs_for_individual]
individual_time = longit_time[obs_for_individual]
last_obs = individual_time[end]
# timerange to sample
timespan = (0, 1600)
timesteps = 15

r = range(timespan..., timesteps)
sld_timepoints = sort(push!(collect(r), last_obs))

m = length(sld_timepoints)
sld_pred = Vector{Union{Missing, Float64}}(undef, m)
sld_id = fill(individual, m)
# survival pred
n = 100
s_time = Vector{Union{Missing, Float64}}(undef, n)
s_id = fill(individual, n)
s_event = ones(n)

pred_model = identity_link(sld_id, sld_timepoints, sld_pred, s_id, s_time, s_event)

```

DynamicPPL.Model{typeof(identity_link), (:longit_ids, :longit_times, :longit_measurements, :surv_ids, :

Predict using zero σ

```

prediction_chn = predict(pred_model, identity_link_chn)

```

Chains MCMC chain (200×116×1 Array{Float64, 3}):

```

Iterations      = 1:1:200
Number of chains = 1
Samples per chain = 200
parameters      = longit_measurements[1], longit_measurements[2], longit_measurements[3], longit_meas
internals       =

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_ta ...
Symbol	Float64	Float64	Float64	Float64	Float ...
longit_measurements[1]	30.3076	7.0092	0.7233	98.9700	160.12 ...

longit_measurements[2]	21.3506	4.3818	0.3341	175.3163	183.81 ...
longit_measurements[3]	16.9512	3.3652	0.2175	218.9222	187.34 ...
longit_measurements[4]	14.9884	2.7380	0.1787	233.2130	183.81 ...
longit_measurements[5]	13.6675	2.8624	0.2255	185.9004	176.75 ...
longit_measurements[6]	13.9603	2.8438	0.2095	174.8270	184.50 ...
longit_measurements[7]	14.8364	3.1372	0.2447	165.2310	215.58 ...
longit_measurements[8]	14.8172	2.9050	0.2407	150.6681	166.98 ...
longit_measurements[9]	15.5785	2.9446	0.2303	164.3407	116.57 ...
longit_measurements[10]	17.3831	4.7075	0.5081	84.2826	153.46 ...
longit_measurements[11]	18.1073	5.3525	0.5742	91.0287	152.81 ...
longit_measurements[12]	20.8240	7.5271	0.7730	79.5649	126.25 ...
longit_measurements[13]	23.1821	9.8574	1.2958	36.0890	94.24 ...
longit_measurements[14]	25.5495	13.2961	1.8476	32.4943	75.28 ...
longit_measurements[15]	29.2241	17.9964	2.2235	64.4496	104.96 ...
longit_measurements[16]	32.7139	24.3487	3.4350	46.9974	86.92 ...
surv_times[1]	1002.3351	893.9392	69.3481	163.9007	217.05 ...
surv_times[2]	1130.6568	954.0524	85.4290	162.7225	149.51 ...
surv_times[3]	1013.7271	861.8532	65.4106	180.4243	217.33 ...
surv_times[4]	1126.5871	999.6195	75.2704	143.0823	152.81 ...
surv_times[5]	1051.1978	929.2822	73.7828	195.6147	117.65 ...
surv_times[6]	1023.0215	856.1567	72.0437	119.8573	146.43 ...
surv_times[7]	909.3009	825.1592	67.0811	155.6549	172.36 ...
:	:	:	:	:	:

3 columns and 93 rows omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97 ...
Symbol	Float64	Float64	Float64	Float64	Floa ...
longit_measurements[1]	17.1211	25.5220	29.8919	34.9607	45.6 ...
longit_measurements[2]	13.6897	18.6784	20.9778	23.9963	29.5 ...
longit_measurements[3]	10.0216	14.6240	16.9937	19.2905	23.2 ...
longit_measurements[4]	9.6878	13.2613	14.8673	16.9262	20.6 ...
longit_measurements[5]	8.8308	11.5778	13.5128	15.7105	18.6 ...
longit_measurements[6]	8.1238	11.9311	14.2454	15.7923	18.6 ...
longit_measurements[7]	9.4894	12.7467	14.5047	16.7396	21.9 ...
longit_measurements[8]	9.5477	12.7652	14.6242	17.2219	19.9 ...
longit_measurements[9]	10.0886	13.6165	15.3303	17.2869	21.8 ...
longit_measurements[10]	10.1727	14.3584	16.6281	19.9043	29.4 ...
longit_measurements[11]	9.6677	14.2244	17.2781	21.3344	30.5 ...
longit_measurements[12]	10.7091	15.8796	19.1355	24.3511	40.6 ...
longit_measurements[13]	11.3359	16.8116	21.5832	26.7467	46.5 ...
longit_measurements[14]	11.3188	17.4420	22.5476	29.3827	61.7 ...

```

longit_measurements[15] 11.8786 18.5671 24.7555 32.7281 87.9 ...
longit_measurements[16] 10.6137 18.9503 25.9952 37.6588 102.5 ...
      surv_times[1] 20.1158 310.1280 718.3796 1461.8964 3296.6 ...
      surv_times[2] 34.3191 393.0075 845.1229 1644.4699 3161.2 ...
      surv_times[3] 56.2779 332.3351 804.6266 1418.6453 3144.2 ...
      surv_times[4] 47.4034 376.4369 825.2966 1568.3063 3604.8 ...
      surv_times[5] 37.1981 369.8628 729.4797 1655.7836 3249.0 ...
      surv_times[6] 43.1537 328.2921 838.5643 1523.2037 3404.8 ...
      surv_times[7] 11.7440 260.2487 655.2396 1339.9349 3050.4 ...
      : : : : : : \
                                1 column and 93 rows omitted

```

Plotting longitudinal PPC

```

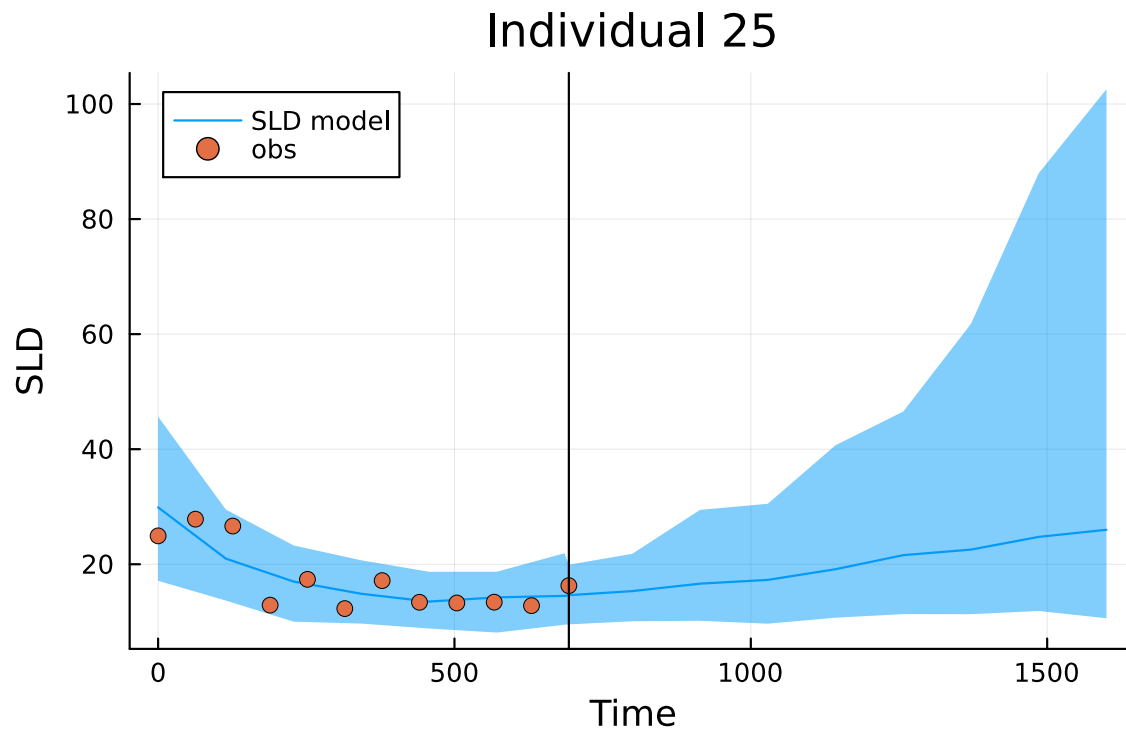
chn_longit = group(prediction_chn, :longit_measurements)
longit_quantiles = quantile(chn_longit)
quantiles_df = DataFrame(longit_quantiles)

lower_q = quantiles_df[:,2]
median_q = quantiles_df[:,4]
upper_q = quantiles_df[:,6]
lower_ribbon = median_q - lower_q
upper_ribbon = upper_q - median_q

plot(sld_timepoints, quantiles_df[:, 4],
      ribbon = (lower_ribbon, upper_ribbon),
      title = "Individual $individual",
      label = "SLD model",
      xlabel = "Time",
      ylabel = "SLD",
      legend = :topleft,
)

scatter!(individual_time, individual_obs, label = "obs")
vline!([last_obs], color = :black, linewidth = 1, label = false)

```



The line “SLD model” represents the median of the posterior predictions and the ribbon represents the 95% quantile.

Now we can add a conditional survival prediction into this graph starting at the last observation.

```
using Survival

"""
Functionality to work with non-parametric KaplanMeier estimators
"""
function npe(km_fit::KaplanMeier, t::Real)
    time_points = km_fit.events.time
    survival = km_fit.survival
    if t <= time_points[1]
        return 1
    elseif t >= time_points[end]
        return survival[end]
    else
        id = findfirst(x -> x >= t, time_points)
    end
end
```

```

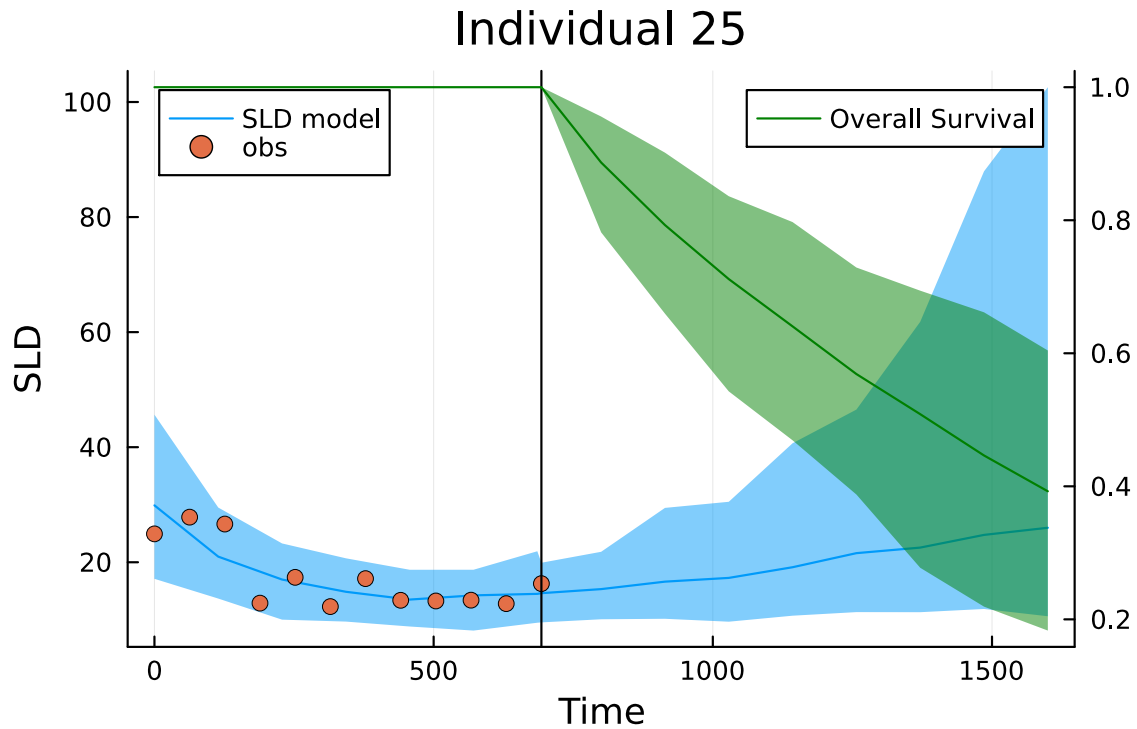
        return survival[id]
    end
end

chn_survival = group(prediction_chn, :surv_times)
indicators = ones(100)
surv_df = select!(DataFrame(chn_survival), Not(1:2))

conditioned_measurements = [row[row .>= last_obs] for row in eachrow(Matrix(surv_df))]
KM = [fit(KaplanMeier, row, indicators[1:length(row)]) for row in conditioned_measurements]
p = [0.025, 0.5, 0.975]
Q = [quantile([npe(km, time_point) for km in KM], p) for time_point in sld_timepoints]
Q_mat = reduce(hcat, Q)
surv_median = Q_mat[2,:]
surv_rib = (surv_median - Q_mat[1,:], Q_mat[3,:] - surv_median)

plot!(twinx(), sld_timepoints, surv_median,
    label = "Overall Survival",
    legend = :topright,
    color = :green,
    ribbon = surv_rib
)

```



The line “Overall Survival” represents the median posterior overall survival conditioned to survival until the last observation and the band represent the 95% quantile. This has been calculated non parametrically with generating 100 events for all posterior samples then dropping all event prior to the last overservation. For each posterior sample a Kaplan Meier was calculated and the median and 95% quantile come from their overall survival.

There is quite some work needed to generate this graph since we combine both the posterior predictive of the longitudinal and survival process. I want to highlight that since we are only using the posterior predictive this plot can be generated for arbitrary joint model formulation. Chaning the baseline hazard, link function or longitudinal model does not change anything in the generation of the plot itself.

Keroui, Marion, Francois Mercier, Julie Bertrand, Coralie Tardivon, René Bruno, Jérémie Guedj, and Solène Desmée. 2020. “Bayesian Inference Using Hamiltonian Monte-Carlo Algorithm for Nonlinear Joint Modeling in the Context of Cancer Immunotherapy.” *Statistics in Medicine* 39 (30): 4853–68. <https://doi.org/10.1002/sim.8756>.

A.5 Julia in Latex

Julia allows you to use many different unicode characters. This helps with readability and allows you to reduce errors when implementing mathematical formulas and ideas.

The problem when representing Julia code in Latex is that most mono-fonts never implemented these extended unicode characters. The solution I used in this document (and the quarto renderings) is to provide my own monofont via a folder in my Latex project. I used JuliaMono for this and downloaded the font from <https://juliamono.netlify.app/>.

When you have the JuliaMono folder in your project you can use it with the fontenc package.

```
\usepackage{fontspec}
\setmonofont{JuliaMono-Regular.ttf}[
  Path = ./JuliaMono/,
  Contextuals = Alternate,
  CharacterVariant=1,
  Scale = MatchLowercase,
  BoldFont = JuliaMono-Bold.ttf,
  %Numbers = OldStyle
]
```

For this to work you need to use LuaLaTeX or XeLaTeX to render the PDF.

Bibliography

- [1] J.J. Allaire, Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. Quarto, January 2022.
- [2] Alex Arslan. Survival.jl: Survival analysis in julia. <https://github.com/JuliaStats/Survival.jl>, 2018.
- [3] Mathieu Besançon, Theodore Papamarkou, David Anthoff, Alex Arslan, Simon Byrne, Dahua Lin, and John Pearson. Distributions.jl: Definition and modeling of probability distributions in the juliastats ecosystem. *Journal of Statistical Software*, 98(16):1–30, 2021.
- [4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, September 2017.
- [5] Carlos Parada <cdp49@cam.ac.uk>. Paretosmooth.jl, 6 2021.
- [6] David Collett. *Modelling Survival Data in Medical Research*. Chapman and Hall/CRC, May 2015.
- [7] D. R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society. Series B (Methodological)*, 34(2):187–220, 1972.
- [8] Milan Bouchet-Valat Dave Kleinschmidt. StatsModels.jl: Specifying, fitting, and evaluating statistical models in julia. <https://github.com/JuliaStats/StatsModels.jl>, 2017.
- [9] P. A. M. Dirac. *The Principles of Quantum Mechanics*. Clarendon Press, 1930.
- [10] H Ge, K Xu, and Z Ghahramani. Turing: A language for flexible probabilistic inference. 2018.

- [11] Craig Gower-Page, Georgios Kazantzidis, Daniel Sabanes Bove, Xiaoyan Fang, Francois Mercier, and Isaac Gravestock. *jmpost: Joint Models for Predicting Overall Survival Trajectories*, 2023. R package version 0.0.0.9000.
- [12] Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo, 2011.
- [13] Michael Innes. Don’t unroll adjoint: Differentiating ssa-form programs. *CoRR*, abs/1810.07951, 2018.
- [14] Michael Innes, Elliot Saba, Keno Fischer, Dhairya Gandhi, Marco Concetto Rudilosso, Neethu Mariya Joy, Tejan Karmali, Avik Pal, and Viral Shah. Fashionable modelling with flux. *CoRR*, abs/1811.01457, 2018.
- [15] Mohamed Tarek Jarrett Revels. ReverseDiff.jl: Reverse mode automatic differentiation for julia. <https://github.com/JuliaDiff/ReverseDiff.jl>, 2019.
- [16] Steven G. Johnson. QuadGK.jl: Gauss–Kronrod integration in Julia. <https://github.com/JuliaMath/QuadGK.jl>, 2013.
- [17] Marion Keroui, Francois Mercier, Julie Bertrand, Coralie Tardivon, René Bruno, Jérémie Guedj, and Solène Desmée. Bayesian inference using hamiltonian monte-carlo algorithm for nonlinear joint modeling in the context of cancer immunotherapy. *Statistics in Medicine*, 39(30):4853–4868, 2020.
- [18] Christopher Rackauckas and Qing Nie. Differentialequations.jl – a performant and feature-rich ecosystem for solving differential equations in julia. *The Journal of Open Research Software*, 5(1), 2017. Exported from <https://app.dimensions.ai> on 2019/05/05.
- [19] Dimitris Rizopoulos. JM: An R package for the joint modelling of longitudinal and time-to-event data. *Journal of Statistical Software*, 35(9):1–33, 2010.
- [20] Dimitris Rizopoulos. The R package JMBayes for fitting joint models for longitudinal and time-to-event data using mcmc. *Journal of Statistical Software*, 72(7):1–45, 2016.
- [21] Elisabeth Roesch, Joe G. Greener, Adam L. MacLean, Huda Nassar, Christopher Rackauckas, Timothy E. Holy, and Michael P. H. Stumpf. Author correction: Julia for biologists. *Nature Methods*, 20(5):771–771, April 2023.

- [22] Stan Development Team. Stan modeling language. <https://github.com/JuliaStats/StatsModels.jl>, 2012.
- [23] Stan Development Team. CmdStanR: the r interface to cmdstan. <https://mc-stan.org>, 2020.
- [24] Aki Vehtari, Andrew Gelman, and Jonah Gabry. Practical bayesian model evaluation using leave-one-out cross-validation and waic. *Statistics and Computing*, 27(5):1413–1432, Sep 2017.
- [25] John Verzani. Roots.jl: Root finding functions for julia. <https://github.com/JuliaMath/Roots.jl>, 2020.